SW-AI 处理器
从核结构手册

（V0.56 版本）

2019 年 11 月 08 日

不要传播，严格管理

前言

本手册描述了 SW-AI 处理器从核的数据类型、寄存器格式、寻址模式、指令格式、指令编码和指令功能，描述了 SW-AI 处理器从核的基本结构和单指令流多数据流（SIMD）扩展结构，描述了从核指令集中所有指令的定义，并对从核支持的浮点数据进行了详细说明。

本手册可用于 SW-AI 处理器从核的开发、SW-AI 处理器从核的基础软件开发和 SW-AI 处理器从核所组成处理器的开发。

本手册为秘密资料，未经授权，禁止复制。

目录

前言 I

简介 6

1.1 从核基本结构 6

1.1.1 数据格式概述 *6*

1.1.2 指令格式概述 *6*

1.1.3 指令概述 *7*

1.1.4 指令集特点 *8*

1.2 从核的扩展结构 8

1.3 SW-AI 从核的具体实现 8

1.4 术语及约定 9

2 数据类型 **10**

2.1 寻址单位 10

2.2 字节（BYTE） 10

2.3 半字（HALFWORD） 10

2.4 字（WORD） 10

2.5 长字（LONGWORD） 11

2.6 浮点数格式 11

2.7 向量 12

2.7.1 半字整数向量 *12*

2.7.2 字整数向量 *12*

2.7.3 HP 浮点向量 *12*

2.7.4 SP 浮点向量 *13*

3 寄存器和寻址模式 **14**

3.1 通用寄存器文件 14

3.2 向量寄存器文件 14

3.3 程序计数器 PC 15

3.4 四种寻址模式 15

3.5 采用小端地址格式 16

4 指令格式 **17**

4.1 指令的有关表示符号 17

4.1.1 操作数的表示符号 *17*

4.1.2 操作数访问类型表示符号 *18*

4.1.3 操作数的数据类型表示符号 *18*

4.1.4 操作符 *18*

4.2 指令格式 20

4.2.1 简单运算指令格式 *21*

4.2.2 复合运算指令格式 *21*

4.2.3 杂项指令格式 *22*

4.2.4 存储指令格式 *22*

4.2.5 转移指令格式 *22*

4.2.6 扩展指令集 23

5 浮点指令说明 24

5.1 浮点数据格式及指令 24

5.1.1 半精度运算 24

5.1.2 单精度运算 24

5.1.3 浮点实现与异常自陷 25

5.1.4 约定的说明 25

5.1.5 浮点数的表示范围 26

5.2 舍入模式 27

5.2.1 就近舍入 27

5.2.2 向"0"舍入 27

5.2.3 向正无穷大舍入 27

5.2.4 向负无穷大舍入 28

5.2.5 SW-AI 处理器的舍入实现 28

5.3 算术异常 28

5.3.1 无效操作（INV） 28

5.3.2 除数为"0"（DZE） 29

5.3.3 上溢（OVF） 29

5.3.4 下溢（UNF） 29

5.3.5 非精确结果（INE） 29

5.3.6 整数溢出（IOV） 30

5.4 非规格化数处理 30

5.5 浮点控制状态寄存器 FPCR 31

6 指令详细描述（199 条） 37

6.1 控制类指令（5 条） 37

6.1.1 停机指令 37

6.1.2 同步 38

6.1.3 存储器栏栅 38

6.2 转移类指令（14 条） 39

6.2.1 整数条件转移指令 39

6.2.2 浮点条件转移指令 40

6.2.3 无条件转移指令 41

6.2.4 寄存器间接转移指令 41

6.3 访存类指令（20 条） 41

6.3.1 对界访存指令（14 条） 42

6.3.2 不对界访存指令（2 条） 46

6.3.3 原子操作（4 条） 47

6.4 整数运算类指令（35 条） 50

6.4.1 算术运算指令（23 条） 50

6.4.2 移位指令（4 条） 57

6.4.3 整数比较指令（5 条） 58

6.4.4 整数条件选择指令（3 条） 59

6.5 标量整理指令（2 条） 60

6.5.1 标量拼接 60

6.5.2 标量字节插入 61
6.6 可重构指令（2 条） 61
6.6.1 两操作数可重构指令 log2x 62
6.6.2 三操作数可重构指令 log3r 63
6.7 单双精度浮点运算类指令（38 条） 64
6.7.1 算术运算指令（10 条） 64
6.7.2 比较指令（4 条） 66
6.7.3 转换指令（8 条） 67
6.7.4 符号拷贝指令（3 条） 70
6.7.5 浮点条件选择指令（3 条） 71
6.7.6 乘加指令（8 条） 72
6.7.7 读写浮点控制寄存器指令（2 条） 74
6.8 SIMD 扩展指令（70 条） 75
6.8.1 SIMD 访存指令（16 条） 75
6.8.2 整数 SIMD 运算指令（22 条） 85
6.8.3 SIMD 可重构指令（2 条） 92
6.8.4 浮点 SIMD 运算指令（19 条） 94
6.8.5 SIMD 整理指令（11 条） 102
6.9 通道类指令（4 条） 108
6.9.1 发起 DMA 109
6.9.2 发起 RMA 111
6.9.3 写通道 113
6.9.4 发起异步计算处理 113
6.10 专用指令（2 条） 116
6.10.1 并行查找表指令（2 条） 116
6.11 其它指令（2 条） 119
6.11.1 从核内部寄存器访问指令（2 条） 119
7 RCSR/WCSR 指令可访问的内部寄存器 121
7.1 CSR_CID（从核唯一 ID） 122
7.2 CSR_ROW（从核行号） 122
7.3 CSR_COL（从核列号） 122
7.4 CSR_GRP（从核核组号） 123
7.5 CSR_TC（周期计数器） 123
7.6 CSR_PCRC（性能计数控制寄存器） 123
7.7 CSR_LDM_CFG（LDM 配置） 124
7.8 CSR_GATER_CTL（门控时钟控制） 125
7.9 CSR_LRF_FLAG（寄存器临时变量标志位） 125
7.10 CSR_PCR0（性能计数器 0） 126
7.11 CSR_PCR1（性能计数器 1） 126
7.12 CSR_PCR2（性能计数器 2） 126
7.13 CSR_PCR3（性能计数器 3） 127
7.14 CSR_PCR4（性能计数器 4） 127
7.15 CSR_PCR5（性能计数器 5） 128
7.16 CSR_PCR6（性能计数器 6） 128
7.17 CSR_PCR7（性能计数器 7） 128

7.18 CSR_LDST_CREDIT（访存缓冲剩余信用计数）.....129
7.19 CSR_DMA_CREDIT（DMA/RMA 缓冲剩余信用计数）.....129
7.20 CSR_DMA_IDLE（DMA/RMA 空闲状态）.....129
7.21 CSR_LOG3R_TRUTH（LOG3R 真值表）.....130
7.22 CSR _USR_DEFINE0（用户自定义项 0）.....130
7.23 CSR _USR_DEFINE1（用户自定义项 1）.....130
7.24 CSR _USR_DEFINE2（用户自定义项 2）.....131
7.25 CSR _USR_DEFINE3（用户自定义项 3）.....131
8 附录 1 指令按操作码及操作类型排序列表.....132
9 附录 2 指令分操作码列表.....139
9.1 杂项指令-4'h0.....139
9.2 无条件转移指令 BR-4'h1.....140
9.3 条件转移指令-4'h2.....140
9.4 向量整理指令-4'h3.....141
9.5 标量整数指令-4'h4.....141
9.6 向量整数指令-4'h5.....142
9.7 LOG3R 指令-4'h6.....143
9.8 VLOG3R 指令-4'h7.....143
9.9 标量浮点指令-4'h8.....144
9.10 向量浮点指令、专用指令-4'h9.....145
9.11 小粒度访存指令、标/向量不对界访存指令-4'hA.....146
9.12 FLDS、FSTS、LDW、STW-4'hB.....147
9.13 LDL、STL -4'hC.....147
9.14 向量对界访存指令-4'hD.....147
9.15 LDI 指令-4'hE.....148
9.16 LDIH 指令-4'hF.....148
10 附录 3 指令数量统计.....148
11 附录 4 向量寄存器中的高位.....149
12 附录 5 浮点条件判断与浮点比较的区别.....149
13 附录 6 立即数说明.....150
14 附录 7 图表目录.....151
15 附录 8 修改记录.....153

简介

SW-AI 处理器是共享内存的主从结构的人工智能处理器，设计遵循面向需求、简洁高效，力求为应用提供超高性能。主核采用 64 位 RISC 结构通用处理单元，从核为一个 64 位 RISC 结构多用处理单元。本手册主要介绍从核指令集，有关主核指令集的详细说明请参照《申威-AI 主核结构手册》。

SW-AI 处理器的从核具有以下主要特点：

- 指令集简单，所有指令定长，均为 32 位；
- 使用一个 64×64 位的统一结构寄存器（SIMD 扩展结构使用高 32 个条目、512 位宽的向量寄存器），所有运算均在寄存器之间进行；
- 存储器访问指令除了离散的装入（Load）和存储（Store）外，还支持原子取并加的锁操作指令、原子更新指令和原子比较并置换指令；
- 采用简单的转移处理机制，条件转移指令使用 18 位偏移地址（指令粒度的偏移地址）进行转移预测，无条件转移指令使用 22 位偏移地址（指令粒度的偏移地址）提前转移，寄存器间接转移不支持转移预测；
- 支持三操作数指令，主要包括整数和浮点数的条件选择指令、浮点乘加指令、标/向量拼接指令和三操作数可重构逻辑运算指令；
- 支持通道消息机制，可以实现 DMA/RMA、写通道和异步计算等功能；
- 只支持小尾端地址格式（little-endian），不支持大尾端地址格式；

1.1 从核基本结构

1.1.1 数据格式概述

1). 64×64 位统一结构寄存器，由整数执行部件和浮点执行部件共享，具体认作定点整数格式还是浮点数格式由程序员自行决定；

2). 支持字节（8 位）、半字（16 位）、字（32 位）和长字（64 位）的整数；

3). 支持 IEEE754-2008 标准的半精度（16 位）、单精度（32 位）和双精度（64 位）三种浮点数据类型。

1.1.2 指令格式概述

从核指令格式如下图所示，指令类型根据主操作码 OPC[3:0]、功能码 FUNC[3:0]、子功能码 SUBF[3:0]，以及部分指令的 Disp 某些位来确定。Ra、Rb、Rc、Rd 为 6 位 GPR 索引；#b 为 8 位立即数；#c 为 6 位立即数；Disp 主要用作立即数偏移。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
简单运算（寄存器格式）	OPC				Ra						Rb						FUNC				SUBF				0	0	Rc					
简单运算（立即数格式）	OPC				Ra						#b[5:0]						FUNC				SUBF				#b[7:6]		Rc					

复合运算(寄存器格式)	OPC	Ra	Rb	FUNC	Rc	Rd
复合运算(立即数格式)	OPC	Ra	Rb	FUNC	#c[5:0]	Rd
杂项指令格式	OPC	Ra	Rb	FUNC	Disp[11:0]	
存储器指令格式	OPC	Ra	Rb	Disp[15:0]		
转移指令格式	OPC	Ra	Disp[21:0]			

图 0-1 从核指令格式

1）简单运算指令格式：主要用于整数操作类指令和浮点操作类指令；

2）复合运算指令格式：包含 4 个寄存器域的指令，主要完成浮点乘加、标/向量整理、整数标量寄存器条件选择、浮点标/向量寄存器条件选择、三操作数可重构指令；

3）杂项指令格式：主要用于杂项指令，部分指令会使用 Disp 的某些位区分不同的功能，包括写通道、访存栏栅、同步、寄存器间接转移、停机、CSR 访问、DMA/RMA、ACE、原子操作、比较并置换，以及小粒度访存指令；

4）存储器指令格式：主要用于访存指令，完成存储器和寄存器 Ra 之间的字、长字、半/单精度浮点的数据传送，使用 Rb 内容加上一个带符号的 16 位偏移值（disp）作为存储器地址

5）转移指令格式：主要用于转移指令，测试寄存器 Ra 的内容，并指定一个带符号、相对于 NPC 的 22 位长（部分指令为 18 位）目标偏移值（disp）。

1.1.3 指令概述

- 控制类指令：该类指令主要完成停机、存储器栏栅、同步支持等；
- 转移类指令：该类指令完成指令流的转移。条件转移指令测试寄存器内容是否满足大于零、小于零或非零等条件来确定是否转移，无条件转移指令总是转移且将返回地址写入寄存器中，间接转移指令则可跳转到寄存器所指示的任意地址（要求该目标地址与间接转移指令地址处于同一个 16MB 对界的空间内）处；
- 访存类指令：该类指令将 8、16、32 或 64 位对界的数据读出或写入存储器，以及进行原子取并加、原子更新和原子比较并置换操作；
- 通道指令：实现通道接口所支持的操作，包括批量数据 DMA、RMA 传输、异步计算加速处理、用户中断等；
- 整数运算类指令：主要按 64 位数据来处理，包括算术、比较和移位指令，也包括 32 位的算术指令，以及 32 位和 64 位乘法指令，没有整数除法指令。还支持一些特殊操作，包括带扩展因子的加法/减法指令，以实现快速的下标计算；条件选择指令，可避免使用转移指令；字节和字操作指令，用于对字节或字数据进行处理；
- 整理指令：该类指令根据地址低位将两个操作数的部分位取出进行拼接，或将一个操作数的最低字节插入到另一个操作数的某一个字节位置，形成一个新的 64 位长字；
- 可重构指令：该类指令可以实现两操作数和三操作数之间的任意逻辑运算；
- 浮点运算类指令：该类指令主要包括符合 IEEE-754 标准的浮点的简单运算和三操作数乘加运算；
- 专用指令：包括为支持特殊应用而实现的指令。

- 其它指令：主要包括读写从核 FPCR 指令和读从核内部寄存器指令。

1.1.4 指令集特点

- 所有指令均为 32 位等长，指令格式简单规整；
- 有一个 64×64 位宽度的统一结构寄存器（$0~$63），其中$31 为常数 0（整数 0，或者浮点正 0）寄存器，读时返回 0，写时被忽略；
- 所有数据操作均在寄存器之间进行，其中 1~3 个寄存器作为源操作数寄存器，1 个寄存器作为目标寄存器；
- 转移指令依据地址偏移的最高位和条件转移标记协同进行转移预测；
- 普通的整数和逻辑指令以 64 位为单位进行运算；
- 两操作数、三操作数可重构指令可以实现两个、三个操作数参与的任意逻辑运算，以获得更好的灵活性；
- 定制面向领域应用和特殊结构的指令，提高关键应用运行效率。

1.2 从核的扩展结构

为提高从核的功能和性能，对从核的基本结构进行单指令流多数据流（SIMD）扩展，形成从核的向量扩展结构。向量扩展包括数据类型的扩展、寄存器的扩展和指令集的扩展。

1）数据类型的扩展：支持短向量数据，包括长度为 16 的字（16×32 位）向量整数、长度为 32 的半字（32×16 位）向量整数和长度为 16 的单精度（16×32 位）向量浮点数据。

2）寄存器的扩展：将 32 个 64 位的统一类型寄存器（$32-$63）扩展成为 32 个 512 位的向量寄存器，每个寄存器的低 64 位仍然是基本结构的统一类型寄存器，存放 8 位、16 位、32 位和 64 位的整数，以及半精度、单精度或双精度浮点数据，而整个 512 位的寄存器则可以存放 512 位的整数向量或浮点向量。

3）指令集的扩展：对存储装入指令的扩展（称为存储装入扩展指令），实现向量寄存器与存储器间的数据传送。对整数运算指令的扩展（称为整数 SIMD 扩展指令），实现整数向量的运算。对浮点运算指令的扩展（称为浮点 SIMD 扩展指令），实现浮点向量的运算。扩展指令还包括对向量元素进行整理的指令。

1.3 SW-AI 从核的具体实现

SW-AI 处理器从核具体实现时，即可以只实现从核的基本结构，也可以实现包括扩展结构在内的从核完整结构。具体实现的指令集中，即可以包括整个指令集，也可以实现部分指令集，比如不实现浮点指令，比如只实现 16×32 的整数向量运算，具体实现的指令集与实现的功能相关。本文档中灰色部分，即为对应的 SW-AI 处理器实现版本中未实现的部分。

1.4 术语及约定

- 数字：所有数字若无指定，则均为十进制表示，前缀为“0x”的数字为十六进制表示；
- 必为零（MBZ）：标识为 MBZ 的域，软件保证不填入非“0”值，将来的扩展可能要使用该域。若发现 MBZ 域为非“0”值，则产生非法操作数异常；
- 读为零（RAZ）：当读标识为 RAZ 的域时，将返回“0”值；
- 应为零（SBZ）：标识为 SBZ 的域应由软件填入“0”值；
- 忽略（IGN）：对标识为 IGN 的域进行写时将被忽略；
- 实现相关（IMP）：标识为 IMP 的域与具体实现相关，每个实现必须对 IMP 域进行详细描述；
- 异常自陷：从核指令在处理过程中会产生与指令流同步的、由程序原因引起的异常问题。目前从核本身并不处理异常，而是通知核组，由核组中断主核的运行，查找问题，修正错误，重新提交执行。为了与其它处理器或核心，比如主核的描述一致，这里对异常处理的一些名称仍采用自陷，而主核中的自陷是发生异常时，核心自身的执行由用户程序陷入到系统特权程序中处理，SW-AI 处理器从核中的自陷只是指通知核组的意思，如有特别要求则会作特别的说明。

2 数据类型

2.1 寻址单位

SW-AI 处理器从核的基本寻址单位为 8 位的字节，采用有效地址（EA）访问，可以访问本地局存空间（LDM）和主存空间。SW-AI 处理器的存储管理只支持按小端地址格式（little-endian）来实现字节寻址，不支持大端格式（big-endian）。

以下各节描述的数据类型按小端地址格式（little-endian）来实现字节寻址。

2.2 字节（byte）

一个字节起始于一个可寻址字节边界上的连续 8 位。这 8 位从右到左依次编号为 0 到 7，如下图所示。一个字节为一个 8 位的值，一个地址可识别一个字节。SW-AI 处理器支持字节数据类型的指令有装入、存储、符号扩展和清除等 4 种。



图 0-2 字节的格式

2.3 半字（halfword）

一个半字起始于任意字节边界上相邻的 2 个字节（16 位）。这 16 位从右到左依次编号为 0 到 15，如下图所示。半字的地址为[0]位所在字节的地址。SW-AI 处理器支持半字数据类型的指令有装入、存储、符号扩展等 3 种。



图 0-3 半字的格式

2.4 字（word）

一个字为起始于任意字节边界上相邻的 4 个字节（32 位）。这 32 位从右到左依次编号为 0 到 31，如下图所示。字的地址为[0]位所在字节的地址。SW-AI 处理器支持字数据类型的指令有符号扩展的存储与装入指令以及算术运算指令。

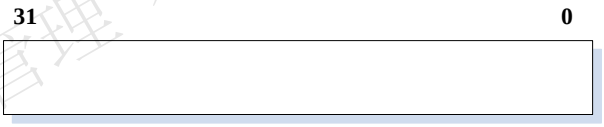


图 0-4 字的格式

2.5 长字（longword）

一个长字为起始于任意字节边界上相邻的 8 个字节（64 位）。这 64 位从右到左依次编号为 0 到 63，如下图所示。长字的地址为[0]位所在字节的地址。SW-AI 处理器支持长字数据类型的指令有存储与装入指令、算术运算指令和逻辑运算指令等。

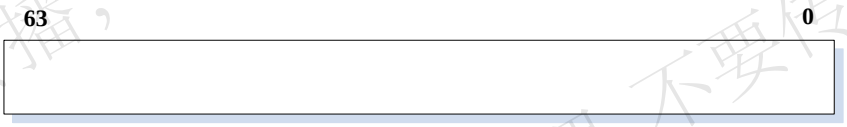


图 0-5 长字的格式

2.6 浮点数格式

ANSI/IEEE Std754-2008 标准（简称 IEEE754 标准）确定了二进制浮点操作的 IEEE 标准，它严格的定义了由一小类基本操作产生的结果，包括极端的情况，以确保编程者使用不同的机器对同样的输入获得同样的结果。

IEEE754 标准中，浮点数是将特定长度的连续字节的所有二进制位分割为特定宽度的符号域、指数域和尾数域三个域，其中保存的值分别用于表示给定二进制浮点数中的符号、指数和尾数。这样，通过尾数和可以调节的指数（所以称为“浮点”）以表达给定的数值。

根据应用的需求，目前从核的浮点指令集主要支持部分半精度型（HP）、单精度型（SP）和双精度型（DP）浮点操作。下图所示为符合 IEEE754 标准的 HP、SP 和 DP 浮点数格式：

IEEE 半精度浮点数

符号 Sign	指数 Exponent	尾数 Mantissa
1 bit	5 bits	10bits

IEEE 单精度浮点数

符号 Sign	指数 Exponent	尾数 Mantissa
1 bit	8 bits	23bits

IEEE 双精度浮点数

符号 Sign	指数 Exponent	尾数 Mantissa
1 bit	11 bits	52bits

图 0-6 IEEE 浮点数格式

格式中第一个域为符号域，0 表示数值为正数，1 表示负数；第二个域为指数域，其中半精度数为 5 位，单精度数为 8 位，双精度数为 11 位。由于指数可以为正数，也可以为负数，为此实际的指数值按要求需要加上一个偏差（Bias）值作为保存在指数域中的值，半精度数的偏差值为 15，单精度数的偏差值为 127，而双精度数的偏差值为 1023。指数域全 0 或全 1 用于表示特殊数据，半精度数实际可表达的指数值范围为[-14，15]，单精度数实际可表达的指数值范围为[-126，127]，双精度数则为[-1022，1023]；第三个域为尾数域，其中半精度数为 10 位，单精度数为 23 位，双精度数为 52 位。除了某些特殊值，IEEE 标准要求浮点数必须是规范的：尾数的小数点左侧必须为 1。实际表示时小数点左侧的 1 省略，以腾出一个二进制位来保存更多的尾数。

2.7 向量

从核的扩展结构支持向量数据，包括长度为 32 的半字整数向量（32×16 位）、长度为 16 的字整数向量（16×32 位）、长度为 16 的单精度浮点向量和长度为 16 的半精度浮点向量。

2.7.1 半字整数向量

半字整数向量在存储器中占用起始于任意字节边界的 32 个半字（512 位），这 512 位从右到左依次编号为 0 到 511。半字整数向量的地址为[0]位所在字节的地址，格式如下图所示。SW-AI 处理器支持半字整数向量的指令主要有存储器读、逻辑运算及算术运算等。



图 0-7 半字整数向量格式

2.7.2 字整数向量

字整数向量在存储器中占用起始于任意字节边界的 16 个字（512 位），这 512 位从右到左依次编号为 0 到 511。字整数向量的地址为[0]位所在字节的地址，格式如下图所示。SW-AI 处理器支持字整数向量的指令主要有存储器读、逻辑运算及算术运算等。



图 0-8 字整数向量格式

2.7.3 HP 浮点向量

HP 浮点向量即半精度浮点向量，在存储器中每个 HP 浮点向量元素占用 2 个字节，HP 浮点向量占用起始于任意字节边界的相邻 32 个字节（256 位），这 256 位从右到左依次编号为 0 到 255，对应 16 个 HP 浮点向量元素的格式如下图所示。HP 浮点向量的地址为[0]位所在字节的地址。HP 浮点向量中每个元素的定义与 HP 浮点的定义完全相同。

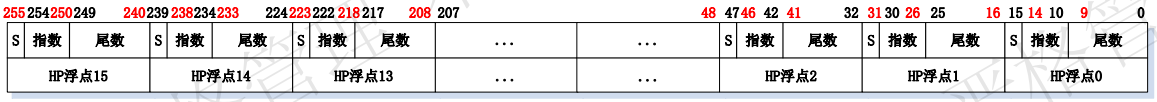


图 0-9 HP 浮点向量的存储器格式

HP 浮点向量在向量寄存器中以 512 位表示，格式如下图所示：

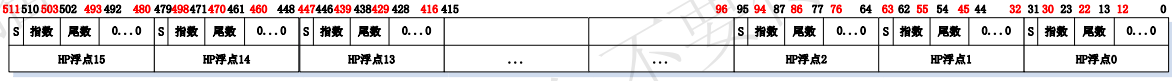


图 0-10 HP 浮点向量的寄存器格式

2.7.4 SP 浮点向量

SP 浮点向量即单精度浮点向量，在存储器中每个 SP 浮点向量元素占用 4 个字节，SP 浮点向量占用起始于任意字节边界的相邻 64 个字节（512 位），这 512 位从右到左依次编号为 0 到 511，对应 16 个 SP 浮点向量元素的格式如下图所示。SP 浮点向量的地址为[0]位所在字节的地址。SP 浮点向量中每个元素的定义与 SP 浮点的定义完全相同。

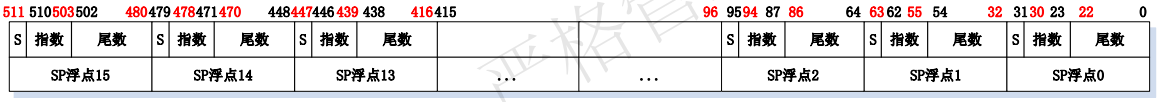


图 0-11 SP 浮点向量的存储器格式

SP 浮点向量在向量寄存器中以 512 位表示，寄存器格式与存储器格式完全相同。

3 寄存器和寻址模式

3.1 通用寄存器文件

SW-AI 处理器从核共设置有 64 个通用寄存器，分别表示为$0~$63（也可以表示为 R0~R63 或者 r0~r63），每个寄存器宽度均为 64 位，由整数运算和浮点运算指令共享。其中$31 为常数 0（整数 0，或者浮点正 0）寄存器，读时返回 0，写时被忽略。SW-AI 从核同时实现两个通用寄存器文件：64 条目*64 位的标量 GPR、32 条目*448 位的向量扩展 GPR。向量扩展 GPR 的 32 个条目，固定一一对应标量 GPR 的 32~63 条目。



图 0-12 通用寄存器文件结构

两个 GPR 统一编址，分别表示为$0~$63（也可以表示为 R0~R63，或者 r0~r63）。标量指令的源/目标操作数寄存器、向量指令中的标量操作数寄存器，可以使用$0~$63；向量指令中的向量操作数寄存器，只能访问$31~$63。其中$31 作为标量时固定为 64'h0（整数 0 或者浮点正 0）寄存器，作为向量时固定为 512'h0，读时返回 0，写时被忽略。

对于无条件转移指令（BR）和寄存器间接转移指令（JMP），当把$31 作为目标寄存器时，程序能正常运行且能用正确的目标地址更新 PC 值，但是 NPC 值不能被$31 保存。

3.2 向量寄存器文件

从核的扩展结构设置有 32 个 512 位向量结构寄存器，其低 64 位也作为基本结构的统一类型寄存器，向量结构寄存器分别表示为 V0~V31，每个寄存器的宽度为 512 位。向量寄存器可以存放三十二个元素的半字整数向量、十六个元素的字整数向量、十六个元素的 SP 浮点向量或十六个元素的 HP 浮点向量。向量寄存器 Vi 包括 32 个半字整数，16 个字整数，或 16 个单精度或半精度浮点。

表示为半字整数向量时，32 个半字元素分别标记为 ViH0~ViH31，每个元素为 16 位，如下图所示：

127 112	111 96	95 80	79 64	63 48	47 32	31 16	15 0
元素7 ViH7	元素6 ViH6	元素5 ViH5	元素4 ViH4	元素3 ViH3	元素2 ViH2	元素1 ViH1	元素0 ViH0
255 240	239 224	223 208	207 192	191 176	175 160	159 144	143 128
元素15 ViH15	元素14 ViH14	元素13 ViH13	元素12 ViH12	元素11 ViH11	元素10 ViH10	元素9 ViH9	元素8 ViH8
383 368	367 352	351 336	335 320	319 304	303 288	287 272	271 256
元素23 ViH23	元素22 ViH22	元素21 ViH21	元素20 ViH20	元素19 ViH19	元素18 ViH18	元素17 ViH17	元素16 ViH16
511 496	495 480	479 464	463 448	447 432	431 416	415 400	399 384
元素31 ViH31	元素30 ViH30	元素29 ViH29	元素28 ViH28	元素27 ViH27	元素26 ViH26	元素25 ViH25	元素24 ViH24

图 0-13 向量寄存器 Vi 表示半字整数向量格式

表示为字整数向量时，16 个字元素分别标记为 ViE0~ViE15，每个元素为 32 位，如下图所示：



图 0-14 向量寄存器 Vi 表示字整数向量格式

表示为浮点向量时，16 个浮点元素分别标记为 ViF0~ViF15，每个元素为 32 位，如下图所示：



图 0-15 向量寄存器 Vi 表示 SP 浮点向量或 HP 浮点向量格式

3.3 程序计数器 PC

程序计数器（PC）：PC 是一个特殊寄存器，用于对指令流进行寻址，一条指令被译码后，PC 将指向下一条指令（称之为已修改的 PC，记为 NPC，即 Next PC）。指令中使用的 PC 值都是已修改的 PC 值，比如转移指令使用的偏移量就是已修改 PC（NPC）之上的相对偏移。具体实现时，SW-AI 处理器从核内使用的 PC 值称为全局 PC（GPC），48 位，指示主存用户区程序段的 PC 值，该 PC 值软件可见，也用于从核脱靶取指

全局 PC 可以通过无条件转移指令（BR）和寄存器间接转移指令（JMP）返回给目标寄存器 Ra 来获得。

3.4 四种寻址模式

从核支持四种寻址模式：

- 寄存器寻址：寄存器寻址就是把操作数保存在寄存器中。由于从核采用 RISC 结构，所有运算均在寄存器之间进行，几乎所有指令都采用寄存器寻址方式，如：

```
addl   $1,$2,$5          //$5←$1+$2
subl   $3,$4,$4          //$4←$3–$4
mulw   $31,$31,$31   //$31←$31*$31
```

- 立即数寻址：立即数寻址就是指令中直接包含源操作数，如加法、减法、乘法、比较等指令，当其中一个源操作数为立即数时，即为立即数寻址方式，如：

```
addl   $1,8,$2       //$2←$1+8
subl   $2,1,$3       //$3←$2–1
mulw   $2,1,$3       //$3←$2*1
```

- 偏移寻址：偏移寻址方式就是操作数的有效地址由指令中的寄存器内容和包含在指令中的偏

移量 disp 求和而形成。如：LDxRa,disp(Rb)；//Ra←LDM[disp+(Rb)]。对于偏移寻址方式，若偏移量 disp 为“0”，则类似于寄存器间接寻址；若寄存器的内容为“0”，则类似于立即数寻址方式，如：

```
ldbu   $1,4($31)      //$1←mem[4+($31)]
stl    $2,8($31)      //mem[8+($31)]←$2
```

- 相对 PC 寻址：相对 PC 寻址就是已修改的 PC 值（NPC）加上指令中包含的偏移量而形成新的访问地址。从核的条件转移和无条件转移指令即采用相对 PC 的寻址方式，如：

```
beq    $1,disp     //若$1 满足条件，则目标 PC←NPC+4*disp
br     $2,disp     //目标 PC←NPC+4*disp; $2←NPC
```

3.5 采用小端地址格式

地址格式主要是指字节数据在更长宽度数据类型中的次序。从核采用小端地址的寻址格式，这时长字内的字节从右至左依次编号，如下图所示。

7	6	5	4	3	2	1	0

图 0-16 小端地址寻址格式

4 指令格式

4.1 指令的有关表示符号

4.1.1 操作数的表示符号

下表列示寄存器操作数、寄存器操作数的数值以及其它操作数的表示符号和含义。

表 0-1 寄存器操作数表示符号

序号	符号	含义
1	Ra	指令 Ra 域中的通用寄存器。在汇编代码中可以使用以“$”或者“r”或者“R”加数字代表通用寄存器
2	Rb	指令 Rb 域中的通用寄存器。在汇编代码中可以使用以“$”或者“r”或者“R”加数字代表通用寄存器
3	Rc	指令 Rc 域中的通用寄存器。在汇编代码中可以使用以“$”或者“r”或者“R”加数字代表通用寄存器
4	Rd	指令 Rd 域中的通用寄存器。在汇编代码中可以使用“$”或者“r”或者“R”加数字代表通用寄存器
5	Va	指令 Va 域中的向量寄存器。在汇编代码中可以使用“$”或者“v”或者“V”加数字代表通用寄存器
6	Vb	指令 Vb 域中的向量寄存器。在汇编代码中可以使用“$”或者“v”或者“V”加数字代表通用寄存器
7	Vc	指令 Vc 域中的向量寄存器。在汇编代码中可以使用“$”或者“v”或者“V”加数字代表通用寄存器
8	Vd	指令 Vd 域中的向量寄存器。在汇编代码中可以使用“$”或者“v”或者“V”加数字代表通用寄存器
9	#b #c	指令中的整数立即数，可以采用 16 进制数（以 0x 或 0X 起始），2 进制数（以 0b 或 0B 起始），8 进制数（以 0 起始）或者 10 进制数
10	disp	指令中的偏移值，可以采用 16 进制数（以 0x 或 0X 起始），2 进制数（以 0b 或 0B 起始），8 进制数（以 0 起始）或者 10 进制数
11	idx	指令中的内部寄存器索引号或真值表索引号，可以采用 16 进制数（以 0x 或 0X 起始），2 进制数（以 0b 或 0B 起始），8 进制数（以 0 起始）或者 10 进制数
12	Rav	Ra 操作数的数值，即寄存器 Ra 中的内容
13	Rbv	Rb 操作数的数值，即寄存器 Rb 中的内容

14	Rcv	Rc 操作数的数值，即寄存器 Rc 中的内容
15	Vav	Va 操作数的数值，即向量寄存器 Va 中的内容
16	Vbv	Vb 操作数的数值，即向量寄存器 Vb 中的内容
17	Vcv	Vc 操作数的数值，即向量寄存器 Vc 中的内容
18	Vdv	Vd 操作数的数值，即向量寄存器 Vd 中的内容
19	NPC	指向当前处理指令的下一条指令地址

4.1.2 操作数访问类型表示符号

下表列示了操作数访问类型的表示符号。

表 0-2 操作数访问类型表示符号

序号	符号	含义
1	a	操作数用来在地址计算中形成有效的地址，其后的数据类型指示寻址的单位
2	i	操作数为立即数
3	r	操作数为只读
4	m	操作数为可读写
5	w	操作数为只写

4.1.3 操作数的数据类型表示符号

下表列示了操作数的数据类型表示符号。

表 0-3 操作数数据类型表示符号

序号	符号	含义
1	b	字节
2	h	半字或 IEEE 的半精度浮点（HP 浮点）
3	w	字或字向量
4	l	长字
5	o	八倍字
6	x	十六倍字
7	z	由具体指令指定的数据类型
8	s	IEEE 的单精度浮点（SP 浮点）
9	d	IEEE 的双精度浮点（DP 浮点）

4.1.4 操作符

下表是指令格式说明中用到的操作符一览表。

表 0-4 操作符一览表

<table>
<tr><th>序号</th><th>符号</th><th>含义</th></tr>
<tr><td>1</td><td>//</td><td>行注释的开始</td></tr>
<tr><td>2</td><td>＋</td><td>加</td></tr>
<tr><td>3</td><td>－</td><td>减</td></tr>
<tr><td>4</td><td>×</td><td>乘</td></tr>
<tr><td>5</td><td>←</td><td>替换</td></tr>
<tr><td>6</td><td>||</td><td>位拼接</td></tr>
<tr><td>7</td><td>{}</td><td>显式指示操作符优先级</td></tr>
<tr><td>8</td><td>(x)</td><td>地址为 x 的存储器单元内容</td></tr>
<tr><td>9</td><td>x<m:n></td><td>x 的[m:n]位域内容</td></tr>
<tr><td>10</td><td>x<m></td><td>x 的第 m 位</td></tr>
<tr><td>11</td><td>AND</td><td>逻辑与</td></tr>
<tr><td>12</td><td>ARITH_RIGHT_SHIFT(x,y)</td><td>x 算术右移 y 位，将最高符号位移进空出位置，移出位丢弃</td></tr>
<tr><td>13</td><td>BYTE_ZAP(x,y)</td><td>x 为长字，y 为一个字节，y 的第一位对应 x 的一个字节，对应关系为 y<n>对应 x<8n+7:8n>。将 x 中对应 y 为“1”的字节清为零，其它字节保持不变</td></tr>
<tr><td>14</td><td>CASE</td><td>根据参数的值从几个行为中选择一个，且仅选择一个</td></tr>
<tr><td>15</td><td>lab:</td><td>程序设计中可以使用标号（label），标号由标识符紧随“:”构成，用来定义函数的入口点、间接分支的目的地或者数据的存储位置</td></tr>
<tr><td>16</td><td>LEFT_ROTATE(x,y)</td><td>将 x 循环左移 y 位，将移出位移进空出位置</td></tr>
<tr><td>17</td><td>LEFT_SHIFT(x,y)</td><td>将 x 逻辑左移 y 位，将“0”移进空出位置，移出位丢弃</td></tr>
<tr><td>18</td><td>MAP_H2D</td><td>将 HP 浮点从存储器装入到寄存器时，进行格式转换的指数映射功能。存储器格式的 5 位指数扩展为寄存器格式的 11 位指数的映射关系如下所示（表中数字为二进制）
<table>
<tr><th>存储器[14:10]位</th><th>寄存 [30:23]位</th></tr>
<tr><td>1 1111</td><td>1 111111 1111</td></tr>
<tr><td>1 xxxx</td><td>1 000000 xxxx(xxxx 不全为“1”)</td></tr>
<tr><td>0 xxxx</td><td>0 111111 xxxx(xxxx 不全为“0”)</td></tr>
<tr><td>0 0000</td><td>0 000000 0000</td></tr>
</table></td></tr>
<tr><td>19</td><td>MAP_H2S</td><td>将 HP 浮点从存储器装入到寄存器时，进行格式转换的指数映射功能。存储器格式的 5 位指数扩展为寄存器格式的 8 位指数的映射关系如下所示（表中数字为二进制）
<table>
<tr><th>存储器[14:10]位</th><th>寄存 [30:23]位</th></tr>
<tr><td>1 1111</td><td>1 111 11 1</td></tr>
</table></td></tr>
</table>

<table>
<tr><td></td><td></td><td>
<table>
<tr><td>1 xxxx</td><td>1 000 xxxx(xxxx 不全为“1”)</td></tr>
<tr><td>0 xxxx</td><td>0 111 xxxx(xxxx 不全为“0”)</td></tr>
<tr><td>0 0000</td><td>0 000 0000</td></tr>
</table>
</td></tr>
<tr><td>20</td><td>MAP_S2D</td><td>将 SP 浮点从存储器装入到寄存器时，进行格式转换的指数映射功能。存储器格式的 8 位指数扩展为寄存器格式的 11 位指数的映射关系如下所示（表中数字为二进制）
<table>
<tr><th>存储器[14:10]位</th><th>寄存 [30:23]位</th></tr>
<tr><td>1 1111111</td><td>1 111 1111111</td></tr>
<tr><td>1 xxxxxxx</td><td>1 000 xxxxxxx(xxxxxxx 不全为“1”)</td></tr>
<tr><td>0 xxxxxxx</td><td>0 111 xxxxxxx(xxxxxxx 不全为“0”)</td></tr>
<tr><td>0 0000000</td><td>0 000 0000000</td></tr>
</table>
</td></tr>
<tr><td>21</td><td>NOT</td><td>逻辑非</td></tr>
<tr><td>22</td><td>OR</td><td>逻辑或</td></tr>
<tr><td>23</td><td>RIGHT_SHIFT(x,y)</td><td>将 x 逻辑右移 y 位，将“0”移进空出位置，移出位丢弃</td></tr>
<tr><td>24</td><td>SEXT(x)</td><td>将 x 符号扩展为需要的位数</td></tr>
<tr><td>25</td><td>TEST(x,cond)</td><td>对寄存器 x 内容按 cond 条件进行测试，以确定转移条件是否为真。若指定关系成立，则 TEST 返回布尔值真，否则返回假</td></tr>
<tr><td>26</td><td>TRUTH</td><td>可重构指令中使用的真值组合</td></tr>
<tr><td>27</td><td>TRUTH_MAP(x,y,z,truth)</td><td>用(x,y,z)构成的位组映射三元真值组合 truth，返回逻辑运算结果。其中 truth 为 8 位的二进制（记为 truth[7:0]），则该映射关系为：
<table>
<tr><th>x</th><th>y</th><th>z</th><th>TRUTH_MAP(x,y,z,truth)</th></tr>
<tr><td>0</td><td>0</td><td>0</td><td>truth[0]</td></tr>
<tr><td>0</td><td>0</td><td>1</td><td>truth 1]</td></tr>
<tr><td>0</td><td></td><td>0</td><td>t uth[2]</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>truth[3]</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>truth[4]</td></tr>
<tr><td>1</td><td>0</td><td>1</td><td>truth[5]</td></tr>
<tr><td>1</td><td>1</td><td>0</td><td>truth[ ]</td></tr>
<tr><td>1</td><td>1</td><td>1</td><td>truth[7]</td></tr>
</table>
</td></tr>
<tr><td>28</td><td>XOR</td><td>逻辑异或</td></tr>
<tr><td>29</td><td>ZEXT(x)</td><td>将 x 零扩展为需要的位数</td></tr>
</table>

4.2 指令格式

从核指令格式如下图所示，指令类型根据主操作码 OPC[3:0]、功能码 FUNC[3:0]、子功能码 SUBF[3:0]，以及部分指令的 Disp 某些位来确定。Ra、Rb、Rc、Rd 为 6 位 GPR 索引；#b 为 8 位立即

数；#c 为 6 位立即数；Disp 主要用作立即数偏移。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
简单运算（寄存器格式）	OPC				Ra						Rb						FUNC				SUBF				0	0	Rc					
简单运算（立即数格式）	OPC				Ra						#b[5:0]						FUNC				SUBF				#b[7:6]		Rc					
复合运算（寄存器格式）	OPC				Ra						Rb						FUNC				Rc						Rd					
复合运算（立即数格式）	OPC				Ra						Rb						FUNC				#c[5:0]						Rd					
杂项指令格式	OPC				Ra						Rb						FUNC				Disp[11:0]											
存储器指令格式	OPC				Ra						Rb						Disp[15:0]															
转移指令格式	OPC				Ra						Disp[21:0]																					

图 0-17 SW-AI 处理器从核指令格式

4.2.1 简单运算指令格式

简单运算指令格式用于两个源操作数的整数、浮点以及 SIMD 运算指令，由 4 位功能码 FUNC 和 4 位子功能码 SUBF 确定具体指令功能。整数运算指令和整数 SIMD 运算指令有 8 位的立即数格式。注意寄存器格式下指令[7:6]位总是全“0”（称为 SBZ 域），具体格式如下图所示，Ra 与 Rb 或 8 位立即数运算，结果保存到 Rc。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
简单运算（寄存器格式）	OPC				Ra						Rb						FUNC				SUBF				0	0	Rc					
简单运算（立即数格式）	OPC				Ra						#b[5:0]						FUNC				SUBF				#b[7:6]		Rc					

图 0-18 简单运算指令格式

1）寄存器格式：

Rb 为寄存器域时，Ra 与 Rb 运算，结果存入 Rc。4 位功能码 FUNC 和 4 位子功能码 SUBF 作为操作码的扩展，表示具体的操作类型。

2）立即数格式：

Rb 域和 SBZ 域的 2 位一起作为 8 位的立即数#b 时，将立即数“0”扩展后与 Ra 进行运算，结果存入 Rc。4 位功能码 FUNC 和 4 位子功能码 SUBF 作为操作码的扩展，表示具体的操作类型。

3）整数 SIMD 扩展指令：

从核扩展结构中的整数 SIMD 扩展指令也使用简单运算指令格式。指令为立即数格式时，8 位立即数当作移位操作的移位量（字整数使用最低 5 位，长字整数使用最低 6 位），也当作操作数参与算术和逻辑运算，作法是将 8 位立即数零扩展为字整数，并复制为相同元素的 16 字或 8 长字整数向量，再参与运算。

4.2.2 复合运算指令格式

复合运算指令包括 4 寄存器指令格式和 3 寄存器立即数格式两种，完成整数和浮点的条件选择，数据整理、浮点的乘法和加法的复合操作，具体包括整数条件选择指令、标/向量整理指令、可重构指令、浮点条件选择指令和浮点乘加操作，其中 Ra、Rb、Rc 为 3 个源操作数，Rd 为运算结果。另外，大部分专用指令也使用本格式。

从核扩展结构中的浮点 SIMD 扩展指令也使用浮点复合运算指令格式，其中的寄存器域 Ra、Rb、Rc（或#c）和 Rd 分别指示向量寄存器 Va、Vb、Vc（或#c）和 Vd。

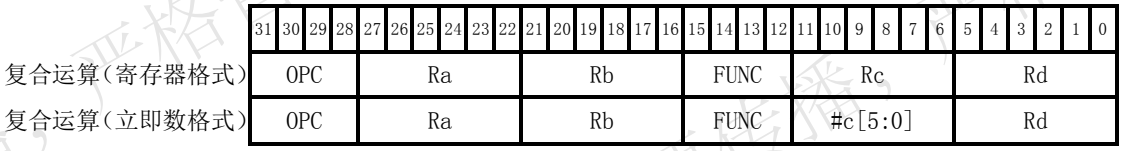


图 0-19 浮点复合运算指令格式

4.2.3 杂项指令格式

杂项指令格式主要用于杂项指令，部分指令会使用 Disp 的某些位区分不同的功能，包括分类、写通道、访存栏栅、同步、间接转移、停机、CSR 访问、DMA/RMA、ACE、原子操作、比较并置换，以及小粒度访存指令。指令格式如下图所示：

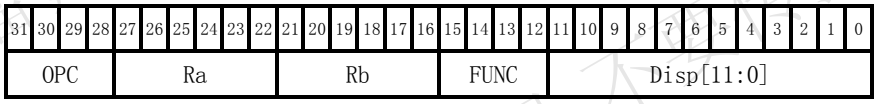


图 0-20 杂项指令格式

4.2.4 存储指令格式

存储指令格式用来定义寄存器与存储器间的数据移动（存储和装入）、地址装载、原子操作、转移等指令。指令格式如下图所示：

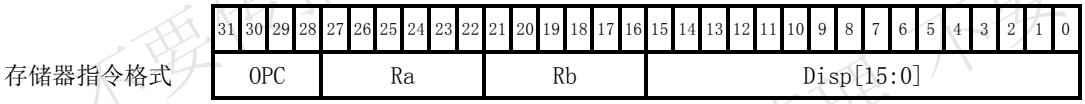


图 0-21 存储器指令格式

1）非小粒度对界存储装入指令

Ra 为读结果或写数据，Rb 加上带符号的 16 位地址偏移 disp 形成存储器有效地址 Vaddr，在 Ra 和 Vaddr 对应的存储器单元之间进行数据移动，数据位宽为 32 位或 64 位。字粒度以上访存的偏移地址，根据对界情况抹去低位，保留[15:x]地址的计算如下：

Vaddr←Rbv＋SEXT（disp[15:x]）

FLDS、FSTS、LDW、STW 的 Disp[1:0]位置用作功能码 FUNC[1:0]。LDL、STL、向量对界访存指，Disp[2:0]位置用作功能码 FUNC[2:0]。

2）地址装载指令

包括 LDI 和 LDIH 共 2 条指令。功能是将 Rbv 加上偏移 disp 形成地址并赋给 Ra。

4.2.5 转移指令格式

转移指令格式用来定义相对于已修改 PC（NPC）转移的指令，它包括条件转移指令、无条件转移指令。因为相对 NPC 的偏移量在指令中直接可见，又称为直接转移。由于偏移量的直观性，便于预测转移目标地址。指令格式如下图所示：

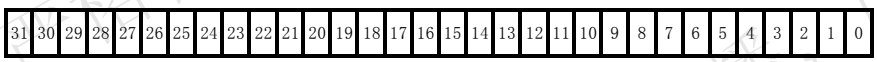




图 0-22 转移指令格式

1）条件转移指令

其中 Disp[21:18]位置用作功能码 FUNC[3:0]，Disp[17:0]为立即数字节偏移地址的[19:2]位。测试寄存器 Ra，若条件满足则跳转到指定的带符号的 20 位地址偏移处（相对于已修改的 PC 值）运行。跳转范围为 PC 前/后 512KB（共 1MB）的区域以内（20 位地址偏移中，最高 1 位为符号位，同时也参与转移预测）。目标地址的计算方法如下：

Vaddr←NPC＋4×SEXT（disp）

2）无条件转移指令

包含 BR。转移的目标地址的计算方法带符号的 22 位地址偏移处（相对于已修改的 PC 值）运行。跳转范围为 PC 前/后 2MB（共 4MB）的区域以内（22 位地址偏移中，最高 1 位为符号位，同时也参与转移预测）。目标地址的计算方法如下：

Vaddr←PC＋4×SEXT（disp）

BR 指令在转移之前须将返回地址保存到 Ra 中。

4.2.6 扩展指令集

从核扩展结构对指令集进行了扩展，具体包含四个方面：

a. 存储装入扩展指令：采用存储器指令格式；

b. 整数 SIMD 扩展指令：采用简单运算指令格式；

c. 浮点 SIMD 扩展指令：包括对浮点运算指令的扩展和对浮点乘加指令的扩展，分别采用简单运算指令格式和浮点复合运算指令格式；

d. SIMD 整理指令：采用立即数格式的浮点复合运算指令格式。

5 浮点指令说明

5.1 浮点数据格式及指令

SW-AI 处理器使用的浮点数据格式遵循 IEEE-754-2008 标准，包括基本半精度（16 位，即 HP 浮点）格式、基本单精度（32 位，即 SP 浮点）格式、基本双精度（64 位，即 DP 浮点）格式，以及扩展精度格式。浮点数据保存在 SW-AI 处理器的存储器中时，采用 IEEE-754 标准的基本半精度、基本单精度和基本双精度三种浮点数据格式；保存在通用寄存器中时，采用 32 位扩展半精度（32 位，称 EHP32）浮点、64 位扩展半精度（64 位，称 EHP64）浮点、基本单精度（32 位）浮点、64 位扩展单精度（64 位，称 ESP64）浮点和基本双精度（64 位）浮点格式。这些浮点格式在存储器和寄存器中的存储格式如下表所示：

表 0-5 各种浮点格式在存储器和寄存器中的存储格式

浮点类型	存储器	寄存器	
		标量 64 位	向量 32 位
双精度浮点	基本双精度（64 位，即 DP 浮点）格式	基本双精度（64 位，即 DP 浮点）格式	--
单精度浮点	基本单精度（32 位，即 SP 浮点）格式	64 位扩展单精度（64 位，称 ESP64）格式	基本单精度（32 位，即 SP 浮点）格式
半精度浮点	基本半精度（16 位，即 HP 浮点）格式	64 位扩展半精度（64 位，称 EHP64）格式	32 位扩展半精度(32 位，称 EHP32)

SW-AI 处理器从核指令集提供的浮点指令包括存储装入、条件转移、条件选择和运算等。其中浮点运算指令包括数据格式转换指令、算术运算指令等。SW-AI 处理器扩展结构还支持浮点 SIMD 运算指令，可实现 16 个浮点数据的并行运算。

5.1.1 半精度运算

半精度数值以扩展的形式保存在通用寄存器中，目前使用 2 种扩展方式：32 位扩展半精度（32 位，称 EHP32）浮点和 64 位扩展半精度（64 位，称 EHP64）浮点。对于规格化数浮点数、正负 0、正负无穷大、非数等，它们可看作是单精度或双精度数值的一个子集，即对于 32 位扩展半精度，5 位指数被限制在相应的单精度范围之内，尾数的最低 13 位全为“0”；对于 64 位扩展半精度，5 位指数被限制在相应的双精度范围之内，尾数的最低 42 位全为“0”；对于非规格化半精度浮点数，其对应阶码偏移值为 15。由基本格式半精度经由 MAP_H2S 或 MAP_H2D 转换得到的扩展格式半精度称为规范的半精度；否则，称之为非规范的半精度。对规范的半精度数值进行单精度运算或双精度运算将产生单精度或双精度的结果，对非规范的半精度数值进行单精度运算将产生不可预测的结果。

5.1.2 单精度运算

单精度数值保存在通用寄存器有 2 种方式，当以标量方式从存储器装入通用寄存器时，以 64 位扩展单精度（64 位，称 ESP64）浮点的形式保存；当以向量方式从存储器装入通用寄存器时，以标准单

精度（32 位）的形式保存。对于 64 位扩展单精度，规格化数浮点数、正负 0、正负无穷大、非数等，它们可看作是双精度数值的一个子集，即 11 位指数被限制在相应的单精度范围之内，尾数的最低 29 位全为“0”；对于非规格化单精度浮点数，其对应阶码偏移值为 127。由基本格式单精度经由 MAP_S2D 转换得到的扩展格式单精度称为规范的单精度；否则，称之为非规范的单精度。对规范的单精度数值进行单精度运算将产生单精度的结果，对非规范的单精度数值进行单精度运算将产生不可预测的结果。

5.1.3 浮点实现与异常自陷

若未实现某条浮点指令，则试图执行该指令将产生非法指令故障。

任何浮点存储与装入指令都可能产生存储管理故障，包括越权访问（ACV：AccessViolation）、对界（Alignment）等。

SW-AI处理器实现IEEE浮点类型时，可以实现浮点控制寄存器FPCR中所有或部分自陷屏蔽标志和非规格化数控制标志，其中自陷屏蔽标志包括无效操作屏蔽（INVD：InvalidOperationDisable）、除数为零屏蔽（DZED：DivisionbyZeroDisable）、下溢屏蔽（UNFD：UnderflowDisable）、上溢屏蔽（OVFD：OverflowDisable）、非精确结果屏蔽（INED：InexactDisable）和整数溢出屏蔽（IOVD：IntegerOverflowDisable）。

SW-AI处理器从核实现了IEEE标准未定义的非规格化数异常屏蔽（DNOD：DenormalOperandExceptionDisable）。另外设置了非规格化数标记DNO，以及相应控制位，包括非规格化硬件处理使能（DNOE：DenormalOperandEnable）、下溢设置为“0”（UNDZ：UnderflowtoZero）和精确非规格化数下溢使能（EXUN：ExactdenormalUnderflowenable）。上述控制位DNOE、UNDZ与EXUN仅对标量双精度浮点指令（除浮点转换指令）有效，对于浮点转换指令、标量单精度浮点指令或者向量浮点指令，上述控制无意义，实际硬件按照DNOE固定为1’b0、UNDZ固定为1’b1和EXUN固定为1’b1控制实现。

标量双精度浮点指令对未实现全部自陷屏蔽标志和非规格化控制标志的情况，必须作如下处理：

1）如果未实现自陷屏蔽标志，则在发生异常后产生相应的自陷；

2）无论是否实现DNOE，任何以非规格化数作为操作数的浮点操作将产生非规格化数异常（DNO），再根据DNOD确定是否自陷；

3）当UNDZ为“1”或UNDZ为“0”且EXUN为“1”时，任何操作过程中产生下溢的浮点操作将产生下溢异常（UNF），再根据UNFD确定是否自陷；

4）当UNDZ为“0”且EXUN为“0”时，除输出结果为精确非规格化数情形外，其它任何操作过程中产生下溢的浮点操作将产生下溢异常（UNF），再根据UNFD确定是否自陷。当输出结果为精确非规格化数时特殊处理，不产生下溢异常。

5.1.4 约定的说明

1）有限数（finitenumber）：一个确定的、值在一定范围内的浮点数。具体来说，所有在－MAX到－MIN、0、＋MIN到＋MAX之间（包括端点）的数都是有限数。其中MAX是可表示的最大有限数，MIN为可表示的最小非零数。有限数不包括无穷大、非数（NaN）、非规格化数，但包含“－0”。

2）非规格化数（denormal/subnormal）：其幅值（或绝对值）处于“0”和最小有限数之间的数称为非规格化数。非规格化数有正非规格化数和负非规格化数之分。

3）无穷大（infinity）：包括正无穷大和负无穷大，任何有限数都小于正无穷大，大于负无穷大。

4）最低有效位（LSB）：即具有最小权值的位。对于可表示的正有限数A，A＋1LSB即为下一个较大的可表示数，A＋1/2LSB正好位于A和A的下一个较大的可表示数的中间。对于可表示的正有限数据A，如果尾数不全为“0”，A－1LSB即为下一个较小的可表示数，A－1/2LSB正好位于A和A的下一个较小的可表示数的中间。

5）非有限数（non-finitenumber）：无穷大、非数（NaN）和非规格化数称为非有限数。

6）非数（NaN：NotaNumber）：是用来指示特定含义而非数据的位串。其格式为指数全“1”而尾数非全“0”。NaN有通知的NaN（SignalingNaN）和静止的非数（QuietNaN）两种类型，在SW-AI处理器中，非数的尾数第一位为“0”表示通知的非数，尾数第一位为“1”表示静止的非数。

7）可表示结果（representableresult）：一个可用IEEE浮点数格式精确表示的实数，具有有限的精度和有限的指数范围。

8）真值（trueresult）：在算术上完全准确的运算结果，即为真值。这里假定输入操作数也是精确的。真值常默认地舍入到最接近的可表示结果。

9）真零（truezero）：浮点寄存器中 64 位“0”所表示的数值“＋0”即为真零。

5.1.5 浮点数的表示范围

SW-AI处理器以符号、指数和尾数来表示IEEE浮点数，其中符号用1位二进制数表示，指数用5、8或11位二进制表示，尾数用10、23或52位二进制表示。下面两张表分别表示浮点数范围和各种浮点数。

表 0-6 SW-AI 处理器表示的浮点数范围

数据格式	最小数	最大数
HP浮点	2**-14*1.0 6.103515625e-5	2**15(2.0-2**-10) 6.5504e4
SP浮点	2**-126*1.0 1.17549435e-38	2**127(2.0-2**-23) 3.40282347e38
DP浮点	2**-1022*1.0 2．2250738585072013e-308	2**1023*(2.0-2**-52) 1.7976931348623158e308

表 0-7 SW-AI 处理器所表示的各种浮点数

符号	指数	尾数	含义	是否为有限数*
0/1	全“1”	非全“0”	正/负非数	否
0/1	全“1”	全“0”	正/负无穷大	否
0	全“0”	非全“0”	正非规格化数	否
1	全“0”	非全“0”	负非规格化数	否
0	全“0”	全“0”	正“0”	是

1	全“0”	全“0”	负“0”	是
任意	其它	任意	有限数	是

*：此处“有限数”是前面约定的专有名词。其实非规格化数表示范围亦是有限的，但按专有名词定义不能称为有限数。

5.2 舍入模式

所有的舍入模式都将某个真值映射成一个可表示的值，即所有浮点操作首先产生一个可精确表达的中间结果，然后由舍入操作将此精确表达的中间结果保留为可表达的结果（即规定的标准格式）。对 IEEE 浮点运算，有四种舍入模式：就近舍入（常规舍入）、向负无穷大舍入、向“0”舍入（截尾）、向正无穷大舍入。SW-AI 处理器浮点运算是在检查上溢和下溢前进行舍入，因此可能导致发生上溢或下溢。不同的舍入方式，对上溢和下溢的判断有所不同，下表为四种舍入模式对应的上溢和下溢条件。其中，MIN 和 MAX 分别表示在指定数据格式下所能表示的最小和最大规格化数据。

表 0-8 IEEE 四种舍入方式对应的上溢和下溢条件

序号	舍入方式	上溢	下溢
1	就近舍入	绝对值>=MAX＋1/2LSB	非“0”绝对值＜MIN－1/2LSB
2	向“0”舍入	绝对值>=MAX＋1LSB	非“0”绝对值＜MIN
3	向正无穷舍入	正真值＞MAX 负真值<=－MAX－1LSB	正真值<=MIN—1LSB 负真值＞－MIN
4	向负无穷舍入	正真值>=MAX＋1LSB 负真值＜－MAX	正真值＜MIN 负真值>=－MIN＋1LSB

5.2.1 就近舍入

就近舍入为缺省舍入方式，也称为常规舍入方式。舍入时向离其最近的可表示数值舍入，如果恰好位于两个可表示数值的中央，则向最低有效位为“0”的数值舍入。当真值的绝对值>=MAX＋1/2LSB 时为上溢，当非“0”真值的绝对值＜MIN－1/2LSB 时为下溢。

5.2.2 向“0”舍入

向“0”舍入是将真值向 2 个可表示数值中的绝对值较小者舍入，即向靠近“0”的方向舍入。当真值的绝对值>=MAX＋1LSB 时为上溢，当非零真值的绝对值＜MIN 时为下溢。

5.2.3 向正无穷大舍入

向正无穷大舍入是将真值向 2 个可表示数值中的较大者舍入，舍入后的结果不应小于原真值。当正的真值＞MAX 或负的真值<=－MAX－1LSB 时为上溢。当正的真值<=＋MIN－1LSB 或负的真值＞－MIN 时为下溢。

5.2.4 向负无穷大舍入

向负无穷大舍入是将真值向 2 个可表示数值中的较小者舍入，舍入后的结果不应大于原数值。当正的真值>=MAX＋1LSB 或负的真值＜－MAX 时为上溢。当正的真值＜＋MIN 或负的真值>=－MIN＋1LSB 时为下溢。

5.2.5 SW-AI 处理器的舍入实现

SW-AI 处理器支持 IEEE 的四种舍入模式，通过 FPCR[59:58]的动态舍入模式位来确定采用哪种舍入模式，对应关系如下表：

表 0-9 SW-AI 处理器浮点指令与 IEEE 舍入方法的对应关系

序号	FPCR[59:58]	舍入模式
1	00_2	向“0”舍入
2	01_2	向负无穷大舍入
3	10_2	就近舍入（默认）
4	11_2	向正无穷大舍入

5.3 算术异常

SW-AI 处理器支持 IEEE 标准定义的五种浮点算术异常，它们分别为无效操作（INV，InvalidOperation）、除数为“0”（DZE，DivisionbyZero）、上溢（OVF，Overflow）、下溢（UNF，Underflow）和非精确结果（INE，InexactResult）。另外，在浮点转换为整数时可能会产生一种整数算术异常，即浮点转换整数溢出（IOV，IntegerOverflow）。

检测到这些异常时必须报告。报告方式包括在浮点控制和状态寄存器 FPCR 中设置状态标志，并检查屏蔽控制位以决定是否向从核外部通知异常。除浮点整数溢出外，发生其它异常时的缺省操作是继续处理而不通知异常（即异常可以屏蔽）。SW-AI 处理器从核不对异常进行处理，向外部通知异常的同时作废流水线，程序员需要根据异常查找问题。

浮点控制和状态寄存器 FPCR 为每一种异常提供一个状态标志，以便在发生异常时设置该标志。SW-AI 处理器支持对这些标志的保留和恢复，也支持单独地检查和清除状态标志。同时 FPCR 还包含各种异常的自陷屏蔽位，在特定条件下可以控制是否通知异常。另外 FPCR 还要一些特殊数据控制位，用于对特殊浮点源操作数或浮点运算结果数据的处理。

5.3.1 无效操作（INV）

若当前操作的一个操作数为非有限数或对要执行的操作而言是非法的（浮点比较指令的操作数为无穷大时不产生自陷），则产生无效操作（InvalidOperation）异常。无效操作主要包括：

1） 对通知的非数（SignalingNaN）进行的操作；

2） 不同符号的无穷大加法或相同符号的无穷大减法（(+∞＋-∞）或（+∞－+∞））；

3）“0”与无穷大的乘法（0×∞）；

4）“0”除以“0”或无穷大除以无穷大（0/0 或∞/∞）；

5）将无穷大或非数（NaN）转换为整数；

6）对非零的负数求平方根；

7）在含有排序语义（<、≤、≥、>）的比较操作中，任意一个操作数是非数（NaN）。

若 INV 产生，结果为浮点数的目标寄存器内容为静止的非数（QuietNaN），结果为整数的目标寄存器内容为“0”，结果为逻辑值的目标寄存器在非数比较 FCMPUN 时为“1”，其余为“0”。

5.3.2 除数为“0”（DZE）

若除数为“0”，而被除数是一个不会引起无效操作异常的数据，则产生除数为“0”（DivisionbyZero）异常。

若 DZE 产生，写入目标寄存器的结果为包含正确符号的无穷大。

5.3.3 上溢（OVF）

上溢是在舍入过程中产生的。当舍入结果的幅值（绝对值）超过目标格式的最大有限值时即产生上溢（Overflow）异常。

若 OVF 产生，上溢的结果将由舍入模式和中间结果的符号确定，具体如下：

1）就近舍入：结果幅值为无穷大，符号为中间结果的符号；

2）向“0”舍入：结果幅值为目标格式的最大有限值，符号为中间结果的符号；

3）向负无穷大舍入：正上溢结果为目标格式的最大正有限值，负上溢结果为负无穷大；

4）向正无穷大舍入：负上溢结果为目标格式的最大负有限值，正上溢结果为正无穷大。

5.3.4 下溢（UNF）

下溢是在舍入过程中产生的。当舍入结果的幅值（绝对值）小于目标格式的最小有限值时产生下溢（Underflow）异常。

若 UNF 产生，对于双精度浮点，将根据 FPCR[UNDZ]控制位确定写入目标寄存器的值。若 FPCR[UNDZ]无效，则将 IEEE754 标准指定的下溢结果写入目标寄存器；否则写入目标寄存器的结果为真零（64 位全“0”）。对于单精度浮点或者半精度浮点（包括扩展半精度浮点），如果 UNF 产生，下溢结果为真 0(+0)。

5.3.5 非精确结果（INE）

如果舍入后的结果与舍入前的真值不一致，则产生非精确结果（InexactResult）异常。

若 INE 产生，写入目标寄存器的结果为舍入后结果或上溢的结果。

5.3.6 整数溢出（IOV）

在浮点向整数（长字）转换的过程中，若舍入后的结果超出了-2^{63}～2^{63-1}的范围，则产生整数溢出（IntegerOverflow）异常。检查整数溢出的 2 条浮点指令分别为 FCVTDL（双精度浮点转换为长字）和 FCVTDLR（双精度浮点转换为长字，自带舍入模式）。若 IOV 产生，则写入目标寄存器的结果为转换后整数中间结果的低 64 位。

注：SW-AI 处理器从核中，标量整数乘法指令虽然复用浮点乘加部件实现，但采用单独的整数溢出标记。该标记与其它整数部件溢出标记集中登记，而非在 FPCR 中。如果乘法结果超出了对应数据格式所能表示的范围，则产生单独的乘法整数溢出（IntegerOverflow）异常。检查整数溢出的 2 条整数乘法指令分别为 MULW 和 MULL。

5.4 非规格化数处理

SW-AI 处理器从核仅支持双精度浮点非规格化数的硬件处理，对于单精度与半精度浮点均不支持非规格化数，输入将其当做同符号 0 处理，输出按下溢处理，且下溢为真 0（+0）。双精度浮点受 FPCR 中 DNOE（非规格化数硬件处理使能）、DNOD（非规格化数自陷屏蔽）、UNFD（下溢自陷屏蔽）、UNDZ（下溢为“0”）和 EXUN（精确非规格化数下溢使能）控制位的控制，具体如下：

1） 输入操作数中包含非规格化数时：

表 0-10 输入操作数中包含非规格化数时处理方式

DNOE	DNOD	处理
1	任意值	硬件实现非规格化数处理，在 FPCR 登记非规格化数标记，不产生非规格化数异常自陷
0	1	非规格化数当 0 处理，在 FPCR 登记非规格化数标记，不产生非规格化数异常自陷
0	0	非规格化数当 0 处理，在 FPCR 登记非规格化数标记，同时产生非规格化数异常自陷

2） 运算结果的绝对值小于最小规格化数，但可以用非规格化数表示时：

表 0-11 结果为非规格化数时处理方式

UNFD	UNDZ	EXUN	处理
1	1	任意值	结果为真 0。在 FPCR 登记下溢异常标记，不产生下溢异常自陷
1	0	1	结果为非规格化数。无论结果是否精确，均在 FPCR 登记下溢异常标记，不产生下溢异常自陷
1	0	0	结果为非规格化数。结果非精确时在 FPCR 登记下溢异常标记；否则，不登记。不产生下溢异常自陷
0	1	任意值	结果为真 0。在 FPCR 登记下溢异常标记，产生下溢异常自陷
0	0	1	结果为非规格化数。在 FPCR 登记下溢异常标记，产生下溢异常自陷
0	0	0	结果为非规格化数。结果非精确时在 FPCR 登记下溢异常标记，产生下溢异常自陷；否则，不登记下溢异常标记，不产生下溢异常自陷

5.5 浮点控制状态寄存器 FPCR

从核执行部件内部设置了一个专门控制浮点指令执行和记录浮点指令执行状态的浮点控制状态寄存器 FPCR，该寄存器属于特殊的控制状态寄存器（CSR），从核只能通过指令 RFPCR 和 WFPCR 读写访问，主核或维护可以通过该寄存器在 IO 空间中的编址进行读写。从核的访问与主核/维护的访问为异步事件，硬件不保证先后关系。由于浮点 SIMD 扩展需要记录 8 个浮点分量执行状态，因此 FPCR 分为两部分：基本 FPCR 和扩展 FPCR（FPCR_EXT）（在《SW-AI IO 寄存器手册》中称为 FPCR 低段（FPCRL）和 FPCR 高段（FPCRH））。

下表为 FPCR 中的内容：

表 0-12 浮点控制寄存器 FPCR 内容描述

寄存器位	标识	含义	
63	SUM	总算术异常和非规格化数异常标志：[57:52]、[46]、[42:36]、[26:20]、[10:4]位中存在“1”，或者扩展FPCR(FPCR_EXT）的[54:48]、[38:32]、[22:16]、[6:0]位中存在“1”，则该位为“1”。	
62	INED	非精确结果自陷屏蔽：屏蔽非精确结果自陷并将IEEE标准指定的结果写入目标寄存器。	
61	UNFD	下溢自陷屏蔽：屏蔽下溢自陷，但写入目标寄存器的结果受UNDZ位控制。	
60	UNDZ	下溢为“0”：控制下溢时写入目标寄存器中的结果，若为“1”，将真零（64位全“0”，参照Alpha实现）写入目标寄存器；否则以IEEE标准指定的结果写入目标寄存器。	
[59:58]	DYN	动态舍入模式：指示指令的具体舍入模式，具体定义如下： 00_2：向“0”舍入 01_2：向负无穷大舍入 10_2：就近舍入 11_2：向正无穷大舍入	
57	IOV0	整数溢出：浮点转换为整数的操作产生整数溢出。	1、浮点标量运算指令产生的六种浮点算术异常。 2、浮点SIMD扩展指令对浮点向量[63:0]位（标量DP或
56	INE0	非精确结果：浮点算术运算或转换操作的结果与精确结果不同。	
55	UNF0	下溢：浮点算术运算或转换操作结果的指数产生下溢。	
54	OVF0	上溢：浮点算术运算或转换操作结果的指数产生上溢。	

53	DZE0	除数为“0”：浮点除法运算的除数为“0”。	标量ESP64或标量EHP64第0个元素，或SP/EHP32 第1~0个元素）进行运算产生的六种浮点算术异常。
52	INV0	无效操作：浮点算术、转换或比较运算中，一个或多个操作数是非有限数，或者对具体操作而言，操作数是非法的。	
51	OVFD	上溢自陷屏蔽：禁止上溢自陷，并将IEEE标准指定的结果写入目标寄存器。	
50	DZED	除数为“0”自陷屏蔽：禁止除数为“0”自陷，并将IEEE标准指定的结果写入目标寄存器。	
49	INVD	无效操作自陷屏蔽：禁止无效操作自陷，并将IEEE标准指定的结果写入目标寄存器。	
48	DNOE	非规格化数硬件处理使能：若为“1”，支持非规格化数硬件处理，不发生非规格化数自陷；若为“0”，将非规格化数当0处理，并根据DNOD位的设置来决定是否发生非规格化数自陷。 注：Alpha和主核此位含义为DNZ，非规格化数设置为0。	
47	DNOD	非规格化数自陷屏蔽：禁止非规格化数自陷。	
46	DNO0	非规格化数异常：浮点输入操作数中包含非规格化数时登记非规格化数标记。（1、浮点标量运算指令产生的非规格化异常。2、浮点SIMD扩展指令对浮点向量第0个元素进行运算产生的非规格化异常。）	
45	IOVD	整数溢出自陷屏蔽：禁止浮点的整数溢出，并将IEEE标准指定的结果写入目标寄存器。	
44	EXUN	精确非规格化数报告下溢使能：当输出结果为精确非规格化数且UNDZ=0时，若为“1”，则登记下溢异常；若为“0”，不登记。	
43	RAZ/IGN	保留位，读出为“0”，写时忽略。	
42	DNO1	非规格化数异常：浮点输入操作数中包含非规格化数时登记非规格化数标记。	浮点SIMD扩展指令对浮点向量[127:64]位（SP或EHP32第3~2个元素）进行运算产生的六
41	IOV1	整数溢出：不使用，固定为“0”。	
40	INE1	非精确结果：浮点算术运算或转换操作的结果与精确结果不同。	
39	UNF1	下溢：浮点算术运算或转换操作结果的指数产生下溢。	

38	OVF1	上溢：浮点算术运算或转换操作结果的指数产生上溢。	种浮点算术异常和非规格化数异常。
37	DZE1	除数为“0”：浮点除法运算的除数为“0”。	
36	INV1	无效操作：浮点算术、转换或比较运算中，一个或多个操作数是非有限数，或者对具体操作而言，操作数是非法的。	
[35:27]	RAZ/IGN	保留位，读出为“0”，写时忽略。	
26	DNO2	非规格化数异常：浮点输入操作数中包含非规格化数时登记非规格化数标记。	浮点SIMD扩展指令对浮点向量[191:128]位（SP或EHP32第5~4个元素）进行运算产生的六种浮点算术异常和非规格化数异常。
25	IOV2	整数溢出：不使用，固定为“0”。	
24	INE2	非精确结果：浮点算术运算或转换操作的结果与精确结果不同。	
23	UNF2	下溢：浮点算术运算或转换操作结果的指数产生下溢。	
22	OVF2	上溢：浮点算术运算或转换操作结果的指数产生上溢。	
21	DZE2	除数为“0”：浮点除法运算的除数为“0”。	
20	INV2	无效操作：浮点算术、转换或比较运算中，一个或多个操作数是非有限数，或者对具体操作而言，操作数是非法的。	
[19:11]	RAZ/IGN	保留位，读出为“0”，写时忽略。	
10	DNO3	非规格化数异常：浮点输入操作数中包含非规格化数时登记非规格化数标记。	浮点SIMD扩展指令对浮点向量[255:192]位（SP或EHP32第7~6个元素）进行运算产生的六种浮点算术异常和非规格化数异常。
9	IOV3	整数溢出：不使用，固定为“0”。	
8	INE3	非精确结果：浮点算术运算或转换操作的结果与精确结果不同。	
7	UNF3	下溢：浮点算术运算或转换操作结果的指数产生下溢。	
6	OVF3	上溢：浮点算术运算或转换操作结果的指数产生上溢。	
5	DZE3	除数为“0”：浮点除法运算的除数为“0”。	
4	INV3	无效操作：浮点算术、转换或比较运算中，一个或多个操作数是非有限数，或者对具体操作而言，操作数是非法的。	

[3]	RAZ/IGN	保留位，读出为“0”，写时忽略。	
[2]	EXC_CTL_WEN	只写，读出为0，写修改FPCR时，写该位为“1”才允许同时更改EXC_CTL域的值。	
[1:0]	EXC_CTL	控制浮点算术自陷（含浮点部件的整数溢出异常自陷，不包括非规格化数异常）屏蔽模式，选择使用FPCR其它控制位的方式： “00”：允许所有浮点算术异常自陷且受FPCR中控制位的控制； “01”：允许除不精确结果之外的所有浮点算术异常自陷且受FPCR中控制位的控制； “10”：允许除不精确结果之外的所有浮点算术异常自陷且不受FPCR中控制位的控制； “11”：允许除浮点下溢、不精确结果和整数溢出之外的所有浮点算术异常自陷且不受FPCR中控制位的控制。	
浮点控制寄存器FPCR的扩展位置（FPCR_EXT）内容描述			
[63:55]	RAZ/IGN	保留位，读出为“0”，写时忽略。	
54	DNO4	非规格化数异常：浮点输入操作数中包含非规格化数时登记非规格化数标记。	浮点SIMD扩展指令对浮点向量[319:255]位（SP或EHP32第9~8个元素）进行运算产生的六种浮点算术异常和非规格化数异常。
53	IOV4	整数溢出：不使用，固定为“0”。	
52	INE4	非精确结果：浮点算术运算或转换操作的结果与精确结果不同。	
51	UNF4	下溢：浮点算术运算或转换操作结果的指数产生下溢。	
50	OVF4	上溢：浮点算术运算或转换操作结果的指数产生上溢。	
49	DZE4	除数为“0”：浮点除法运算的除数为“0”。	
48	INV4	无效操作：浮点算术、转换或比较运算中，一个或多个操作数是非有限数，或者对具体操作而言，操作数是非法的。	
[47:39]	RAZ/IGN	保留位，读出为“0”，写时忽略。	
38	DNO5	非规格化数异常：浮点输入操作数中包含非规格化数时登记非规格化数标记。	浮点SIMD扩展指令对浮点向量[383:320]位（SP或
37	IOV5	整数溢出：不使用，固定为“0”。	
36	INE5	非精确结果：浮点算术运算或转换操作的结果与精确结果不同。	

35	UNF5	下溢：浮点算术运算或转换操作结果的指数产生下溢。	EHP32 第11~10 个元素）进行运算产生的六种浮点算术异常和非规格化数异常。
34	OVF5	上溢：浮点算术运算或转换操作结果的指数产生上溢。	
33	DZE5	除数为“0”：浮点除法运算的除数为“0”。	
32	INV5	无效操作：浮点算术、转换或比较运算中，一个或多个操作数是非有限数，或者对具体操作而言，操作数是非法的。	
[31:23]	RAZ/IGN	保留位，读出为“0”，写时忽略。	
22	DNO6	非规格化数异常：浮点输入操作数中包含非规格化数时登记非规格化数标记。	浮点SIMD扩展指令对浮点向量[447:384]位（SP 或EHP32 第13~12 个元素）进行运算产生的六种浮点算术异常和非规格化数异常。
21	IOV6	整数溢出：不使用，固定为“0”。	
20	INE6	非精确结果：浮点算术运算或转换操作的结果与精确结果不同。	
19	UNF6	下溢：浮点算术运算或转换操作结果的指数产生下溢。	
18	OVF6	上溢：浮点算术运算或转换操作结果的指数产生上溢。	
17	DZE6	除数为“0”：浮点除法运算的除数为“0”。	
16	INV6	无效操作：浮点算术、转换或比较运算中，一个或多个操作数是非有限数，或者对具体操作而言，操作数是非法的。	
[15:7]	RAZ/IGN	保留位，读出为“0”，写时忽略。	
6	DNO7	非规格化数异常：浮点输入操作数中包含非规格化数时登记非规格化数标记。	浮点SIMD扩展指令对浮点向量[511:448]位（SP 或EHP32 第15~14 个元素）进行运算产生的六种浮点算术异常和非规格化数异常。
5	IOV7	整数溢出：不使用，固定为“0”。	
4	INE7	非精确结果：浮点算术运算或转换操作的结果与精确结果不同。	
3	UNF7	下溢：浮点算术运算或转换操作结果的指数产生下溢。	
2	OVF7	上溢：浮点算术运算或转换操作结果的指数产生上溢。	
1	DZE7	除数为“0”：浮点除法运算的除数为“0”。	
0	INV7	无效操作：浮点算术、转换或比较运算中，一个	

		或多个操作数是非有限数，或者对具体操作而言，操作数是非法的。	

除新增 DNO 非规格化数标记、EXUN 精确非规格化数下溢使能和 IOVD 浮点整数溢出自陷屏蔽位外，FPCR_BASE 其它位与 Alpha/主核 FPCR 定义保持兼容（DNOE 位替代 DNZ 位，恢复 DNOD 位）。

说明：

（1）本寄存器中控制位直接写，异常状态位“写 1 清”。

（2）从核不支持浮点转换指令的 SIMD 扩展，因此基本 FPCR 的[9]、[25]、[41]位固定为 0。从核不支持浮点除法平方根指令的 SIMD 扩展，因此基本 FPCR 的[5]、[21]、[37]位固定为 0。

（3）异常登记位的值与对应屏蔽位的值无关，即使某些异常被屏蔽，一旦产生异常，仍要在 FPCR 中登记相应的异常标志。这些位记录为“1”后，只有在软件对这些位进行“写 1 清”，才能清除对应的异常标志位。

（4）从核自身可以通过 RFPCR、WFPCR 指令来完成对 FPCR 的读写访问。写 FPCR 时，硬件保证其不影响 WFPCR 指令之前的浮点指令，并对 WFPCR 指令之后的浮点指令确保产生作用。读 FPCR 时，硬件保证现场准确，即 RFPCR 指令之前的浮点指令都完成，且不执行新的浮点指令。

（5）主核和维护部件对本寄存器的访问，与从核流水线执行 RFPCR/WFPCR 或浮点指令，是两个独立的异步事件。硬件不保证其先后关系。

（6）复位后 FPCR 寄存器初始值为：64’h680f_8000_0000_0000。即舍入模式默认为就近舍入；除浮点整数溢出自陷屏蔽位 IOVD 位默认为 0 外，其它屏蔽位默认为 1；非规格化数默认采用硬件处理，DNOE 位为 1，UNDZ 为 0，EXUN 为 0。

（7）复位后 FPCR 寄存器扩展部分初始值为：64’h0000_0000_0000_0000。

6 指令详细描述（199 条）

下面描述的指令，在列表中的“操作码”项，其“.”之前的编码为指令格式中的操作码（opcode），之后的编码为指令格式中的功能码（opfunc）。通过操作码和功能码来区分不同的指令。在指令详细描述中操作数的描述格式为“**<名称>.<访问类型><数据类型>**”，各部分的含义见“指令格式说明”一节。

6.1 控制类指令（5 条）

控制类指令采用存储器指令格式，包括 1 条停机指令，3 条同步指令和 1 条存储器栏栅指令，共 5 条指令。如下表所示：

表 0-13 控制指令列表

序号	指令	操作码	描述
1	halt	0x0.7	停机指令
2	synr Ra	0x0.4.0	核组内行同步，Ra 的低 8 位存放同步向量
3	sync Ra	0x0.4.1	核组内列同步，Ra 的低 8 位存放同步向量
4	synp Ra	0x0.4.2	核组内点点同步，Ra 的低 6 位存放同步从核心号
5	memb	0x0.3	存储器栏栅

6.1.1 停机指令

格式： HALT //存储器指令格式

操作： {在预译码站台识别后，推测暂停取指，直至进入执行站台后，
使指令流水线停顿。}

异常： 无

助记符： HALT 停机指令

指令描述：

HALT 停机指令，使从核停止取指，停顿从核核心流水线，程序计数器（PC）保持在 HALT 指令 PC。HALT 指令之后（程序序）的所有指令不会被执行，HALT 指令之前（程序序）的所有指令正常排空。

执行 HALT 指令之后的从核，必须通过主核或维护进行 IO 操作（修改 PC、设置启动标志），才能重新启动执行。如果需要转入下一个应用的运行，必须先刷新指令存储。

6.1.2 同步

格式： SYNx Ra.rl //存储器指令（杂项指令）格式

操作： {进入执行站台后，发出同步请求，阻塞其后所有指令的执行，等待同步响应到达后继续}

异常： 同步异常

助记符：

SYNR	行同步
SYNC	列同步
SYNP	点点同步

指令描述：

核组内行、或列、或点点同步指令用于核组内同一行、或同一列、或2个从核之间的同步。同步指令必须在其前面（程序序）所有指令发射后才能够发射执行，并且其后面（程序序）所有指令不能跨越其先执行。同步指令进入执行站后，发出同步请求，阻塞其后所有指令的执行，等待同步响应到达后继续。

对于SYNR行同步指令，Ra是64位寄存器，低8位有效，[7:0]分别指示同行从核7到从核0是否同步，为1时指示该从核参与同步，为0时屏蔽该从核。对于SYNC列同步指令，Ra是64位寄存器，低4位有效，[3:0]分别指示同行从核3到从核0是否同步，为1时指示该从核参与同步，为0时屏蔽该从核。

对于SYNP同步指令，Ra是64位寄存器，低5位有效，表示与当前从核同步的对应从核心号，其中[4:3]表示从核行号，[2:0]表示从核列号。

行同步和列同步时，本从核不包含在同步向量时产生同步向量异常。

单个从核与自身点对点同步时不报异常，可正常完成。

6.1.3 存储器栏栅

格式： MEMB //存储器指令格式

操作： {阻塞其后所有指令的执行，直到确认之前的所有存储器访问指令都已完成处理}

异常： 无

助记符： MEMB 存储器栏栅

指令描述：

MEMB 指令用于保证存储器访问指令的顺序性。

不使用 MEMB 指令时，其它处理核心观测到的本核心发出的对不同地址空间的存储器访问操作可能是乱序完成的，而 MEMB 指令则用于确保访存操作顺序进行。

6.2 转移类指令（14 条）

SW-AI 处理器从核提供了整数条件转移、浮点条件转移、无条件转移指令和寄存器间接转移指令，这些指令使用的 PC 都是已更新的 PC 值。此类指令共有 14 条，其中条件转移使用转移指令格式、无条件转移指令使用转移指令格式，其中 Disp[21:18]位置用作功能码 FUNC[3:0]，寄存器间接转移指令使用杂项指令格式。

表 0-14 转移指令列表

序号	指令		操作码	描述
1	beq	Ra,disp	0x2.1	若 Ra＝“0”则转移
2	bne	Ra,disp	0x2.6	若 Ra≠“0”则转移
3	blt	Ra,disp	0x2.2	若 Ra＜“0”则转移
4	ble	Ra,disp	0x2.3	若 Ra≤“0”则转移
5	bgt	Ra,disp	0x2.4	若 Ra＞“0”则转移
6	bge	Ra,disp	0x2.5	若 Ra≥“0”则转移
7	fbeq	Ra,disp	0x2.9	若 Ra＝浮点“0”则转移
8	fbne	Ra,disp	0x2.e	若 Ra≠浮点“0”则转移
9	fblt	Ra,disp	0x2.a	若 Ra＜浮点“0”则转移
10	fble	Ra,disp	0x2.b	若 Ra≤浮点“0”则转移
11	fbgt	Ra,disp	0x2.c	若 Ra＞浮点“0”则转移
12	fbge	Ra,disp	0x2.d	若 Ra≥浮点“0”则转移
13	jmp	Ra,Rb[,disp]	0x0.6	寄存器间接转移
14	br	Ra,disp	0x1	无条件转移

1) 对于条件转移指令，使用转移预测位（disp<17:0>的最高位）进行预测。如果为“0”，则预测为不发生跳转，顺序取当前指令紧接着的下一条指令；如果为“1”，则预测为发生跳转。同时，该预测位还兼符号位的作用，因此目标地址为 PC+4*SEXT(disp<17:0>)。如果最终发现预测失败，则撤销该转移指令之后取出的所有指令；
2) 对于无条件转移指令 BR，总是发生跳转，目标地址为 PC+4*SEXT(disp<21:0>)；
3) 对于寄存器间接转移指令 JMP，不进行转移预测，暂停取指，待指令执行结束后从正确的目标地址重新开始取指。

6.2.1 整数条件转移指令

格式： Bxx Ra.rl,disp.al //转移指令格式

操作： {update PC}

Vaddr←PC+{4×SEXT(disp<17:0>)}

IF TEST(Rav,Condition_based_on_Opcode) THEN PC←Vaddr

异常： 无

助记符：

BEQ	如果寄存器的值等于 0 则转移
BNE	如果寄存器的值不等 0 则转移
BLT	如果寄存器的值小于 0 则转移
BLE	如果寄存器的值小于等于 0 则转移
BGT	如果寄存器的值大于 0 则转移
BGE	如果寄存器的值大于等于 0 则转移

指令描述：

测试寄存器 Ra，如果指定的关系为真，则用目标地址装填 PC，否则继续顺序执行。其中测试是对整个 64 位进行的。指令中偏移量（disp）的最高位<17>即作转移预测位，也作符号位。<17:0>为有符号的指令偏移，需要先左移 2 位，再符号扩展为 64 位值，然后加上更新后的 PC 值形成目标指令地址。

6.2.2 浮点条件转移指令

格式： **F**Bxx Ra.rl,disp.al //转移指令格式

操作： {update PC}

Vaddr←PC+{4×SEXT(disp<17:0>)}

IF TEST(Rav,Condition_based_on_Opcode) THEN PC←Vaddr

异常： 无

助记符：

FBEQ	如果寄存器的值（符合 DP 浮点规范）等于浮点 0 转移
FBNE	如果寄存器的值（符合 DP 浮点规范）不等于浮点 0 则转移
FBLT	如果寄存器的值（符合 DP 浮点规范）小于浮点 0 则转移
FBLE	如果寄存器的值（符合 DP 浮点规范）小于等于浮点 0 则转移
FBGT	如果寄存器的值（符合 DP 浮点规范）大于浮点 0 则转移
FBGE	如果寄存器的值（符合 DP 浮点规范）大于等于浮点 0 则转移

指令描述：

测试寄存器 Ra，如果指定的关系为真，则用目标地址装填 PC，否则继续顺序执行。其中测试是对整个 64 位进行的。指令中偏移量（disp）的最高位<17>即作转移预测位，也作符号位。<17:0>为有符号的指令偏移，需要先左移 2 位，再符号扩展为 64 位值，然后加上更新后的 PC 值形成目标指令地址。

SW-AI 处理器在实现浮点条件转移指令时，直接根据 Ra 的“[63]位值”和“[62:0]位是否为全 0”来进行条件判断。在 Ra 为非数时，条件判断的结果和“Ra 与 0 比较”的结果不一致，详细说明见附录 5“浮点条件判断与浮点比较的区别”。

6.2.3 无条件转移指令

格式： BR Ra.wl,disp.al //转移指令格式

操作： {update PC}

Ra←PC AND {NOT3}

PC←PC+{4×SEXT(disp<22:0>)}

异常： 无

助记符： BR 无条件转移

指令描述：

将当前更新的 PC 值（NPC）最低两位变为全“0”后保存到寄存器 Ra 中，然后将目标地址装载到 PC 中。指令中偏移量为有符号的指令偏移，需要先左移两位，再符号扩展为 64 位值，然后加上更新的 PC 值形成目标指令地址。偏移量为 22 位符号值，可在前后各 2M 条指令的范围内实现跳转。

受指令编码限制，BR 指令的目标寄存器 Ra 只能使用$0~$31。

6.2.4 寄存器间接转移指令

格式： JMP Ra.wl,Rb.al //杂项指令格式

操作： {update PC}

Vaddr←Rbv AND {NOT3}

Ra←PC AND {NOT3}

PC←Vaddr

异常： 无

助记符： JMP 寄存器间接转移

指令描述：

当前更新过的 PC 值（NPC）最低 2 位变为全“0”后保存到寄存器 Ra 中，新 PC 是由寄存器 Rb 提供，它的最低两位忽略。Ra 和 Rb 也许指定相同的寄存器，在赋新值之前，用旧值计算目标地址。

6.3 访存类指令（20 条）

访存类指令除了基本的访问存储器的对界访存指令外，还包括原子操作指令。原子操作指令只允许访问主存空间。

除装入立即数指令外，其余访存类指令可能产生异常，大致可分为两类：

一类是对界异常：除不对界访存以外的其它访存指令，要求存储器访问时的地址对界。如果地址不对界，则产生对界异常。

一类是数据流异常，主要包括以下情况：

1）从核根据完成地址计算之后的目标地址的高段来确定访问空间类型（LDM 私有/共享空间），有关虚地址指示空间类型的划分参考《SW-AI 处理器软件接口手册》。如果目标地址没有落在合法的访问空间，或者对只读空间进行写操作，则产生越权访问异常；

2）从核访问 LDM 私有/共享空间过程中，如果超出对应空间容量，则发生越界异常。LDM 全 0 地址访问使能（CSR_LDM_CFG[0]）未打开（即取值为 1’b0）时，如果访问 LDM 0 地址，即不论何种访问粒度，只要访问地址为 0，则产生地址保护的越权访问异常；

3）LDM 共享空间不允许字节和半字粒度的访问，即 LDBU、LDHU 和 STB、STH 不能访问 LDM 共享空间，如果这些指令的地址落在 LDM 共享空间，则产生越权访问异常；

4）从核访问主存过程中，在进行 SDLB 表代换时，可能产生地址越权或越界异常，详见《SW-AI 处理器软件接口手册》；

5）从核在进行主存访问过程中，可能产生主存地址溢出异常，详见《SW-AI 处理器软件接口手册》；

6）从核在进行主存访问过程中，如果对断连的核组进行访问，可能产生核组断连访问异常，详见《SW-AI 处理器软件接口手册》。

6.3.1 对界访存指令（14 条）

表 0-15 对界访存指令列表(存储器指令格式)

序号	指令		操作码	描述
1	ldw	Ra,disp(Rb)	0xb.2	装入字（32 位），符号扩展
2	ldl	Ra,disp(Rb)	0xc.4	装入长字（64 位）或 DP 浮点
3	stw	Ra,disp(Rb)	0xb.3	存储字
4	stl	Ra,disp(Rb)	0xc.5	存储长字
5	ldi	Ra,disp(Rb)	0xe	装入立即数
6	ldih	Ra,disp(Rb)	0xf	装入立即数高位
7	flds	Ra,disp(Rb)	0xb.0	SP 浮点读
8	fsts	Ra,disp(Rb)	0xb.1	SP 浮点写

表 0-16 对界访存指令列表(杂项指令格式)

序号	指令		操作码	描述
1	ldbu	Ra,disp(Rb)	0xa.5	装入字节并作“0”扩展
2	ldhu	Ra,disp(Rb)	0xa.6	装入半字并作“0”扩展
3	stb	Ra,disp(Rb)	0xa.d	存储字节
4	sth	Ra,disp(Rb)	0xa.e	存储半字
5	fldh	Ra,disp(Rb)	0xa.4	HP 浮点读
6	fsth	Ra,disp(Rb)	0xa.c	HP 浮点写

访问粒度为字节、半字和半精度浮点的指令，使用杂项指令格式。FLDS、FSTS、LDW、STW 的 Disp[1:0]位置用作功能码 FUNC[1:0]，LDL、STL 的 Disp[2:0]位置用作功能码 FUNC[2:0]

6.3.1.1 装入立即数指令

格式： LDIx Ra.wl,disp.ab(Rb.ab) //存储器指令格式

操作：
```
CASE
    LDI:  Ra ←Rbv+SEXT(disp)
    LDIH: Ra ←Rbv+SEXT(disp*65536)
ENDCASE
```

异常： 无

助记符：

LDI	装入地址
LDIH	装入高段地址

指令描述：

对于LDI指令，指令中16位偏移量Disp[15:0]经符号扩展后与Rb相加，并赋给Ra；对于LDIH指令，16位偏移量Disp[15:0]左移16位后进行符号扩展，然后与Rb相加，并赋给Ra。

6.3.1.2 装入指令

格式： LDx Ra.wl,disp.ab(Rb.ab) //存储器指令格式/杂项指令格式

操作：
```
Vaddr←{Rbv+SEXT(disp)}
CASE
    LDBU:  Ra←ZEXT((Vaddr)<7:0>)
    LDHU:  Ra←ZEXT((Vaddr)<15:0>)
    LDW:   Ra←SEXT((Vaddr)<31:0>)
    LDL:   Ra←(Vaddr)<63:0>
ENDCASE
```

异常： 对界 （Alignment）

数据流异常

助记符：

LDBU	从存储器装入零扩展字节到寄存器
LDHU	从存储器装入零扩展半字到寄存器
LDW	从存储器装入符号扩展字到寄存器
LDL	从存储器装入长字到寄存器

指令描述：

LDBU 和 LDHU 的 Disp[11:0]为立即数字节偏移地址的[11:0]，偏移量符号扩展后加上寄存器 Rb 内的值形成有效地址 Vaddr，将 Vaddr 对应的存储器内容装入到 Ra 中。指令 LDBU 与 LDHU 需要将

有效结果数据段作零扩展到 64 位后放入 Ra 寄存器中。

LDW 的 Disp[15:2]为立即数字节偏移地址的[15:2]，偏移地址低 2 位[1:0]默认为 0（偏移地址低 2 位[1:0]非全 0 由软件编译静态检查）。LDL 的 Disp[15:3]为立即数字节偏移地址[15:3]位，偏移地址低 3 位[2:0]默认为 0（偏移地址低 3 位[2:0]非全 0 由软件编译静态检查）。偏移量符号扩展后加上寄存器 Rb 内的值形成有效地址 Vaddr，将 Vaddr 对应的存储器内容装入到 Ra 中。指令 LDW 是将有效的结果数据段经符号扩展后放入 Ra 寄存器中。

在所有情况下如果有效地址不是对界的，则产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.3.1.3 存储指令

格式： STx Ra.rx,disp.ab(Rb.ab) //存储器指令格式/杂项指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

CASE

STB: (Vaddr)<7:0>←Rav<7:0>

STH: (Vaddr)<15:0>←Rav<15:0>

STW: (Vaddr)<31:0>←Rav<31:0>

STL: (Vaddr)←Rav

ENDCASE

异常： 对界 （Alignment）

数据流异常

助记符：

STB	从寄存器存储字节到存储器
STH	从寄存器存储半字到存储器
STW	从寄存器存储字到存储器
STL	从寄存器存储长字到存储器

指令描述：

STB 和 STH 的 Disp[11:0]为立即数字节偏移地址的[11:0]；STW 的 Disp[15:2]为立即数字节偏移地址的[15:2]，偏移地址低 2 位[1:0]默认为 0（偏移地址低 2 位[1:0]非全 0 由软件编译静态检查）；STL 的 Disp[15:3]为立即数字节偏移地址的[15:3]，偏移地址低 3 位[2:0]默认为 0（偏移地址低 3 位[2:0]非全 0 由软件编译静态检查）。偏移量符号扩展后加上寄存器 Rb 内的值形成有效地址 Vaddr。然后将寄存器 Ra 中的操作数写入 Vaddr 对应的存储器单元中。

如果有效地址不对界，则产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.3.1.4 SP 浮点读

格式： FLDS Ra.wl,disp.ab(Rb.ab) //存储器指令格式

操作： Vaddr ← {Rbv+SEXT(disp)}

Ra ← (Vaddr)<31> || MAP_S((Vaddr)<30:23>) || (Vaddr)<22:0> || 0<28:0>

异常： 对界 （Alignment）

数据流异常

助记符： FLDS 装入SP浮点

指令描述：

Disp[15:2]为立即数字节偏移地址的[15:2]，偏移地址低2位[1:0]默认为0（偏移地址低2位[1:0]非全0由软件编译静态检查），偏移量符号扩展后加上寄存器Rb内的值形成有效地址Vaddr。FLDS从存储器中取出一个SP浮点写到寄存器Ra。在此过程中，将8位存储器格式的指数扩展为11位寄存器格式的指数，尾数部分进行“0“扩展。

如果有效地址不自然对界，则产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.3.1.5 SP 浮点写

格式： FSTS Ra.rl,disp.ab(Rb.ab) //存储器指令格式

操作： Vaddr ← {Rbv+SEXT(disp)}

(Vaddr)<31:0> ← Rav<63:62> || Rav<58:29>

异常： 对界 （Alignment）

数据流异常

助记符： FSTS 存储SP浮点

限定符： 无

指令描述：

Disp[15:2]为立即数字节偏移地址的[15:2]，偏移地址低 2 位[1:0]默认为 0（偏移地址低 2 位[1:0]非全 0 由软件编译静态检查），偏移量的符号扩展加上寄存器 Rb 的值形成虚地址 Vaddr。FSTS 指令将 Ra 中的 SP 浮点数据存储到存储器中。取自寄存器 Ra 中的源操作数需重新排序以符合 SP 浮点的存储器格式，然后将重新排序后的结果写到存储器中。忽略 Ra 的<61:59>和<28:0>位，但并不检查这些位的合法性。

如果数据不自然对界，会产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.3.1.6 HP 浮点读

格式： FLDH Ra.wl,disp.ab(Rb.ab) //存储器指令格式

操作： Vaddr ← {Rbv+SEXT(disp)}

Ra ← MAP_H2D((Vaddr)<15:0>)

异常： 对界 （Alignment）

数据流异常

助记符： FLDH 装入HP浮点

指令描述：

Disp[11:0]为立即数字节偏移地址的[11:0]，偏移量符号扩展后加上寄存器Rb内的值形成有效地址Vaddr。FLDH从存储器中取出一个基本HP浮点写到寄存器Ra。在此过程中，将5位存储器格式的指数扩展为11位寄存器格式的指数，尾数部分进行“0“扩展。

如果有效地址不自然对界，则产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.3.1.7 HP 浮点写

格式： FSTH Ra.rl,disp.ab(Rb.ab) //存储器指令格式

操作： Vaddr ← {Rbv+SEXT(disp)}

(Vaddr)<15:0> ← Rav<63:62> || Rav<55:42>

异常： 对界 （Alignment）

数据流异常

助记符： FSTH 存储HP浮点

限定符： 无

指令描述：

Disp[11:0]为立即数字节偏移地址的[11:0]，偏移量的符号扩展加上寄存器 Rb 的值形成虚地址Vaddr。FSTH 指令将 Ra 中的 HP 浮点数据存储到存储器中。取自寄存器 Ra 中的源操作数需重新排序以符合基本 HP 浮点的存储器格式，然后将重新排序后的结果写到存储器中。忽略 Ra 的<61:56>和<41:0>位，但并不检查这些位的合法性。

如果数据不自然对界，会产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.3.2 不对界访存指令（2 条）

表 0-17 不对界访存指令列表

序号	指令	操作码	描述
1	ldl_u Ra,disp(Rb)	0xa.7	长字不对界装入

2	stl_u Ra,disp(Rb)	0xa.f	长字不对界存储

6.3.2.1 不对界长字装入指令

格式： LDL_U Ra.wl,disp.ab(Rb.ab) //杂项指令格式

操作： temp←0 //temp为64位数

temp←{Rbv+SEXT(disp)}

Vaddr←temp AND NOT 7 //Vaddr为自然对界地址

Ra←(Vaddr)<63:0>

异常： 数据流异常

助记符： LDL_U 从存储器装入不对界长字

指令描述：

Disp[11:0]为立即数字节偏移地址的[11:0]，偏移量符号扩展后加上寄存器 Rb 内的值，并将低 3 位清 0 后形成有效地址 Vaddr，即地址强制对界后访问存储器，将 Vaddr 对应的存储器内容装入到 Ra 中。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.3.2.2 不对界长字存储指令

格式： STL_U Ra.rl,disp.ab(Rb.ab) //杂项指令格式

操作： temp←0 //temp为64位数

temp←{Rbv+SEXT(disp)}

Vaddr←temp AND NOT 7 //Vaddr为自然对界地址

(Vaddr)←Rav

异常： 数据流异常

助记符： STL_U 从寄存器存储不对界长字到存储器

指令描述：

Disp[11:0]为立即数字节偏移地址的[11:0]，偏移量符号扩展后加上寄存器 Rb 的值，并将低 3 位清 0 后形成有效地址 Vaddr，即地址强制对界后访问存储器，将 Ra 的内容存储到 Vaddr 对应的存储器地址。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.3.3 原子操作（4 条）

实现主存地址空间的取并加（F&A，主要用于锁操作）、原子更新（update，主要用于用户造表操作）和比较并置换操作。

表 0-18 原子操作指令列表

序号	指令	操作码	描述
1	faal Ra,disp(Rb)	0x0.c	长字取并加 1
2	updt Va,disp(Rb)	0x0.d	原子更新
3	casw Va,disp(Rb)	0x0.e	字比较并转换
4	casl Va,disp(Rb)	0x0.f	长字比较并转换

6.3.3.1 原子取并加 1

格式： FAAL Ra.wl,disp.ra(Rb.al) //杂项指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

Ra←原(Vaddr)<63:0> 且 (Vaddr)<63:0>←原(Vaddr)<63:0>+1

异常： 数据流异常

对界 （Alignment）

整数溢出（Integer Overflow）

助记符： FAAL 长字取并加1

指令描述：

对主存地址空间进行“长字取并加 l”操作，该操作不能被打断。

指令中 12 位偏移量 Disp[11:0]经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。根据 Vaddr，从存储空间取出指定地址的老值保存到寄存器 Ra 中，“+1”操作的操作数作为无符号数处理，出现溢出时按照无符号数的方式判整数溢出异常。

如果有效地址不自然对界，则产生地址对界异常。

该操作只能针对主存空间，如果访问 LDM 私有/共享空间，则产生越权访问异常。

该操作只能针对主存地址，访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.3.3.2 原子更新

格式： UPDT Va.rl,disp.ra(Rb.al) //杂项指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

(Vaddr)<63:0>←{原(Vaddr)<63:0>与Vav进行指定操作}

或(Vaddr)<31:0>←{原(Vaddr)<31:0>与Vav进行指定操作}

异常： 数据流异常

对界 （Alignment）

整数溢出（Integer Overflow）

助记符： UPDT 原子更新

指令描述：

对主存地址空间进行更新，对指定位置的值进行“原子更新”后更新到原位置，该操作不能被打断。

本指令中的 Va 操作数用于描述具体的更新操作。指令中 12 位偏移量 Disp[11:0]经符号扩展加上寄存器 Rb 的值形成虚地址 Vaddr。

Va 的格式如下图所示，其中：

- 更新粒度。1’b0 表示字操作，1’b1 表示长字操作；
- 更新类型。3’b010 表示加 n 读改写，3’b011 表示减 n 读改写；
- 有符号更新数据 n。字操作时，更新数据 n 是低 32bit 有效的有符号字；长字操作时，更新数据 n 是有符号长字。

127-64	63-4	3-1	0
有符号更新数据 n	保留	更新类型	更新粒度

更新操作的操作数作为有符号数处理，出现溢出时按照有符号数的方式判定整数溢出异常。

如果有效地址不自然对界，则产生地址对界异常。

该操作只能针对主存空间，如果访问 LDM 私有/共享空间，则产生越权访问异常。

该操作只能针对主存地址，访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.3.3.3 原子比较并转换

格式： CASx Va.wl,disp.ra(Rb.al) //杂项指令格式

操作：

```
flag_ok←1
flag_failed←0
Vaddr←{Rbv+SEXT(disp)}
CASE
    CASW：
        IF(Va<95:64>==(Vaddr)<31:0>)  THEN
            Vaddr←Va<159:128>
            Va<63:0>←flag_ok
        ELSE
            Va<63:0>←flag_failed
    CASL：
        IF(Va<127:64>==(Vaddr)<63:0>)  THEN
            Vaddr←Va<191:128>
            Va<63:0>←flag_ok
        ELSE
            Va<63:0>←flag_failed
```

ENDCASE

异常： 数据流异常

对界 （Alignment）

助记符： CASW 字比较并置换

CASL 长字比较并置换

指令描述：

比较并置换的含义是将 Old 值与主存中某个地址保存的值进行比较，如果相等则将主存中该值设定为 New 值，并返回从核交换成功标志；比较不相等时，则仅返回交换不成功标志。

指令中 12 位偏移量 Disp[11:0]经符号扩展加上寄存器 Rb 的值形成虚地址 Vaddr。比较并置换指令包括字类型和长字类型两种，其中 Va 即是源操作数，也是目标操作数，交换完成信息保留在 Va 的低 64bit 中，高位信息不变。具体含义如下：

- 交换成功标志：交换操作成功时该域为 1，否则为 0；
- OLD 值，用于与主存数据进行比较，字粒度时低 32bit 有效，长字粒度时 64bit 有效；
- NEW 值，用于比较匹配后的设置，字粒度时低 32bit 有效，长字粒度时 64bit 有效；

191-128	127-64	63-0
New 值	Old 值	交换成功标志

如果有效地址不自然对界，则产生地址对界异常。

该操作只能针对主存空间，如果访问 LDM 私有/共享空间，则产生越权访问异常。

该操作只能针对主存地址，访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.4 整数运算类指令（35 条）

整数运算类指令共 35 条，包括算术运算指令 23 条，移位指令 4 条，比较指令 5 条，条件选择指令 3 条。

6.4.1 算术运算指令（23 条）

算术运算指令包括加、减、乘、位计数等指令。

表 0-19 算术运算指令列表

序号	指令	寄存器格式操作码	立即数格式操作码	描述
1	addw Ra,Rb/#b,Rc	0x4.0.0	0x4.1.0	字加
2	subw Ra,Rb/#b,Rc	0x4.0.1	0x4.1.1	字减
3	addl Ra,Rb/#b,Rc	0x4.0.2	0x4.1.2	长字加

4	subl	Ra,Rb/#b,Rc	0x4.0.3	0x4.1.3	长字减
5	s4addl	Ra,Rb/#b,Rc	0x4.2.0	0x4.3.0	乘 4 的长字加
6	s4subl	Ra,Rb/#b,Rc	0x4.2.1	0x4.3.1	乘 4 的长字减
7	s8addl	Ra,Rb/#b,Rc	0x4.2.2	0x4.3.2	乘 8 的长字加
8	s8subl	Ra,Rb/#b,Rc	0x4.2.3	0x4.3.3	乘 8 的长字减
9	s32addl	Ra,Rb/#b,Rc	0x4.0.c	0x4.1.c	乘 32 的长字加
10	s32subl	Ra,Rb/#b,Rc	0x4.0.d	0x4.1.d	乘 32 的长字减
11	s64addl	Ra,Rb/#b,Rc	0x4.0.e	0x4.1.e	乘 64 的长字加
12	s64subl	Ra,Rb/#b,Rc	0x4.0.f	0x4.1.f	乘 64 的长字减
13	mulw	Ra,Rb/#b,Rc	0x4.2.4	0x4.3.4	字乘
14	umulw	Ra,Rb/#b,Rc	0x4.2.5	0x4.3.5	无符号字乘
15	mull	Ra,Rb/#b,Rc	0x4.2.6	0x4.3.6	长字乘
16	umulh	Ra,Rb/#b,Rc	0x4.2.7	0x4.3.7	无符号长字乘高位
17	zap	Ra,Rb/#b,Rc	0x4.6.4	0x4.7.4	字节清除
18	zapnot	Ra,Rb/#b,Rc	0x4.6.5	0x4.7.5	字节反码清除
19	sextb	Rb/#b,Rc	0x4.6.6	0x4.7.6	字节符号扩展
20	sexth	Rb/#b,Rc	0x4.6.7	0x4.7.7	半字符号扩展
21	ctpop	Rb,Rc	0x4.6.0	—	数“1”
22	ctlz	Rb,Rc	0x4.6.1	—	数头“0”
23	cttz	Rb,Rc	0x4.6.2	—	数尾“0”

6.4.1.1 字加

格式： ADDW Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

ADDW Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←SEXT((Rav+Rbv)<31:0>)

异常： 整数溢出（Integer Overflow）

助记符： ADDW 字加

指令描述：

Ra 加上 Rb（或 8 位无符号立即数），和的低 32 位进行符号扩展后送到 Rc。运算时 Ra 与 Rb 的高 32 位忽略。溢出检查基于 32 位字加的和。

6.4.1.2 长字加

格式： ADDL Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

ADDL Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←Rav+Rbv

异常： 整数溢出

助记符： ADDL //长字加

指令描述：

Ra 加上 Rb（或 8 位无符号立即数），和的低 64 位送到 Rc 中。溢出检查基于 64 位长字加的和。可用无符号比较指令来判断长字加操作的最高位进位，如果结果比 2 个加数（当作无符号数）都小，则有一个进位。

6.4.1.3 带扩展因子的长字加

格式： SxADDL Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

SxADDL Ra.rl,#b.rl,Rc.wl //简单运算指令格式

操作： CASE

S4ADDL: Rc←LEFT_SHIFT(Rav,2)+Rbv

S8ADDL: Rc←LEFT_SHIFT(Rav,3)+Rbv

S32ADDL: Rc←LEFT_SHIFT(Rav,5)+Rbv

S64ADDL: Rc←LEFT_SHIFT(Rav,6)+Rbv

ENDCASE

异常： 无

助记符： S4ADDL 乘4的长字加

S8ADDL 乘8的长字加

S32ADDL 乘32的长字加

S64ADDL 乘64的长字加

指令描述：

Ra 乘以 4、8、32 或 64，再加上 Rb（或 8 位无符号立即数），和写入 Rc。

6.4.1.4 字减

格式： SUBW Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

SUBW Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←SEXT((Rav–Rbv)<31:0>)

异常： 整数溢出（Integer Overflow）

助记符： SUBW //字减

指令描述：

Ra 减去 Rb（或 8 位无符号立即数），差的低 32 位经符号扩展后写入 Rc。运算时 Ra 和 Rb 的高 32 位忽略。溢出检查基于 32 位字减的差。

6.4.1.5 长字减

格式： SUBL Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

SUBL Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←Rav–Rbv

异常： 整数溢出（Integer Overflow）

助记符： SUBL 长字减

指令描述：

Ra 减去 Rb（或 8 位无符号立即数），差写入 Rc。溢出检查基于 64 位长字减的差。无符号的比较指令可用来判断长字减操作的最高位借位，如果无符号数 Rav 小于 Rbv，则产生一个借位。

6.4.1.6 带扩展因子的长字减

格式： SxSUBL Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

SxSUBL Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： CASE

S4SUBL: Rc←LEFT_SHIFT(Rav,2)–Rbv

S8SUBL: Rc←LEFT_SHIFT(Rav,3)–Rbv

S32SUBL: Rc←LEFT_SHIFT(Rav,5)–Rbv

S64SUBL: Rc←LEFT_SHIFT(Rav,6)–Rbv

ENDCASE

异常： 无

助记符： S4SUBL 乘4的长字减

S8SUBL 乘8的长字减

S32SUBL 乘32的长字减

S64SUBL 乘64的长字减

指令描述：

Ra 乘以 4、8、32 或 64，再减去 Rb（或 8 位无符号立即数），差的低 64 位写入 Rc。

6.4.1.7 字乘

格式： MULW Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

MULW Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←SEXT((Rav *Rbv)<31:0>)

异常： 整数溢出（Integer Overflow）

助记符： MULW 字乘

指令描述：

Ra 乘以 Rb（或 8 位无符号立即数），积的低 32 位进行符号扩展后写入 Rc。运算时 Ra 和 Rb 的高 32 位忽略。溢出检查基于 32 位字乘产生的 64 位乘积，如果乘积结果的高 32 位和第[31]位不为全 0 或全 1，认为发生整数溢出。发生整数溢出时，只取积的低 32 位，并符号扩展成 64 位写入 Rc。

6.4.1.8 无符号字乘

格式： UMULW Ra.rl,Rb.rw,Rc.wl //简单运算指令格式

UMULW Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←(U(Rav[31:0]) * U(Rbv[31:0]))<63:0>

异常： 无

助记符： UMULW 无符号字位乘

指令描述：

Ra 乘以 Rb（或 8 位无符号立即数）作无符号 32 位乘，将 64 位积写入 Rc。

6.4.1.9 长字乘

格式： MULL Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

MULL Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←(Rav[63:0] * Rbv[63:0])<63:0>

异常： 整数溢出（Integer Overflow）

助记符： MULL 长字乘

指令描述：

Ra 乘以 Rb（或 8 位无符号立即数），积的低 64 位写入 Rc。溢出检查基于长字乘产生的 128 位乘积，如果乘积的高 65 位不为全 0 或全 1，则发生整数溢出。

6.4.1.10 无符号长字乘高位

格式： UMULH Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

UMULH Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←(U(Rav[63:0] * U(Rbv[63:0]))<127:64>

异常： 无

助记符： UMULH 无符号长字乘高位

指令描述：

Ra 乘以 Rb（或 8 位无符号立即数）作无符号长字乘，将 128 位积的高 64 位写入 Rc。

该指令的一个特别用处是，可以用于产生有符号长字乘的高 64 位结果：（UMULH 结果）－Ra[63]*Rb[62:0]－Rb[63]*Ra[62:0]。其中 Ra、Rb 均为有符号数。

6.4.1.11 字节清除

格式： ZAPx Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

ZAPx Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： CASE

ZAP: Rc←BYTE_ZAP(Rav,Rbv<7:0>)

ZAPNOT: Rc←BYTE_ZAP(Rav,NOT Rbv<7:0>)

ENDCASE

异常： 无

助记符： ZAP 字节清除

ZAPNOT 字节反码清除

指令描述：

ZAP 和 ZAPNOT 指令可将寄存器 Ra 中指定的字节清为“0”，并将结果保存到寄存器 Rc 中。Rb 的低 8 位（或 8 位立即数）用来选择被清除的字节，其中 Rbv 的[0]位对应第 0 字节，[1]位对应第 1 字节，依此类推。若 Rbv 中相应位为“1”（ZAP 指令）或“0”（ZAPNOT 指令），则将 Ra 对应的字节清“0”。

6.4.1.12 符号扩展

格式： SEXTx Rb.rl,Rc.wl //简单运算指令格式

SEXTx #b.ib,Rc.wl //简单运算指令格式

操作： CASE

SEXTB: Rc←SEXT(Rbv<7:0>)

SEXTH: Rc←SEXT(Rbv<15:0>)

ENDCASE

异常： 无

助记符： SEXTB 字节符号扩展

SEXTH 半字符号扩展

指令描述：

将 Rb（或 8 位立即数）中的字节或半字符号扩展为 64 位，并写入结果寄存器 Rc 中。Ra 必须为 R31。

6.4.1.13 数头“0”

格式： CTLZ Rb.rl,Rc.wl //简单运算指令格式

操作：

```
temp←0
FOR(i=63;i>=0;i--)
    IF(Rbv<i>==1)  THEN  BREAK
    temp←temp+1
END
Rc<6:0>←temp<6:0>
Rc<63:7>←0
```

异常： 无

助记符： CTLZ 数头0

指令描述：

将 Rb 中从最高位起连续“0”的个数写入 Rc。Ra 必须为 R31。

6.4.1.14 数“1”

格式： CTPOP Rb.rl,Rc.wl //简单运算指令格式

操作：

```
temp←0
FOR(i=0;i<=63;i++)
    IF(Rbv<i>==1)  THEN  temp←temp+1
END
Rc<6:0>←temp<6:0>
Rc<63:7>←0
```

异常： 无

助记符： CTPOP 数“1”的个数

指令描述：

将 Rb 中“1”的个数写入 Rc。Ra 必须为 R31。

6.4.1.15 数尾“0”

格式： CTTZ Rb.rl,Rc.wl //简单运算指令格式

操作：

```
temp←0
FOR(i=0;i<=63;i++)
    IF(Rbv<i>==1)  THEN  BREAK
    temp←temp+1
```

END

Rc<6:0>←temp<6:0>

Rc<63:7>←0

异常： 无

助记符： CTTZ 数尾“0”

指令描述：

将 Rb 中从最低位起连续“0”的个数写入 Rc。Ra 必须为 R31。

6.4.2 移位指令（4 条）

移位指令包括逻辑移位、算术移位和循环移位等指令，均是针对64位长字进行的移位。

表 0-20 移位指令列表

序号	指令	寄存器格式操作码	立即数格式操作码	描述
1	sll Ra,Rb/#b,Rc	0x4.0.8	0x4.1.8	逻辑左移
2	srl Ra,Rb/#b,Rc	0x4.0.9	0x4.1.9	逻辑右移
3	sra Ra,Rb/#b,Rc	0x4.0.a	0x4.1.a	算术右移
4	rotl Ra,Rb/#b,Rc	0x4.0.b	0x4.1.b	循环左移

6.4.2.1 逻辑移位

格式： SxL Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

SxL Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： CASE

SLL： Rc←LEFT_SHIFT(Rav,Rbv<5:0>)

SRL： Rc←RIGHT_SHIFT(Rav,Rbv<5:0>)

ENDCASE

异常： 无

助记符： SLL 逻辑左移

SRL 逻辑右移

指令描述：

Ra 逻辑移若干位，移位位数由 Rb（或 8 位立即数#b）的低 6 位指定，结果写入 Rc。移出的空位用“0”填充。Rb（或 8 位立即数#b）的高位被忽略。

6.4.2.2 算术移位

格式： SRA Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

SRA Ra.rl,#b.rl,Rc.wl //简单运算指令格式

操作： Rc←ARITH_RIGHT_SHIFT(Rav,Rbv<5:0>)

异常： 无

助记符： SRA 算术右移

指令描述：

Ra 算术右移由 Rb（或 8 位立即数#b）的低 6 位指定的位数，结果写入 Rc。移出空位用 Rav 的符号位填充。Rb（或 8 位立即数#b）的高位被忽略。

6.4.2.3 循环移位

格式： ROTL Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

ROTL Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←LEFT_ROTATE(Rav,Rbv<5:0>) //ROTL

异常： 无

助记符： ROTL 循环左移

指令描述：

Ra 循环左移若干位，移位位数由 Rb（或 8 位立即数#b）的低 6 位指定，结果写入 Rc。移出位移进空出的位置。Rb（或 8 位立即数#b）的高位被忽略。

6.4.3 整数比较指令（5 条）

比较指令包括符号和无符号比较指令。

表 0-21 整数比较指令列表

序号	指令	寄存器格式操作码	立即数格式操作码	描述
1	cmpeq Ra,Rb/#b,Rc	0x4.2.9	0x4.3.9	有符号数等于比较
2	cmplt Ra,Rb/#b,Rc	0x4.2.a	0x4.3.a	有符号数小于比较
3	cmple Ra,Rb/#b,Rc	0x4.2.b	0x4.3.b	有符号数小于等于比较
4	cmpult Ra,Rb/#b,Rc	0x4.2.e	0x4.3.e	无符号数小于比较
5	cmpule Ra,Rb/#b,Rc	0x4.2.f	0x4.3.f	无符号数小于等于比较

6.4.3.1 带符号的整数比较

格式： CMPxx Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

CMPxx Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作： CASE

CMPEQ: IF(Rav==Rbv) THEN Rc←1 ELSE Rc←0

CMPLE: IF(Rav<=Rbv) THEN Rc←1 ELSE Rc←0

CMPLT: IF(Rav<Rbv) THEN Rc←1 ELSE Rc←0

ENDCASE

异常： 无

助记符：

CMPEQ	等于比较
CMPLE	小于等于比较
CMPLT	小于比较

指令描述：

对 Ra 和 Rb（或 8 位无符号立即数）的值进行符号比较，若满足指定关系，则 Rc 的值为“1”，否则为“0”。

6.4.3.2 无符号的整数比较

格式：

```
CMPUxx  Ra.rl,Rb.rl,Rc.wl   //简单运算指令格式
CMPUxx  Ra.rl,#b.ib,Rc.wl   //简单运算指令格式
```

操作：

CASE

CMPULT: IF(Rav<Rbv) THEN Rc←1 ELSE Rc←0

CMPULE: IF(Rav<=Rbv) THEN Rc←1 ELSE Rc←0

ENDCASE

异常： 无

助记符：

CMPULT	小于比较
CMPULE	小于等于比较

指令描述：

对 Ra 和 Rb（或 8 位无符号立即数）的值进行无符号比较，若满足指定关系，则 Rc 的值为“1”，否则为“0”。

6.4.4 整数条件选择指令（3 条）

按条件将不同的源操作数存入目标寄存器。

表 0-22 条件选择指令列表

序号	指令	寄存器格式操作码	立即数格式操作码	描述
1	seleq Ra,Rb,Rc/#c,Rd	0x4.a	0x4.b	若 Ra=“0”则选择数据 Rb 写入 Rd，否则选择 Rc/#c 写入 Rd

2	selle Ra,Rb,Rc/#c,Rd	0x4.e	0x4.f	若 Ra≤“0”则选择数据 Rb 写入 Rd，否则选择 Rc/#c 写入 Rd
3	sellt Ra,Rb,Rc/#c,Rd	0x4.c	0x4.d	若 Ra＜“0”则选择数据 Rb 写入 Rd，否则选择 Rc/#c 写入 Rd

格式： SELxx Ra.rl,Rb.rl,Rc.rl,Rd.wl //复合运算指令格式

SELxx Ra.rl,Rb,#c,Rd.wl //复合运算指令格式

操作： IF TEST(Rav,Condition_based_on_Opcode)

THEN Rd←Rbv

ELSE Rd←Rcv

异常： 无

助记符： SELEQ 若Ra＝“0”则选择数据Rb写入Rd，否则选择Rc/#c写入Rd

SELLT 若Ra＜“0”则选择数据Rb写入Rd，否则选择Rc/#c写入Rd

SELLE 若Ra≤“0”则选择数据Rb写入Rd，否则选择Rc/#c写入Rd

指令描述：

对Ra进行测试，若条件成立，则Rb的值被写入Rd，否则Rc（或6位立即数#c零扩展到64位）的值被写入Rd。

6.5 标量整理指令（2 条）

标量拼接指令主要用于配合不对界访存使用。

表 0-23 标量拼接指令列表

序号	指令	寄存器格式操作码	立即数格式操作码	描述
1	conl Ra,Rb,Rc,Rd	0x4.8		标量按字节拼接长字
2	insb Ra,Rb,#c,Rd		0x4.9	标量字节插入

6.5.1 标量拼接

格式： CONL Ra.rl,Rb.rl,Rc.wl,Rd.w1 //复合运算指令寄存器格式

操作：

```
temp←0
FOR(i=0;i<=7;i++)
    IF（i < Rc<2:0>）
        temp<(i+(8-Rc<2:0>))*8+7:(i+(8-Rc<2:0>))*8>←Ra(i*8+7:i*8);
    ELSE
        temp<(i-Rc<2:0>)*8+7:(i-Rc<2:0>)*8>←Rb(i*8+7:i*8);
END
```

Rd←temp

异常： 无

助记符： CONL 标量按字节拼接长字

指令描述：

标量按字节拼接长字，通常和LDL_U配合使用。其中Ra存放长字高段，Rb存放长字低段，Rc低3位存放拼接字节位置，如果Rc低3位为全0，则默认将低段Rb的值写入目标寄存器。

6.5.2 标量字节插入

格式： INSB Ra.rl,Rb.rl,#c.ib,Rd.wl //浮点复合运算指令格式

操作： temp←0

FOR(i=0;i<8;i++)

BEGIN

IF TEST(i==#c) THEN temp<8*i+7:8*i>←Rav<7:0>

ELSE temp<8*i+7:8*i>←Rbv<8*i+7:8*i>

END

Rd←temp

异常： 无

助记符： INSB 标量字节插入

指令描述：

操作数为 Ra、Rb、#c 和 Rd，其中#c 为 6 位立即数。功能是将寄存器 Ra 低 8 位字节，替代 Rb 中由#c 低 3 位（有效值为 0~7）指定的 8 位字节元素，组成新的长字整数写入 Rd。#c 的[5:3]位被忽略。

6.6 可重构指令（2 条）

可重构指令是可重构逻辑运算指令的简称，包括两操作数可重构指令和三操作数可重构指令，可实现操作数之间的任意逻辑运算。

表 0-24 可重构指令列表

序号	指令	寄存器格式操作码	立即数格式操作码	描述
1	log2x<truth> Ra,Rb/#b,Rc	0x4.4.0~f	0x4.5.0~f	两操作数任意逻辑运算
2	log3r<idx> Ra,Rb,Rc,Rd	0x6	—	三操作数任意逻辑运算

6.6.1 两操作数可重构指令 log2x

两操作数可重构指令，通过把两个操作数的任意逻辑运算的真值组合作为操作码的一部分来重构逻辑指令的功能，以实现更加灵活的逻辑运算，并减少运算量。虽然只有一条指令，但是由于两个操作数逻辑运算的真值有16（2^4）种可能，因此有16种操作码形式。

两操作数可重构指令的操作码以“log2”开头，后面跟以“x”起始的16进制表示的真值组合。由于两个操作数的真值表共有4项，如下表所示，运算结果用m到p的4个字母表示，则真值组合为4’bponm（自底向上取）。如某一种两操作数逻辑运算的真值组合为4’b1100，16进制表示为0xc，则该逻辑运算的可重构指令的操作码为“log2xc”。

表 0-25 两操作数逻辑运算真值表

X	Y	F(X,Y)
0	0	m
0	1	n
1	0	o
1	1	p

格式： LOG2X<TRUTH> Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

LOG2X<TRUTH>Ra.rl,#b.ib,Rc.wl //简单运算指令格式

操作：

```
temp←0
FOR(i=0;i<=63;i++)
BEGIN
    temp<i>←TRUTH_MAP(Rav<i>,Rbv<i>,TRUTH)
END
Rc←temp
```

异常： 无

助记符： LOG2X<TRUTH> 两操作数可重构指令

指令描述：

将 Ra 和 Rb（或 8 位立即数零扩展到 64 位）中对应的每一位，组成 2 位的二进制数，从指令的<TRUTH>域的 4 位二进制值中选出一位，写入 Rc 的对应位中。

原有的两操作数逻辑运算指令到 log2x 的真值映射如下所示。由于两操作数有 4 位的真值，共 16 种情况，而现有逻辑运算只对应 6 种情况，其它情况对应其它逻辑运算，比如 log2x7 可以映射到先与后非运算（如~(a&b)）：

表 0-26 常见逻辑运算对应的可重构指令

助记符	**逻辑运算**	**log2x**
AND	Rav&Rbv	log2x8
BIC	Rav&(~Rbv)	log2x4
BIS	Rav|Rbv	log2xe

ORNOT	Rav|(~Rbv)	log2xd
XOR	Rav^Rbv	log2x6
EQV	(Rav&Rbv)|(~Rav&~Rbv)	log2x9

6.6.2 三操作数可重构指令 log3r

三操作数可重构指令的思想，是通过显式提供三操作数的任意逻辑运算的真值组合，来重构逻辑指令的功能，以实现更加灵活的逻辑运算，并减少运算量。三个操作数的真值表共有8项，如下表所示，运算结果用m到t的8个字母表示，则真值组合为8'btsrqponm（自底向上取）。

表 0-27 三操作数逻辑运算真值表

X	Y	Z	F(X,Y,Z)
0	0	0	m
0	0	1	n
0	1	0	o
0	1	1	p
1	0	0	q
1	0	1	r
1	1	0	s
1	1	1	t

三个操作数逻辑运算的真值有256（2^8）种可能，如果直接把真值组合作为操作码的一部分，需要256种操作码形式。基于节省指令编码空间的考虑，SW-AI处理器从核提供16个8位真值表配置寄存器，将配置寄存器的索引IDX作为操作码的一部分。

三操作数可重构指令的操作码以"log3"开头，后面跟以"r"起始的10进制表示的真值表配置寄存器索引。每个真值表配置寄存器是一个8位的内部寄存器，可以通过rcsr/wcsr指令进行访问。

格式： LOG3R<IDX> Ra.rl,Rb.rl,Rc.rl,Rd.wl //浮点复合运算指令格式

操作：

```
temp←0
TRUTH←真值表配置寄存器<IDX>
FOR(i=0;i<=63;i++)
BEGIN
    temp<i>←TRUTH_MAP(Rav<i>,Rbv<i>,Rcv<i>,TRUTH)
END
Rd←temp
```

异常： 无

助记符： LOG3R<IDX> 三操作数可重构指令

指令描述：

FUNC[3:0]用作 IDX，根据 IDX，从配置寄存器取出真值表 TRUTH，然后 Ra、Rb、Rc 每一个对应的位分别进行由 TRUTH 指定的逻辑运算，结果保存到 Rd 中。SW-AI 从核提供 16 个真值表配置寄存器（与 VLOG3R 指令共用），IDX 有效值为 0~15。

6.7 单双精度浮点运算类指令（38 条）

单双精度浮点运算指令包含采用简单运算指令格式的 27 条指令，以及采用复合运算指令格式的 11 条指令。只有 FCVTDLR 使用立即数格式。

6.7.1 算术运算指令（10 条）

实现基本的浮点（单/双精度）加、减、乘、除和开平方根运算。

表 0-28 单双精度浮点算术运算指令列表

序号	指令		操作码	描述
1	fadds	Ra,Rb,Rc	0x8.0.0	SP 加
2	faddd	Ra,Rb,Rc	0x8.0.1	DP 加
3	fsubs	Ra,Rb,Rc	0x8.0.2	SP 减
4	fsubd	Ra,Rb,Rc	0x8.0.3	DP 减
5	fmuls	Ra,Rb,Rc	0x8.0.4	SP 乘
6	fmuld	Ra,Rb,Rc	0x8.0.5	DP 乘
7	fdivs	Ra,Rb,Rc	0x8.1.8	SP 除
8	fdivd	Ra,Rb,Rc	0x8.1.9	DP 除
9	fsqrts	Rb,Rc	0x8.1.a	SP 平方根
10	fsqrtd	Rb,Rc	0x8.1.b	DP 平方根

6.7.1.1 浮点加

格式： FADDx Ra.rf,Rb.rf,Rc.wf //简单运算指令格式

操作： Rc←Rav+Rbv

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FADDS SP浮点加

FADDD DP浮点加

指令描述：

Ra 加上 Rb，结果写入 Rc。结果被舍入到指定的精度，并检查上溢和下溢。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.1.2 浮点减

格式： FSUBx Ra.rf,Rb.rf,Rc.wf //简单运算指令格式

操作： Rc←Rav–Rbv

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FSUBS SP浮点减

FSUBD DP浮点减

指令描述：

Rb 为减数，Ra 为被减数，差写入 Rc。差先舍入到指定精度，然后在相应范围内检查上溢和下溢。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.1.3 浮点乘

格式： FMULx Ra.rf,Rb.rf,Rc.wf //简单运算指令格式

操作： Rc←Rav*Rbv

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FMULS SP浮点乘

FMULD DP浮点乘

指令描述：

Rb 为乘数，Ra 为被乘数，乘积写入 Rc。乘积先舍入到指定精度，然后在相应范围内检查上溢和下溢。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.1.4 浮点除

格式： FDIVx Ra.rf,Rb.rf,Rc.wf //简单运算指令格式

操作： Rc←Rav/Rbv

异常： 无效操作（Invalid Operation）

除数为“0”（Divided by Zero）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FDIVS SP浮点除

FDIVD DP浮点除

指令描述：

Rb 为除数，Ra 为被除数，商写入 Rc。商先舍入到指定精度，然后在相应范围内检查上溢和下溢。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.1.5 浮点平方根

格式： FSQRTx Rb.rf,Rc.wf //简单运算指令格式

操作： Rc←Rbv**(1/2)

异常： 无效操作（Invalid Operation）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FSQRTS SP浮点平方根

FSQRTD DP浮点平方根

指令描述：

Rb 的平方根写入 Rc。指令中 Ra 域可以为任意值（最好为 R31）。结果先舍入到指定精度。如果 Rb 小于“0”，则产生无效操作自陷。对“－0”开平方根的结果为“－0”。该指令不会出现上溢和下溢出。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.2 比较指令（4 条）

实现基本的浮点比较功能。

表 0-29 比较指令列表

序号	指令	操作码	描述
1	fcmpeq Ra,Rb,Rc	0x8.0.d	浮点等于比较

2	fcmple Ra,Rb,Rc	0x8.0.f	浮点小于等于比较
3	fcmplt Ra,Rb,Rc	0x8.0.e	浮点小于比较
4	fcmpun Ra,Rb,Rc	0x8.0.c	浮点不可排序比较

格式： FCMPxx Ra.rd,Rb.rd,Rc.wd //简单运算指令格式

操作： IF Rav SIGNED_RELATION Rbv THEN

Rc←0x4000000000000000

ELSE

Rc←0x0000000000000000

异常： 无效操作（Invalid Operation）

非规格化数异常（Denormal）

助记符：

FCMPEQ	浮点等于比较
FCMPLE	浮点小于等于比较
FCMPLT	浮点小于比较
FCMPUN	浮点无序（Unordered）比较

指令描述：

将 Ra 和 Rb 中的操作数相比较，如果条件成立，则将非“0”浮点值（2.0）写入 Rc 中，否则将真“0”写入 Rc 中。比较是精确的，不会产生上溢和下溢。当两个操作数中任意一个为 NaN 时，无序关系为真。比较时忽略“0”的符号位，因此“+0”与“–0”是相等的。另外，与+∞、－∞的比较正常进行，不会产生无效操作异常。

浮点比较指令没有明确说明参与比较的浮点操作数数据格式，既可以用于双精度格式数据之间的比较，也可以用于 64 位扩展单精度格式数据之间的比较。用户必须保证不会进行不同数据格式操作数之间的比较，否则可能导致不可预知的结果。

6.7.3 转换指令（8 条）

浮点转换指令包含浮点单双精度的转换、整数到浮点和浮点到整数的转换。

表 0-30 转换指令列表

序号	指令	操作码	描述
1	fcvths Rb,Rc	0x8.1.7	半精度转单精度
2	fcvtsd Rb,Rc	0x8.1.1	单精度转双精度
3	fcvtds Rb,Rc	0x8.1.0	双精度转单精度
4	fcvtsh Rb,Rc	0x8.1.6	单精度转半精度
5	fcvtls Rb,Rc	0x8.1.2	长字转单精度
6	fcvtld Rb,Rc	0x8.1.3	长字转双精度
7	fcvtdl Rb,Rc	0x8.1.4	双精度转长字整数

8	fcvtdlr Ra,#b,Rc	0x8.1.5	双精度转长字整数，可自带舍入模式

6.7.3.1 半精度转换为单精度

格式： FCVTHS Rb.rh,Rc.rd //简单运算指令格式

操作： Rc←{conversion of Rbv<63:0>}

异常： 无效操作（Invalid Operation）

非规格化数异常（Denormal）

助记符： FCVTHS //将HP浮点数转化成SP浮点数

指令描述：

Rb 中的 64 位扩展 HP 浮点数（EHP64）被转换成 64 位扩展 SP 浮点数（ESP64），并写入 Rc。Ra 必须为 R31。从 HP 浮点到 SP 浮点的转换是精确、无舍入的，是有限值的恒等转换。无上溢、下溢和非精确结果自陷产生。

SW-AI 处理器半精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.3.2 单精度转换为双精度

格式： FCVTSD Rb.rs,Rc.rd //简单运算指令格式

操作： Rc←{conversion of Rbv<63:0>}

异常： 无效操作（Invalid Operation）

非规格化数异常（Denormal）

助记符： FCVTSD //将SP浮点数转化成DP浮点数

指令描述：

Rb 中的 64 位扩展 SP 浮点数（ESP64）被转换成 DP 浮点数，并写入 Rc。Ra 必须为 R31。从 SP 浮点到 DP 浮点的转换是精确、无舍入的，是有限值的恒等转换。无上溢、下溢和非精确结果自陷产生。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.3.3 双精度转换为单精度

格式： FCVTDS Rb.rd,Rc.ws //简单运算指令格式

操作： Rc←{conversion of Rbv<63:0>}

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FCVTDS 将DP浮点数转化成SP浮点数

指令描述：

Rb 中的 DP 浮点数被转换成 SP 浮点数并写入 Rc。Ra 必须为 R31。

6.7.3.4 单精度转换为半精度

格式： FCVTSH Rb.rd,Rc.ws //简单运算指令格式

操作： Rc←{conversion of Rbv<63:0>}

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FCVTSH 将DP浮点数转化成HP浮点数

指令描述：

Rb 中的 64 位扩展 SP 浮点数（ESP64）被转换成 64 位扩展 HP 浮点数（EHP64）并写入 Rc。Ra 必须为 R31。

6.7.3.5 长字转浮点

格式： FCVTLx Rb.rl,Rc.wf //简单运算指令格式

操作： Rc←{conversion of Rbv<63:0>}

异常： 非精确结果（Inexact Result）

助记符：

FCVTLS 将长字整数转化成SP浮点数

FCVTLD 将长字整数转化成DP浮点数

指令描述：

将 Rb 中以补码表示的二进制整数转换成 64 位扩展单精度（ESP64）或双精度浮点数，并写入 Rc。该转换进行补码计算（若是负数）、规格化、舍入到指定精度，然后用恰当的符号位和指令域进行表示。指令中 Ra 必须为 R31。

6.7.3.6 双精度转换为长字整数

格式： FCVTDL Rb.rd,Rc.wl //简单运算指令格式

操作： Rc←{conversion of Rbv<63:0>}

异常： 无效操作（Invalid Operation）

非精确结果（Inexact Result）

浮点整数溢出（Integer Overflow）

非规格化数异常（Denormal）

助记符： FCVTDL 将DP浮点数转化成长字整数

指令描述：

将 Rb 中的操作数转换成补码格式的二进制整数并写入 Rc。转换时将尾数与位[0]右侧的小数点对齐，然后按 FPCR 指定的方法进行舍入。指令中 Ra 必须为 R31。

6.7.3.7 双精度转换为长字整数，自带舍入模式

格式： FCVTDLR Ra.rd,#b.ib,Rc.wl //简单运算指令格式

操作： Rc←{conversion of Rav<63:0>}

异常： 无效操作（Invalid Operation）

非精确结果（Inexact Result）

浮点整数溢出（Integer Overflow）

非规格化数异常（Denormal）

助记符： FCVTDLR 将DP浮点数转化成长字整数，自带舍入模式

指令描述：

将 Ra 中的操作数转换成补码格式的二进制整数并写入 Rc，支持自带舍入模式。转换时将尾数与位[0]右侧的小数点对齐，然后按 8 位立即数#b 的[2:0]位所指定的方法进行舍入。#b 的[7:3]位被忽略。

表 0-31 FCVTDLR 指令立即数（舍入模式）含义

#b[2]	#b[1:0]	含义
1	2’b00	向“0”舍入
1	2’b01	向负无穷大舍入
1	2’b10	常规舍入
1	2’b11	向正无穷大舍入
0	任意	按照 FPCR 中舍入模式位指定的方式舍入

6.7.4 符号拷贝指令（3 条）

符号拷贝指令采用简单运算指令格式，共 3 条指令。

表 0-32 符号拷贝指令列表

序号	指令	操作码	描述
1	fcpys Ra,Rb,Rc	0x8.0.8	拷贝符号
2	fcpyse Ra,Rb,Rc	0x8.0.9	拷贝符号和指数

3	fcpysn Ra,Rb,Rc	0x8.0.a	拷贝符号的反码

6.7.4.1 拷贝符号

格式： FCPYSx Ra.rl,Rb.rl,Rc.wl //简单运算指令格式

操作： CASE

FCPYS: Rc ← Rav<63> || Rbv<62:0>

FCPYSE: Rc ← Rav<63:52> || Rbv<51:0>

FCPYSN: Rc ← NOT(Rav<63>) || Rbv<62:0>

ENDCASE

异常： 无

助记符： FCPYS 拷贝符号

FCPYSE 拷贝符号和指数

FCPYSN 拷贝符号取反

指令描述：

对于 FCPYS 或 FCPYSN，取 Ra 的符号位或符号的反码，与 Rb 的指数和尾数拼接起来，存入 Rc。对于 FCPYSE，取 Ra 的符号位和指数位与 Rb 的尾数位拼接起来存入 Rc。不检查操作数的合法性。

使用 FCPYSRx,Rx,Ry 可以实现寄存器间的传送功能，利用 FCPYSR31,Rx,Ry 可以实现浮点绝对值功能，利用 FCPYSNRx,Rx,Ry 可以实现浮点数符号取反的功能，利用 FCPYSE 可以将浮点值比例缩放到某个已知的范围。

6.7.5 浮点条件选择指令（3 条）

浮点条件选择指令采用浮点复合运算指令格式，有四个操作数，实现浮点数的有条件选择。

表 0-33 浮点条件选择指令列表

序号	指令	操作码	描述
1	fseleq Ra,Rb,Rc,Rd	0x8.5	若 Ra＝“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd
2	fsellt Ra,Rb,Rc,Rd	0x8.6	若 Ra＜“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd
3	fselle Ra,Rb,Rc,Rd	0x8.7	若 Ra≤“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd

6.7.5.1 条件判断

格式： FSELxx Ra.rl,Rb.rl,Rc.rl,Rd.wl //浮点复合运算指令格式

操作： IF TEST(Rav,Condition_based_on_Opcode)

THEN Rd←Rbv

ELSE Rd←Rcv

异常： 无

助记符： FSELEQ 若Ra＝“0”则选择数据Rb写入Rd，否则选择Rc写入Rd

FSELLT 若Ra＜“0”则选择数据Rb写入Rd，否则选择Rc写入Rd

FSELLE 若Ra≤“0”则选择数据Rb写入Rd，否则选择Rc写入Rd

指令描述：

对Ra进行测试，若条件成立，则Rb的值被写入Rd，否则Rc的值被写入Rd。

SW-AI 处理器在实现浮点条件选择指令时，直接根据 Ra 的“[63]位值”和“[62:0]位是否为全 0”来进行条件判断。在 Ra 为非数时，条件判断的结果和“Ra 与 0 比较”的结果不一致，详细说明见附录 5“浮点条件判断与浮点比较的区别”。

6.7.6 乘加指令（8 条）

实现浮点（单/双精度）三操作数的乘加运算。

表 0-34 乘加指令列表

序号	指令	操作码	描述
1	fmas Ra,Rb,Rc,Rd	0x8.8	SP 乘加
2	fmad Ra,Rb,Rc,Rd	0x8.9	DP 乘加
3	fmss Ra,Rb,Rc,Rd	0x8.a	SP 乘减
4	fmsd Ra,Rb,Rc,Rd	0x8.b	DP 乘减
5	fnmas Ra,Rb,Rc,Rd	0x8.c	SP 负乘加
6	fnmad Ra,Rb,Rc,Rd	0x8.d	DP 负乘加
7	fnmss Ra,Rb,Rc,Rd	0x8.e	SP 负乘减
8	fnmsd Ra,Rb,Rc,Rd	0x8.f	DP 负乘减

6.7.6.1 浮点乘加

格式： FMAx Ra.rf,Rb.rf,Rc.rf,Rd.wf //浮点复合运算指令格式

操作： Rd←Rav*Rbv+Rcv**异常：** 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FMAS SP浮点乘加

FMAD DP浮点乘加

指令描述：

Ra 与 Rb 相乘后再与 Rc 相加，最后结果写入 Rd。不对中间结果进行舍入，也不作上/下溢检查。只对最终的结果进行舍入，然后再做上/下溢检查。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.6.2 浮点乘减

格式： FMSx Ra.rf,Rb.rf,Rc.rf,Rd.wf //浮点复合运算指令格式

操作： Rd←Rav*Rbv–Rcv

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FMSS SP浮点乘减

FMSD DP浮点乘减

指令描述：

Ra 与 Rb 相乘后再减去 Rc，最后结果写入 Rd。不对中间结果进行舍入，也不作上/下溢检查。只对最终的结果进行舍入，然后再做上/下溢检查。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.6.3 浮点负乘加

格式： FNMAx Ra.rf,Rb.rf,Rc.rf,Rd.wf //浮点复合运算指令格式

操作： Rd←－Rav*Rbv+Rcv

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FNMAS SP浮点负乘加

FNMAD DP浮点负乘加

指令描述：

Ra 与 Rb 相乘后对乘积取负数，再与 Rc 相加，最后结果写入 Rd。不对中间结果进行舍入，也不作上/下溢检查。只对最终的结果进行舍入，然后再做上/下溢检查。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.6.4 浮点负乘减

格式： FNMSx Ra.rf,Rb.rf,Rc.rf,Rd.wf //浮点复合运算指令格式

操作： Rd←－Rav*Rbv–Rcv

操作： Rd←－Rav*Rbv–Rcv

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： FNMSS SP浮点负乘减

FNMSD DP浮点负乘减

指令描述：

Ra 与 Rb 相乘后对乘积取负数，再减去 Rc，最后结果写入 Rd。不对中间结果进行舍入，也不作上/下溢检查。只对最终的结果进行舍入，然后再做上/下溢检查。

SW-AI 处理器单精度浮点不支持非规格化数，输入数据存在非规格化数时，直接当同符号 0 处理。

6.7.7 读写浮点控制寄存器指令（2 条）

读写浮点控制寄存器指令采用简单运算指令格式，共有 2 条指令，如下表所示。

表 0-35 读写浮点控制寄存器指令列表

序号	指令	操作码	描述
1	rfpcr Vc	0x8.1.c	读浮点控制寄存器 FPCR
2	wfpcr Va	0x8.1.d	写浮点控制寄存器 FPCR

格式： RFPCR Vc.wl //简单运算指令格式

WFPCR Va.rl //简单运算指令格式

操作： CASE

RFPCR: Vc←FPCR

WFPCR: FPCR←Vav

ENDCASE

异常： 无

助记符： RFPCR 读浮点控制寄存器

WFPCR 写浮点控制寄存器

指令描述：

读出浮点控制寄存器的值，并传送到寄存器 Vc 的低 128 位。或将寄存器 Va 低 128 位的值写入浮

点控制寄存器。

为了保证在正确的时刻读出或写入，本指令会停顿在流水线 0 发射站上，直到流水线 0 的执行站排空（之前的所有浮点指令全部结束）。

6.8 SIMD 扩展指令（70 条）

SIMD扩展指令包括7类指令：对界SIMD访存指令16条、整数SIMD运算指令22条、整数SIMD可重构指令2条、浮点SIMD运算指令19条，SIMD整理指令11条。

6.8.1 SIMD 访存指令（16 条）

SIMD访存指令包括2类指令：对界SIMD访存指令8条、不对界SIMD访存指令8条。

6.8.1.1 对界 SIMD 访存指令（8 条）

对界SIMD访存指令采用存储器指令格式，共8条指令。

表 0-36 对界 SIMD 访存指令列表

序号	指令	操作码	描述
1	vfldh Va,disp(Rb)	0xd.0	装入 HP 浮点向量
2	vfsth Va,disp(Rb)	0xd.1	存储 HP 浮点向量
3	vldw Va,disp(Rb)	0xd.2	装入字向量或 SP 浮点向量
4	vstw Va,disp(Rb)	0xd.39	存储字向量或 SP 浮点向量
5	vldh Va,disp(Rb)	0xd.4	装入 16 个半字整数向量，每个半字分量符号扩展为 32 位字
6	vsth Va,disp(Rb)	0xd.5	存储 16 个半字整数向量
7	vldb Va,disp(Rb)	0xd.6	装入 32 个字节整数向量，每个字节分量符号扩展为 16 位半字
8	vstb Va,disp(Rb)	0xd.7	存储 32 个字节整数向量

Disp[2:0]位置用作功能码FUNC[2:0]。

从核进行对界SIMD存储访问时，可能产生以下异常，大致可分为两类：

一类是对界异常：要求存储器访问时的地址按访问宽度对界。如果地址不对界，则产生对界异常。

一类是数据流异常，主要包括以下几种情况：

1）从核根据完成地址计算之后的目标地址的高段来确定访问空间类型（LDM 私有/共享空间），有关虚地址指示空间类型的划分参考《SW-AI 处理器软件接口手册》。如果目标地址没有落在合法的访问空间，或者对只读空间进行写操作，则产生越权访问异常。

2）从核访问 LDM 私有/共享空间过程中，如果超出对应空间容量，则发生越界异常。LDM 全 0

地址访问使能（CSR_LDM_CFG[0]）未打开（即取值为 1’b0）时，如果访问 LDM 0 地址，即不论何种访问粒度，只要访问地址为 0，则产生地址保护的越权访问异常；

3）从核访问主存过程中，在进行 SDLB 表代换时，可能产生地址越权或越界异常，详见《SW-AI 处理器软件接口手册》。

4）从核在进行主存访问过程中，可能产生主存地址溢出异常，详见《SW-AI 处理器软件接口手册》。

5）从核在进行主存访问过程中，如果对断连的核组进行访问，可能产生核组断连访问异常，详见《SW-AI 处理器软件接口手册》。

6.8.1.1.1 HP 浮点向量装入

格式： VFLDH Va.ws,disp.ab(Rb.ab) //存储器指令格式

操作： temp←0

Vaddr←{Rbv+SEXT(disp)}

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>← MAP_H2S((Vaddr)<16*i+15:16*i+0>)

END

Va←temp

异常： 数据流异常

对界 （Alignment）

助记符： VFLDH 装入HP浮点向量

指令描述：

Disp[15:0]为立即数字节偏移地址[15:5]位，偏移地址低 5 位[4:0]默认为 0（偏移地址低 5 位[4:0]非全 0 由软件编译静态检查）。16 位偏移量{Disp[15:5], 5’h0}经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。VFLDH 从存储器中取出一个 HP 浮点向量，将其中的 16 个 HP 浮点分别写到向量寄存器 Va 的 16 个 32 位元素位置中。在数据装入过程中，将 5 位存储器格式的指数扩展为 8 位寄存器格式的指数，尾数部分进行“0”扩展。

VFLDH 指令要求有效地址按 32 字节自然对界，否则将产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.8.1.1.2 HP 浮点向量存储

格式： VFSTH Va.rs,disp.ab(Rb.ab) //存储器指令格式

操作： temp←0

Vaddr←{Rbv+SEXT(disp)}

FOR(i=0;i<16;i++)

BEGIN

temp<16*i+15:16*i>←Va<32*i+31:32*i+30> || Va<32*i+26:32*i+13>

END

(Vaddr)<255:0>←temp

异常： 数据流异常

对界 （Alignment）

助记符： VFSTH HP浮点向量存储

指令描述：

Disp[15:5]为立即数字节偏移地址[15:5]位，偏移地址低 5 位[4:0]默认为 0（偏移地址低 5 位[4:0]非全 0 由软件编译静态检查）。16 位偏移量{Disp[15:5], 5'h0}经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。VFSTH 指令将向量寄存器 Va 的 16 个 32 位元素位置中的 HP 浮点同时存储到存储器中。取自寄存器 Va 中的源操作数需要重新排序以符合 HP 浮点的存储器格式，然后将重新排序后的结果写入存储器中。忽略 Va 的 16 个 32 位寄存器中的<29:27>和<12:0>位，但并不检查这些位的合法性。

VFSTH 指令要求有效地址按 32 字节自然对界，否则将产生对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.8.1.1.3字向量或 SP 浮点向量装入

格式： VLDW Va.wd,disp.ab(Rb.ab) //存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

Va<511:0>←(Vaddr)<511:0>

异常： 数据流异常

对界 （Alignment）

助记符： VLDW 装入SP浮点向量

指令描述：

Disp[15:6]为立即数字节偏移地址[15:6]位，偏移地址低 6 位[5:0]默认为 0（偏移地址低 6 位[5:0]非全 0 由软件编译静态检查）。16 位偏移量{Disp[15:6], 6'h0}经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。VLDW 从存储器中取出 16 个字向量，分别写到向量寄存器 Va 的 16 个 32 位元素位置中。

VLDW 指令也用于将 512 位的 SP 浮点向量从存储器载入到寄存器中。

VLDW 指令要求有效地址按 64 字节自然对界，否则将产生对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.8.1.1.4字向量或 SP 浮点向量存储

格式： VSTW Va.rd,disp.ab(Rb.ab) //存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

(Vaddr)<511:0>←Va<511:0>

异常： 数据流异常

对界 （Alignment）

助记符： VSTW SP浮点向量存储

指令描述：

Disp[15:6]为立即数字节偏移地址[15:6]位，偏移地址低 6 位[5:0]默认为 0（偏移地址低 6 位[5:0]非全 0 由软件编译静态检查）。16 位偏移量{Disp[15:6], 6'h0}经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。VSTW 指令将向量寄存器 Va 的 16 个 32 位元素位置中的字向量写入存储器中。

VSTW 指令也用于将 512 位的 SP 浮点向量从寄存器写到存储器中。

VSTW 指令要求有效地址按 64 字节自然对界，否则将产生对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.8.1.1.5半字向量装入并符号扩展

格式： VLDH Va.wd,disp.ab(Rb.ab) //存储器指令格式

操作： temp←0

Vaddr←{Rbv+SEXT(disp)}

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←SEXT((Vaddr)<16*i+15:16*i+0>)

END

Va<511:0>←temp

异常： 数据流异常

对界 （Alignment）

助记符： VLDH 装入半字向量并符号扩展

指令描述：

Disp[15:5]为立即数字节偏移地址[15:5]位，偏移地址低 5 位[4:0]默认为 0（偏移地址低 5 位[4:0]非全 0 由软件编译静态检查）。16 位偏移量{Disp[15:5], 5'h0}经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。VLDH 从存储器中取出 16 个半字向量，进行符号扩展后，分别写到向量寄存器 Va 的 16 个 32 位元素位置中。

VLDH 指令要求有效地址按 32 字节自然对界，否则将产生对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.8.1.1.6 半字向量存储

格式： VSTH Va.rd,disp.ab(Rb.ab) //存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

FOR(i=0;i<16;i++)

BEGIN

(Vaddr)<16*i+15:16*i+0>←Va<32*i+15:32*i+0>

END

异常： 数据流异常

对界 （Alignment）

助记符： VSTH 半字向量存储

指令描述：

Disp[15:5]为立即数字节偏移地址[15:5]位，偏移地址低 5 位[4:0]默认为 0（偏移地址低 5 位[4:0]非全 0 由软件编译静态检查）。16 位偏移量{Disp[15:5], 5'h0}经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。VSTH 指令将向量寄存器 Va 的 16 个 32 位元素位置中的低位半字分量写入存储器中。

VSTH 指令要求有效地址按 32 字节自然对界，否则将产生对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.8.1.1.7 字节向量扩展装入并符号扩展

格式： VLDB Va.wd,disp.ab(Rb.ab) //存储器指令格式

操作： temp←0

Vaddr←{Rbv+SEXT(disp)}

FOR(i=0;i<32;i++)

BEGIN

temp<16*i+15:16*i>←SEXT((Vaddr)<8*i+7:8*i+0>)

END

Va<511:0>←temp

异常： 数据流异常

对界 （Alignment）

助记符： VLDB 装入字节向量并符号扩展

指令描述：

Disp[15:5]为立即数字节偏移地址[15:5]位，偏移地址低 5 位[4:0]默认为 0（偏移地址低 5 位[4:0]非全 0 由软件编译静态检查）。16 位偏移量{Disp[15:5], 5'h0}经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。VLDB 从存储器中取出 32 个字节向量，进行符号扩展后，分别写到向量寄存器 Va 的 32 个 16 位元素位置中。

VLDB 指令要求有效地址按 32 字节自然对界，否则将产生对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.8.1.1.8字节向量存储

格式： VSTB Va.rd,disp.ab(Rb.ab) //存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

FOR(i=0;i<32;i++)

BEGIN

(Vaddr)<8*i+7:8*i+0>←Va<16*i+7:16*i+0>

END

异常： 数据流异常

对界 （Alignment）

助记符： VSTB 字节向量存储

指令描述：

Disp[15:5]为立即数字节偏移地址[15:5]位，偏移地址低 5 位[4:0]默认为 0（偏移地址低 5 位[4:0]非全 0 由软件编译静态检查）。16 位偏移量{Disp[15:5], 5'h0}经符号扩展加上寄存器 Rb 的值形成有效地址 Vaddr。VSTB 指令将向量寄存器 Va 的 32 个 16 位元素位置中的低位字节分量写入存储器中。

VSTB 指令要求有效地址按 32 字节自然对界，否则将产生对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》。

6.8.1.2 不对界 SIMD 访存指令（8 条）

不对界SIMD访存指令采用杂项存储器指令格式，共8条指令。

表 0-37 不对界 SIMD 访存指令列表

序号	指令	操作码	描述
1	vfldh_u Va,disp(Rb)	0xa.0	装入不对界 HP 浮点向量
2	vldb_u Va,disp(Rb)	0xa.1	装入不对界字节整数向量
3	vldh_u Va,disp(Rb)	0xa.2	装入不对界半字整数向量
4	vldw_u Va,disp(Rb)	0xa.3	装入不对界字整数/SP 浮点向量
5	vfsth_u Va,disp(Rb)	0xa.8	存储不对界 HP 浮点向量

6	vstb_u Va,disp(Rb)	0xa.9	存储不对界字节整数向量
7	vsth_u Va,disp(Rb)	0xa.a	存储不对界半字整数向量
8	vstw_u Va,disp(Rb)	0xa.b	存储不对界字整数/SP 浮点向量

从核进行不对界SIMD存储访问时，可能产生以下异常，大致可分为两类：

一类是对界异常：要求存储器访问时的地址按分量数据粒度对界。如果地址不对界，则产生对界异常。

一类是数据流异常，主要包括以下情况：

1）从核根据完成地址计算之后的目标地址的高段来确定访问空间类型（LDM 私有/共享空间、主存空间），有关虚地址指示空间类型的划分参考《SW-AI 处理器软件接口手册》。如果目标地址没有落在合法的访问空间，或者对只读空间进行写操作，则产生越权访问异常。

2）从核访问 LDM 私有/共享空间过程中，如果超出对应空间容量，则发生越界异常。LDM 全 0 地址访问使能（CSR_LDM_CFG[0]）未打开（即取值为 1’b0）时，如果访问 LDM 0 地址，即不论何种访问粒度，只要访问地址为 0，则产生地址保护的越权访问异常；

3）从核访问主存过程中，在进行 SDLB 表代换时，可能产生地址越权或越界异常，详见《SW-AI 处理器软件接口手册》。

4）从核在进行主存访问过程中，可能产生主存地址溢出异常，详见《SW-AI 处理器软件接口手册》。

5）从核在进行主存访问过程中，如果对断连的核组进行访问，可能产生核组断连访问异常，详见《SW-AI 处理器软件接口手册》。

6.8.1.2.1装入不对界 HP 浮点向量

格式： VFLDH_U Va.ws,disp.ab(Rb.ab) //杂项存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

Vaddr'←Vaddr AND NOT 31 //Vaddr'为256位自然对界地址

FOR（i=0；i<16；i++）

BEGIN

Va<32*i+31:32*i>← MAP_H2S((Vaddr)<16*i+15>)

END

异常： 数据流异常

对界 （Alignment）

助记符： VFLDH_U 装入不对界HP浮点向量

指令描述：

Disp[11:0]为立即数字节偏移地址[11:0]位，经符号扩展后加上寄存器Rb的值形成有效地址Vaddr。将Vaddr低5位清0，即地址强制对界形成有效地址Vaddr'，并从Vaddr'对应的存储器地址中取出16个连

续的HP浮点向量元素，按照寄存器格式写到向量寄存器Va。

要求有效地址Vaddr按2字节分量自然对界，否则将产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI处理器软件接口手册》

6.8.1.2.2装入不对界字节向量

格式： VLDB_UVa.ww,disp.ab(Rb.ab) //杂项存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

Vaddr'←Vaddr AND NOT 31 //Vaddr'为256位自然对界地址

FOR（i=0；i<32；i++）

BEGIN

Va<16*(i)+15:16*(i)>←SEXT((Vaddr')<8*i+7:8*i>)

END

异常： 数据流异常

助记符： VLDB_U 装入不对界字节整数向量

指令描述：

Disp[11:0]为立即数字节偏移地址[11:0]位，经符号扩展后加上寄存器Rb的值形成有效地址Vaddr。将Vaddr低5位清0，即地址强制对界形成有效地址Vaddr'，从Vaddr'对应的存储器中取出32个字节向量，进行符号扩展后，装入到Va中。

访存过程中还可能发生数据流异常，具体参见《SW-AI处理器软件接口手册》

6.8.1.2.3装入不对界半字向量

格式： VLDH_U Va.ww,disp.ab(Rb.ab) //杂项存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

Vaddr'←Vaddr AND NOT 31 //Vaddr'为256位自然对界地址

FOR（i=0；i<16；i++）

BEGIN

Va<32*(i)+31:32*(i)>←SEXT((Vaddr')<16*i+15:16*i>)

END

异常： 数据流异常

对界 （Alignment）

助记符： VLDH_U 装入不对界半字整数向量

指令描述：

Disp[11:0]为立即数字节偏移地址[11:0]位，经符号扩展后加上寄存器Rb的值形成有效地址Vaddr。将Vaddr低5位清0，即地址强制对界形成有效地址Vaddr'，从Vaddr'对应的存储器中取出16个半字向量，进行符号扩展后，装入到Va中。

要求有效地址Vaddr按2字节分量自然对界，否则将产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI处理器软件接口手册》

6.8.1.2.4装入不对界字向量/装入不对界 SP 浮点向量

格式： VLDW_U Va.ww,disp.ab(Rb.ab) //杂项存储器指令格式

操作：

```
Vaddr←{Rbv+SEXT(disp)}
Vaddr'←Vaddr AND NOT 63                    //Vaddr'为512位自然对界地址
FOR（i=0；i<16；i++）
BEGIN
    Va<32*(i)+31:32*(i)>←(Vaddr')<32*i+31:32*i>
END
```

异常： 数据流异常

对界 （Alignment）

助记符： VLDW_U 装入不对界字向量或SP浮点向量

指令描述：

Disp[11:0]为立即数字节偏移地址[11:0]位，经符号扩展后加上寄存器Rb的值形成有效地址Vaddr。将Vaddr低6位清0，即地址强制对界形成有效地址Vaddr'，并将Vaddr'对应的存储器内容装入到Va中。

VLDW_U 指令也用于将 512 位的 SP 浮点向量从存储器载入到寄存器中。

要求有效地址Vaddr按4字节分量自然对界，否则将产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI处理器软件接口手册》

6.8.1.2.5存储不对界 HP 浮点向量

格式： VFSTH_U Va.rs,disp.ab(Rb.ab) //杂项存储器指令格式

操作：

```
Vaddr←{Rbv+SEXT(disp)}
Vaddr'←Vaddr AND NOT 31                    //Vaddr'为256位自然对界地址
FOR（i=0；i<16；i++）
BEGIN
    Vaddr'<16*i+15:16*i>←MAP_S2H(Va<32*i+31:32*i>)
END
```

异常： 对界 （Alignment）

数据流异常

助记符： VFSTH_U 存储不对界HP浮点向量

指令描述：

Disp[11:0]为立即数字节偏移地址[11:0]位，经符号扩展后加上寄存器Rb的值形成有效地址Vaddr。将Vaddr低5位清0，即地址强制对界形成有效地址Vaddr'，并针Va寄存器中16个连续的寄存器格式HP浮点向量元素，按照存储器格式写入Vaddr'对应的存储器地址中。

要求有效地址Vaddr按2字节分量自然对界，否则将产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.8.1.2.6存储不对界字节向量

格式： VSTB_U Va.rs,disp.ab(Rb.ab) //杂项存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

Vaddr'←Vaddr AND NOT 31 //Vaddr'为256位自然对界地址

FOR（i=0；i<32；i++）

BEGIN

Vaddr'<8*i+7:8*i>←(Va<16*i+7:16*i>)

END

异常： 数据流异常

助记符： VSTB_U 存储不对界字节向量

指令描述：

Disp[11:0]为立即数字节偏移地址[11:0]位，经符号扩展后加上寄存器Rb的值形成有效地址Vaddr。将Vaddr低5位清0，即地址强制对界形成有效地址Vaddr'，并将Va寄存器中32个16位元素位置中的低位字节分量写入Vaddr'对应的存储器地址中。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.8.1.2.7存储不对界半字向量

格式： VSTH_U Va.rs,disp.ab(Rb.ab) //杂项存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

Vaddr'←Vaddr AND NOT 31 //Vaddr'为256位自然对界地址

FOR（i=0；i<16；i++）

BEGIN

Vaddr'<16*i+15:16*i>←(Va<32*i+15:32*i>

END

异常： 对界 （Alignment）

数据流异常

助记符： VSTH_U 存储不对界半字向量

指令描述：

Disp[11:0]为立即数字节偏移地址[11:0]位，经符号扩展后加上寄存器Rb的值形成有效地址Vaddr。将Vaddr低5位清0，即地址强制对界形成有效地址Vaddr'，并将Va寄存器中16个32位元素位置中的低位半字分量写入Vaddr'对应的存储器地址中。

要求有效地址Vaddr按2字节分量自然对界，否则将产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.8.1.2.8存储不对界字向量/存储不对界 SP 浮点向量

格式： VSTW_U Va.rw,disp.ab(Rb.ab) //杂项存储器指令格式

操作： Vaddr←{Rbv+SEXT(disp)}

Vaddr'←Vaddr AND NOT 63 //Vaddr'为512位自然对界地址

FOR（i=0；i<16；i++）

BEGIN

Vaddr'<32*i+31:32*i>←Va<32*i+31:32*i)

END

异常： 对界 （Alignment）

数据流异常

助记符： VSTW_U 存储不对界字向量

指令描述：

Disp[11:0]为立即数字节偏移地址[11:0]位，经符号扩展后加上寄存器Rb的值形成有效地址Vaddr。将Vaddr低6位清0，即地址强制对界形成有效地址Vaddr'，并将Va寄存器中16个连续的字向量元素写入Vaddr'对应的存储器地址中。

VSTW_U 指令也用于将 512 位的 SP 浮点向量从寄存器写到存储器中。

要求有效地址Vaddr按4字节分量自然对界，否则将产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI 处理器软件接口手册》

6.8.2 整数 SIMD 运算指令（22 条）

整数SIMD运算指令实现8个32位整数字向量的加、减、移位运算和16个16位整数半字向量的加、减运算，包括采用简单运算指令格式的12条指令。

表 0-38 整数 SIMD 运算指令列表(简单运算指令格式)

序号	指令		寄存器格式操作码	立即数格式操作码	描述
1	vaddw	Va,Vb/#b,Vc	0x5.0.0	0x5.1.0	字向量加
2	vsubw	Va,Vb/#b,Vc	0x5.0.1	0x5.1.1	字向量减
3	vmulw	Va,Vb/#b,Vc	0x5.0.2	0x5.1.2	字向量乘
4	vaddh	Va,Vb/#b,Vc	0x5.0.4	0x5.1.4	半字向量加
5	vsubh	Va,Vb/#b,Vc	0x5.0.5	0x5.1.5	半字向量减
6	vmulh	Va,Vb/#b,Vc	0x5.0.6	0x5.1.6	半字向量乘
7	vcmpeqw	Va,Vb/#b,Vc	0x5.4.0	0x5.5.0	字向量等于比较
8	vcmpltw	Va,Vb/#b,Vc	0x5.4.1	0x5.5.1	字向量小于比较
9	vcmplew	Va,Vb/#b,Vc	0x5.4.2	0x5.5.2	字向量小于等于比较
10	vcmpeqh	Va,Vb/#b,Vc	0x5.4.4	0x5.5.4	半字向量等于比较
11	vcmplth	Va,Vb/#b,Vc	0x5.4.5	0x5.5.5	半字向量小于比较
12	vcmpleh	Va,Vb/#b,Vc	0x5.4.6	0x5.5.6	半字向量小于等于比较

采用浮点复合运算指令格式的浮点 SIMD 运算指令共 10 条，如下表所示：

表 0-39 整数 SIMD 运算指令列表（复合运算指令格式）

序号	指令		操作码	描述
1	vmaw	Va,Vb,Vc,Vd	0x5.d	字向量乘加
2	vmah	Va,Vb,Vc,Vd	0x5.c	半字向量乘加
3	vnmaw	Va,Vb,Vc,Vd	0x5.9	字向量负乘加
4	vnmah	Va,Vb,Vc,Vd	0x5.8	半字向量负乘加
5	vseleqw	Va,Vb,Vc,Vd	0x5.6	字向量等于“0”条件判断并选择
6	vselltw	Va,Vb,Vc,Vd	0x5.a	字向量小于“0”条件判断并选择
7	vsellew	Va,Vb,Vc,Vd	0x5.e	字向量小于等于“0”条件判断并选择
8	vseleqh	Va,Vb,Vc,Vd	0x5.7	半字向量等于“0”条件判断并选择
9	vsellth	Va,Vb,Vc,Vd	0x5.b	半字向量小于“0”条件判断并选择
10	vselleh	Va,Vb,Vc,Vd	0x5.f	半字向量小于等于“0”条件判断并选择

整数SIMD运算指令不产生任何整数算术异常。

对立即数格式的整数SIMD运算指令中加、减运算指令，先将8位立即数零扩展为字整数或半字整数，再分别与Va寄存器中整数向量的每个元素进行运算。

6.8.2.1 算术运算类（10 条）

6.8.2.1.1 整数向量加

格式： VADDx Va.rw,Vb.rw.Vc.ww //简单运算指令格式

VADDx Va.rw,#b.ib,Vc.ww //简单运算指令格式

操作： temp←0

VADDW：

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←Vav<32*i+31:32*i>+Vbv<32*i+31:32*i>

END

VADDH：

FOR(i=0;i<32;i++)

BEGIN

temp<16*i+15:16*i>←Vav<16*i+15:16*i>+Vbv<16*i+15:16*i>

END

Vc←temp

异常： 无

助记符： VADDW 字向量加

VADDH 半字向量加

指令描述：

Va、Vb、Vc 均为存放 16 个字整数或 32 个半字整数的短向量寄存器，将 Va 和 Vb 中 16 个 32 位字或 32 个 16 位半字（或 8 位无符号立即数#b）分别对应相加，取低 32 位结果或低 16 位结果保存到 Vc 中对应位置，不进行溢出检查。

对立即数格式的 vaddw/vaddh 指令，先将 8 位立即数零扩展为字整数或半字整数，再分别与 Va 寄存器中字整数向量或半字整数向量的每个元素进行运算。

6.8.2.1.2 整数向量减

格式： VSUBx Va.rw,Vb.rw,Vc.ww //简单运算指令格式

VSUBx Va.rw,#b.ib,Vc.ww //简单运算指令格式

操作： temp←0

VSUBW：

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←Vav<32*i+31:32*i>–Vbv<32*i+31:32*i>

END

VSUBH：

FOR(i=0;i<32;i++)

BEGIN

temp<16*i+15:16*i>←Vav<16*i+15:16*i>–Vbv<16*i+15:16*i>

END

Vc←temp

异常： 无

助记符： VSUBW 字向量减

VSUBH 半字向量减

指令描述：

Va、Vb、Vc 均为存放 16 个字整数或 32 个半字整数的短向量寄存器，将 Va 和 Vb 中 16 个 32 位字或 32 个 16 位半字（或 8 位无符号立即数#b）分别对应相减，取低 32 位结果或低 16 位结果保存到 Vc 中对应位置，不进行溢出检查。

对立即数格式的 vsubw/vsubh 指令，先将 8 位立即数零扩展为字整数或半字整数，再分别与 Va 寄存器中字整数向量或半字整数向量的每个元素进行运算。

6.8.2.1.3 整数向量乘

格式： VMULx Va.rw,Vb.rw,Vc.ww //简单运算指令格式

VMULx Va.rw,#b.ib,Vc.ww //简单运算指令格式

操作： temp←0

VMULW：

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←Vav<32*i+31:32*i>×Vbv<32*i+31:32*i>

END

VMULH：

FOR(i=0;i<32;i++)

BEGIN

temp<16*i+15:16*i>←Vav<16*i+15:16*i>×Vbv<16*i+15:16*i>

END

Vc←temp

异常： 无

助记符： VMULW 字向量乘

VMULH 半字向量乘

指令描述：

Va、Vb、Vc 均为存放 16 个字整数或 32 个半字整数的短向量寄存器，将 Va 和 Vb 中 16 个 32 位字或 32 个 16 位半字（或 8 位无符号立即数#b）分别对应相乘，取低 32 位结果或低 16 位结果保存到 Vc 中对应位置，不进行溢出检查。

对立即数格式的 vmulw/vmulh 指令，先将 8 位立即数零扩展为字整数或半字整数，再分别与 Va 寄存器中字整数向量或半字整数向量的每个元素进行运算。

6.8.2.1.4 整数向量乘加

格式： VMAx Va.rw,Vb.rw,Vc.rw,Vd,ww //简单运算指令格式

操作：

```
temp←0
VMAW：
    FOR(i=0;i<16;i++)
    BEGIN
        temp<32*i+31:32*i>←Vav<32*i+31:32*i>×Vbv<32*i+31:32*i>
                          +Vcv<32*i+31:32*i>
    END
VMAH：
    FOR(i=0;i<32;i++)
    BEGIN
        temp<16*i+15:16*i>←Vav<16*i+15:16*i>×Vbv<16*i+15:16*i>
                          +Vcv<16*i+15:16*i>
    END
Vc←temp
```

异常： 无

助记符： VMAW 字向量乘加

VMAH 半字向量乘加

指令描述：

Va、Vb、Vc 和 Vd 均为存放 16 个字整数或 32 个半字整数的短向量寄存器，将 Va、Vb 和 Vc 中 16 个 32 位字或 32 个 16 位半字分别对应相乘加，取低 32 位结果或低 16 位结果保存到 Vd 中对应位置，不进行溢出检查。

6.8.2.1.5 整数向量负乘加

格式： VNMAx Va.rx,Vb.rx,Vc.rx,Vd.wx //浮点复合运算指令格式

操作： temp←0

VNMAW：

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←－Vav<32*i+31:32*i>×Vbv<32*i+31:32*i>
+Vcv<32*i+31:32*i>

END

VNMAH：

FOR(i=0;i<32;i++)

BEGIN

temp<16*i+15:16*i>←－Vav<16*i+15:16*i>×Vbv<16*i+15:16*i>
+Vcv<16*i+15:16*i>

END

Vd←temp

异常： 无

助记符： VNMAW 整数字向量负乘加

VNMAH 整数半字向量负乘加

指令描述：

Va、Vb、Vc 和 Vd 均为存放 16 个字整数或 32 个半字整数的短向量寄存器，将 Va 和 Vb 中 16 个 32 位字或 32 个 16 位半字分别相乘，得到乘积取负数，再分别加上 Vc 中的 16 个 32 位字或 32 个 16 位半字，取低 32 位结果或低 16 位结果保存到 Vd 中对应位置，不进行溢出检查。

6.8.2.2 整数向量比较（6 条）

格式： VCMPxxW Va.rw,Vb.rw/#b.ib,Vc.ww //简单运算指令格式

VCMPxxH Va.rh,Vb.rh/#b.ib,Vc.wh //简单运算指令格式

操作： VCMPxxW：

FOR(i=0;i<16;i++)

BEGIN

IF(Vav<32*i+31:32*i> RELATION Vbv<32*i+31:32*i>)

THEN

Vc<32*i+31:32*i>←0x00000001

ELSE

```
            Vc<32*i+31:32*i>←0x00000000
        END
    VCMPxxH：
        FOR(i=0;i<32;i++)
        BEGIN
            IF(Vav<16*i+15:16*i> RELATION Vbv<16*i+15:16*i>)
            THEN
                Vc<16*i+15:16*i>←0x0001
            ELSE
                Vc<16*i+15:16*i>←0x0000
        END
```

异常： 无

助记符：

VCMPEQW	字向量等于比较
VCMPLEW	字向量小于等于比较
VCMPLTW	字向量小于比较
VCMPEQH	半字向量等于比较
VCMPLEH	半字向量小于等于比较
VCMPLTH	半字向量小于比较

指令描述：

Va、Vb（或 8 位无符号立即数#b）、Vc 均为存放 16 个有符号字整数或 32 个有符号半字整数的短向量寄存器，将 Va 和 Vb 中 16 个 32 位字或 32 个 16 位半字分别进行符号比较，若满足指定关系，则的值为“1”，否则为“0”，并保存到 Vc 中对应位置。

6.8.2.3 条件选择指令（6 条）

格式：

```
VSELxxW     Va.rx,Vb.rx,Vc.rx,Vd.wx     //浮点复合运算指令格式
VSELxxH     Va.rx,Vb.rx,Vc.rx,Vd.wx     //浮点复合运算指令格式
```

操作：

```
VSELxxW：
    FOR(i=0;i<16;i++)
    BEGIN
        IF  TEST(Va<32*i+31:32*i>,Condition_based_on_Opcode)
        THEN
            Vd<32*i+31:32*i>←Vbv<32*i+31:32*i>
        ELSE
            Vd<32*i+31:32*i>←Vcv<32*i+31:32*i>
    END
```

```
VSELxxH：
    FOR(i=0;i<32;i++)
    BEGIN
        IF  TEST(Va<16*i+15:16*i>,Condition_based_on_Opcode)
        THEN
            Vd<16*i+15:16*i>←Vbv<16*i+15:16*i>
        ELSE
            Vd<16*i+15:16*i>←Vcv<16*i+15:16*i>
    END
```

异常： 无

助记符：

VSELEQW	字向量等于“0”条件判断并选择
VSELLTW	字向量小于“0”条件判断并选择
VSELLEW	字向量小于等于“0”条件判断并选择
VSELEQH	半字向量等于“0”条件判断并选择
VSELLTH	半字向量小于“0”条件判断并选择
VSELLEH	半字向量小于等于“0”条件判断并选择

指令描述：

Va、Vb、Vc、Vd均为存放16个32位有符号字向量或32个16位有符号半字向量的短向量寄存器，对Va的每个分量元素分别进行测试，若条件成立，则将向量Vb对应分向量元素写入Vd，否则将向量Vc对应分量元素数据写入Vd。

6.8.3 SIMD 可重构指令（2 条）

表 0-40 SIMD 可重构指令列表

序号	指令	寄存器格式操作码	立即数格式操作码	描述
1	vlog2x<truth> Va,Vb/#b,Vc	0x5.2.0~f	0x5.3.0~f	两操作数 SIMD 可重构运算
2	vlog3r<idx> Va,Vb,Vc,Vd	0x7	—	三操作数 SIMD 可重构运算

6.8.3.1 两操作数 SIMD 可重构指令 vlog2x

格式：

```
VLOG2X<TRUTH>   Va.ro,Vb.rq,Vc.wo    //简单运算指令格式
VLOG2X<TRUTH>   Va.ro,#b.ib,Vc.wo    //简单运算指令格式
```

操作：

```
temp←0
FOR(i=0;i<=511;i++)
BEGIN
```

temp<i>←TRUTH_MAP(Vav<i>,Vbv<i>,TRUTH)

END

Vc←temp

异常： 无

助记符： LOG2X<TRUTH> 两操作数SIMD可重构指令

指令描述：

将 Va 和 Vb（或 8 位立即数）中对应的每一位，组成 2 位的二进制数，从指令的<TRUTH>域的 4 位二进制值中选出一位，写入 Vc 的对应位中。

对立即数格式的vlog2x指令，先将8位立即数零扩展为字整数，再分别与Va寄存器中字整数向量的每个元素进行运算。

原有的两操作数向量逻辑运算指令到 vlog2x 的真值映射如下所示。

表 0-41 常见向量逻辑运算指令对应的向量可重构指令

助记符	逻辑运算	vlog2x
VAND	Vav & Vbv	vlog2x8
VBIC	Vav & (~Vbv)	vlog2x4
VBIS	Vav | Vbv	vlog2xe
VORNOT	Vav | (~Vbv)	vlog2xd
VXOR	Vav ^ Vbv	vlog2x6
VEQV	(Vav & Vbv) | (~Vav & ~Vbv)	vlog2x9

6.8.3.2 三操作数 SIMD 可重构指令 vlog3r

格式： VLOG3R<IDX> Va.ro,Vb.ro,Vc.ro,Vd.wo //浮点复合运算指令格式

操作： temp←0

TRUTH←真值表配置寄存器<IDX>

FOR(i=0;i<=511;i++)

BEGIN

temp<i>←TRUTH_MAP(Vav<i>,Vbv<i>,Vcv<i>,TRUTH)

END

Vd←temp

异常： 无

助记符： VLOG3R<IDX> 三操作数SIMD可重构指令

指令描述：

FUNC[3:0]用作 IDX，根据 IDX，从配置寄存器取出真值表 TRUTH，然后将 Va、Vb、Vc 中对应的每一位，组成 3 位的二进制数，从指令的<TRUTH>域的 8 位二进制值中选出一位，写入 Vd 的对应位中。SW-AI 从核提供 16 个真值表配置寄存器（与 LOG3R 指令共用），IDX 有效值为 0~15。

6.8.4 浮点 SIMD 运算指令（19 条）

浮点 SIMD 操作包含采用简单运算指令格式的 10 条指令和采用浮点复合运算指令格式的 9 条指令。通过操作码和功能码区分不同的指令。指令列表中的操作码，其“.”之前的编码为操作码，之后的编码为功能码，采用简单运算指令格式的指令功能码为 8 位，浮点复合运算指令格式的指令功能码为 6 位。

采用简单运算指令格式的浮点 SIMD 运算指令共 10 条，如下表所示：

表 0-42 浮点 SIMD 运算指令列表（简单运算指令格式）

序号	指令	操作码	描述
1	vfadds Va,Vb,Vc	0x9.0.0	SP 浮点向量加
2	vfsubs Va,Vb,Vc	0x9.0.2	SP 浮点向量减
3	vfmuls Va,Vb,Vc	0x9.0.4	SP 浮点向量乘
4	vfcmpeqs Va,Vb,Vc	0x9.0.d	SP 浮点向量等于比较
5	vfcmples Va,Vb,Vc	0x9.0.f	SP 浮点向量小于等于比较
6	vfcmplts Va,Vb,Vc	0x9.0.e	SP 浮点向量小于比较
7	vfcmpuns Va,Vb,Vc	0x9.0.c	SP 浮点向量不可排序比较
8	vfcpyss Va,Vb,Vc	0x9.0.8	SP 浮点向量拷贝符号
9	vfcpyses Va,Vb,Vc	0x9.0.9	SP 浮点向量拷贝符号和指数
10	vfcpysns Va,Vb,Vc	0x9.0.a	SP 浮点向量拷贝符号的反码

采用浮点复合运算指令格式的浮点 SIMD 运算指令共 9 条，如下表所示：

表 0-43 浮点 SIMD 运算指令列表（复合运算指令格式）

序号	指令	操作码	描述
1	vfmas Va,Vb,Vc,Vd	0x9.8	SP 浮点向量乘加
2	vfmss Va,Vb,Vc,Vd	0x9.a	SP 浮点向量乘减
3	vfnmas Va,Vb,Vc,Vd	0x9.c	SP 浮点向量负乘加
4	vfnmss Va,Vb,Vc,Vd	0x9.e	SP 浮点向量负乘减
5	vfseleqs Va,Vb,Vc,Vd	0x9.1	SP 浮点向量等于“0”条件判断并选择
6	vfsellts Va,Vb,Vc,Vd	0x9.2	SP 浮点向量小于“0”条件判断并选择
7	vfselles Va,Vb,Vc,Vd	0x9.3	SP 浮点向量小于等于“0”条件判断并选择
8	vfcvtsh Vb,Vc	0x9.4.0	浮点向量单精度转半精度
9	vfcvths Vb,Vc	0x9.4.1	浮点向量半精度转单精度

6.8.4.1 浮点算术类（3 条）

6.8.4.1.1 SP 浮点向量加

格式： VFADDS Va.rx,Vb.rx,Vc.wx //简单运算指令格式

操作： temp←0

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←Vav<32*i+31:32*i>+Vbv<32*i+31:32*i>

END

Vc←temp

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： VFADDS SP浮点向量加

指令描述：

Va、Vb、Vc 均为存放 16 个 32 位浮点格式数据的向量寄存器，将 Va 和 Vb 中 16 个 32 位浮点操作数分别对应相加，结果写入 Vc。结果被舍入到指定的精度，并检查上溢和下溢。

6.8.4.1.2 SP 浮点向量减

格式： VFSUBS Va.rx,Vb.rx,Vc.wx //简单运算指令格式

操作： temp←0

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←Vav<32*i+31:32*i>－Vbv<32*i+31:32*i>

END

Vc←temp

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： VFSUBS SP浮点向量减

指令描述：

Va、Vb、Vc 均为存放 16 个 32 位浮点格式数据的向量寄存器，将 Va 和 Vb 中 16 个 32 位浮点操作数分别对应相减，结果写入 Vc。结果被舍入到指定的精度，并检查上溢和下溢。

6.8.4.1.3 SP 浮点向量乘

格式： VFMULS Va.rx,Vb.rx,Vc.wx //简单运算指令格式

操作：

```
temp←0
FOR(i=0;i<16;i++)
BEGIN
    temp<32*i+31:32*i>←Vav<32*i+31:32*i>×Vbv<32*i+31:32*i>
END
Vc←temp
```

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： VFMULS SP浮点向量乘

指令描述：

Va、Vb、Vc 均为存放 16 个 32 位浮点格式数据的向量寄存器，将 Va 和 Vb 中 16 个 32 位浮点操作数分别对应相乘，结果写入 Vc。结果被舍入到指定的精度，并检查上溢和下溢。

6.8.4.2 浮点向量符号拷贝（3 条）

格式： VFCPYxxS Va.rx,Vb.rx,Vc.wx //简单运算指令格式

操作：

```
temp←0
CASE
    VFCPYSS:
        FOR(i=0;i<16;i++)
        BEGIN
            temp<32*i+31:32*i>←Vav<32*i+31>||Vbv<32*i+30:32*i+0>
        END
    VFCPYSNS:
```

```
        FOR(i=0;i<16;i++)
        BEGIN
            temp<32*i+31:32*i>←NOT(Vav<32*i+31>)||Vbv<32*i+30:32*i+0>
        END
    VFCPYSES:
        FOR(i=0;i<16;i++)
        BEGIN
            temp<32*i+31:32*i>←Vav<32*i+31:32*i+23>||Vbv<32*i+22:32*i+0>
        END
ENDCASE
Vc←temp
```

异常： 无

助记符：		
	VFCPYSS	SP浮点向量拷贝符号
	VFCPYSES	SP浮点向量拷贝符号和指数
	VFCPYSNS	SP浮点向量拷贝符号的反码

指令描述：

对于 VFCPYSS 或 VFCPYSNS，取 Va 的符号位或符号的反码，与 Vb 的指数和尾数拼接起来，存入 Vc。对于 VFCPYSES，取 Va 的符号位和指数位与 Vb 的尾数位拼接起来存入 Vc，不检查操作数的合法性。

利用 VFCPYSS V31,Vx,Vy 可以实现浮点向量绝对值功能，利用 VFCPYSNS Vx,Vx,Vy 可以实现浮点向量数符号取反的功能，利用 VFCPYSES 可以将浮点向量值比例绽放到某个已知的范围。

6.8.4.3 浮点向量比较（4 条）

格式： VFCMPxxS Va.rd,Vb.rd,Vc.wd //简单运算指令格式

操作：

```
FOR(i=0;i<16;i++)
BEGIN
    IF(Vav<32*i+31:32*i>RELATIONVbv<32*i+31:32*i>)
    THEN
        Vc<32*i+31:32*i>←0x40000000
    ELSE
        Vc<32*i+31:32*i>←0x00000000
END
```

异常： 无效操作（Invalid Operation）

非规格化数异常（Denormal）

助记符：		
	VFCMPEQS	SP 浮点向量等于比较

VFCMPLES	SP 浮点向量小于等于比较
VFCMPLTS	SP 浮点向量小于比较
VFCMPUNS	SP浮点向量不可排序比较

指令描述：

Va、Vb、Vc 均为存放 16 个浮点向量元素的向量寄存器，将 Va 和 Vb 中的对应浮点向量元素进行比较，如果条件成立，则将非“0”浮点值（2.0）写入 Vc 中的对应位置，否则将真“0”写入 Vc 中的对应位置。比较是精确的，不会产生上溢和下溢。当两个操作数中任意一个为 NaN 时，无序关系为真。比较时忽略“0”的符号位，因此“＋0”与“－0”是相等的。另外，与+∞、－∞的比较正常进行，不会产生无效操作自陷。

浮点向量比较指令没有明确说明参与比较的浮点操作数数据格式，既可以用于基本单精度格式数据之间的比较，也可以用于 32 位扩展半精度格式数据之间的比较。用户必须保证不会进行不同数据格式操作数之间的比较，否则可能导致不可预知的结果。

6.8.4.4 浮点向量转换（2 条）

6.8.4.4.1 浮点向量单精度转半精度

格式： VFCVTSH Vb.rx,Vc.wx //简单运算指令格式

操作：
```
FOR(i=0;i<16;i++)
BEGIN
    Vc<32*i+31:32*i>←{conversion of Vbv<32*i+31:32*i>}
END
```

异常： 无效操作（Invalid Operation）
上溢（Overflow）
下溢（Underflow）
非精确结果（Inexact Result）
非规格化数异常（Denormal）

助记符： VFCVTSH 浮点向量单精度转半精度

指令描述：

Vb、Vc 均为存放 16 个 32 位浮点向量元素的向量寄存器，将 Vb 的每个浮点向量元素转换成 32 位扩展的 HP 浮点（EHP32）并写入 Vc 对应元素位置。

6.8.4.4.2浮点向量半精度转单精度

格式： VFCVTHS Vb.rx,Vc.wx //简单运算指令格式

操作：
```
FOR(i=0;i<16;i++)
BEGIN
    Vc<32*i+31:32*i>←{conversion of Vbv<32*i+31:32*i>}
END
```

异常： 无效操作（Invalid Operation）

非规格化数异常（Denormal）

助记符：		
	VFCVTHS	浮点向量半精度转单精度

指令描述：

Vb、Vc 均为存放 16 个 32 位扩展 HP 浮点（EHP32）向量元素的向量寄存器，将 Vb 的每个浮点向量元素转换成 32 位 SP 浮点并写入 Vc 对应元素位置。从 HSP 浮点到 DPSP 浮点的转换是精确、无舍入的，是有限值的恒等转换。无上溢、下溢和非精确结果自陷产生。

6.8.4.5 浮点向量条件选择（3 条）

格式： VFSELxxS Va.rx,Vb.rx,Vc.rx,Vd.wx //浮点复合运算指令格式

操作：
```
FOR(i=0;i<16;i++)
BEGIN
    IF  TEST(Va<32*i+31:32*i>,Condition_based_on_Opcode)
    THEN
        Vd<32*i+31:32*i>←Vbv<32*i+31:32*i>
    ELSE
        Vd<32*i+31:32*i>←Vcv<32*i+31:32*i>
END
```

异常： 无

助记符：		
	VFSELEQS	SP浮点向量等于“0”条件判断并选择
	VFSELLTS	SP浮点向量小于“0”条件判断并选择
	VFSELLES	SP浮点向量小于等于“0”条件判断并选择

指令描述：

Va、Vb、Vc、Vd均为存放16个32位浮点向量元素的向量寄存器，对Va的每个浮点向量元素分别进行测试，若条件成立，则将向量Vb对应浮点向量元素写入Vd，否则将向量Vc对应浮点向量元素数据写入Vd。

SW-AI 处理器在实现浮点向量条件选择指令时，直接根据 Va 中每个浮点向量元素的“[31]位值”和“[30:0]位是否为全 0”来进行条件判断。在 Va 的浮点向量元素中存在非数时，条件判断的结果和“Va

与 0 比较”的结果不一致，详细说明见附录 5“浮点条件判断与浮点比较的区别”。

6.8.4.6 浮点乘加类（4 条）

6.8.4.6.1浮点向量乘加

格式： VFMAS Va.rx,Vb.rx,Vc.rx,Vd.wx //浮点复合运算指令格式

操作：
```
temp←0
FOR(i=0;i<16;i++)
BEGIN
    temp<32*i+31:32*i>←Vav<32*i+31:32*i>×Vbv<32*i+31:32*i>
                       +Vcv<32*i+31:32*i>
END
Vd←temp
```

异常： 无效操作（Invalid Operation）
上溢（Overflow）
下溢（Underflow）
非精确结果（Inexact Result）
非规格化数异常（Denormal）

助记符： VFMAS SP浮点向量乘加

指令描述：

Va、Vb、Vc 和 Vd 均为存放 16 个浮点格式数据的向量寄存器，将 Va 和 Vb 中 16 个 32 位浮点操作数分别相乘后再与 Vc 中的 16 个 32 位浮点数分别相加，最后结果写入 Vd。不对中间结果进行舍入，也不作上溢和下溢检查。只对最终结果进行舍入，然后再做上溢和下溢检查。

6.8.4.6.2浮点向量乘减

格式： VFMSS Va.rx,Vb.rx,Vc.rx,Vd.wx //浮点复合运算指令格式

操作：
```
temp←0
FOR(i=0;i<16;i++)
BEGIN
    temp<32*i+31:32*i>←Vav<32*i+31:32*i>×Vbv<32*i+31:32*i>
                       －Vcv<32*i+31:32*i>
END
Vd←temp
```

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： VFMSS SP浮点向量乘减

指令描述：

Va、Vb、Vc 和 Vd 均为存放 16 个浮点格式数据的向量寄存器，将 Va 和 Vb 中 16 个 32 位浮点操作数分别相乘后再分别减去 Vc 中的 16 个 32 位浮点数，最后结果写入 Vd。不对中间结果进行舍入，也不作上溢和下溢检查。只对最终结果进行舍入，然后再做上溢和下溢检查。

6.8.4.6.3浮点向量负乘加

格式： VFNMAS Va.rx,Vb.rx,Vc.rx,Vd.wx //浮点复合运算指令格式

操作： temp←0

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←－Vav<32*i+31:32*i>×Vbv<32*i+31:32*i>

+Vcv<32*i+31:32*i>

END

Vd←temp

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： VFNMAS SP浮点向量负乘加

指令描述：

Va、Vb、Vc 和 Vd 均为存放 16 个浮点格式数据的向量寄存器，将 Va 和 Vb 中 16 个 32 位浮点操作数分别相乘，得到乘积取负数，再分别加上 Vc 中的 16 个 32 位浮点数，最后结果写入 Vd。不对中间结果进行舍入，也不作上溢和下溢检查。只对最终结果进行舍入，然后再做上溢和下溢检查。

6.8.4.6.4浮点向量负乘减

格式： VFNMSS Va.rx,Vb.rx,Vc.rx,Vd.wx //浮点复合运算指令格式

操作： temp←0

FOR(i=0;i<16;i++)

BEGIN

temp<32*i+31:32*i>←－Vav<32*i+31:32*i>×Vbv<32*i+31:32*i>
－Vcv<32*i+31:32*i>

END

Vd←temp

异常： 无效操作（Invalid Operation）

上溢（Overflow）

下溢（Underflow）

非精确结果（Inexact Result）

非规格化数异常（Denormal）

助记符： VFNMSS SP浮点向量负乘减

指令描述：

Va、Vb、Vc 和 Vd 均为存放 16 个浮点格式数据的向量寄存器，将 Va 和 Vb 中 16 个 32 位浮点操作数分别相乘，得到乘积取负数，再分别减去 Vc 中的 16 个 32 位浮点数，最后结果写入 Vd。不对中间结果进行舍入，也不作上溢和下溢检查。只对最终结果进行舍入，然后再做上溢和下溢检查。

6.8.5 SIMD 整理指令（11 条）

SIMD 整理指令采用浮点复合运算指令格式，包含 11 条指令，实现对整数字向量/半字向量或浮点 SP 向量的整理操作。

表 0-44 SIMD 整理指令列表

序号	指令		寄存器格式操作码	立即数格式操作码	描述
1	VCONW	Va,Vb,Rc,Vd	0x3.0	—	字向量或 SP 浮点向量拼接
2	VINSW	Ra,Vb,#c,Vd	—	0x3.8	字向量元素插入
3	VEXTW	Vb,#c,Rd	—	0x3.c	字向量元素提取
4	VCPYW	Ra,Vd	0x3.4	—	字向量拷贝指令
5	VFINSS	Ra,Vb,#c,Vd		0x3.9	SP 浮点向量元素插入
6	VFEXTS	Vb,#c,Rd		0x3.d	SP 浮点向量元素提取
7	VFCPYS	Ra,Vd	0x3.5		SP 浮点向量拷贝指令

8	VCONH	Va,Vb,Rc,Vd	0x3.2		半字向量拼接
9	VINSH	Ra,Vb,#c,Vd		0x3.a	半字向量元素插入
10	VEXTH	Vb,#c,Rd		0x3.e	半字向量元素提取
11	VCPYH	Ra,Vd	0x3.6		半字向量拷贝指令

6.8.5.1 拼接操作（2 条）

6.8.5.1.1 字向量元素拼接

格式： VCONW Va.rw,Vb.rw,Rc.rb,Vd.ww //复合运算指令格式

操作：

```
Byte4_len←Rc<5:0> >> 2
FOR(i=0;i<16;i++)
BEGIN
    IF(i<Byte4_len)  THEN
        Vd<32*(i+(16-Byte4_len))+31:32*(i+(16-Byte4_len))><←Va<32*i+31:32*i>
    ELSE
        Vd<32*(i-Byte4_len)+31:32*(i-Byte4_len)><←Vb<32*i+31:32*i>
END
```

异常： 无

助记符： VCONW 字向量元素拼接

指令描述：

字向量拼接，以字分量为单位，通常和 VLDW_U、VLDH_U 配合使用。其中 Va 存放不对界高段，Vb 存放不对界低段，Rc 存放不对界地址低 6 位（忽略其低 2 位）。根据不对界地址低 6 位，将 Va 存放的高段数据和 Vb 存放的低段数据拼接后形成新的向量，存放到 Vd 中。

VCONW 指令也用于 SP 浮点向量的拼接，通常和 VFLDH_U 配合使用。

6.8.5.1.2 半字向量元素拼接

格式： VCONH Va.rh,Vb.rh,Rc.rb,Vd.wh //复合运算指令格式

操作：

```
Byte2_len←Rc<5:0> >> 1
FOR(i=0;i<32;i++)
BEGIN
    IF(i<Byte2_len)  THEN
        Vd<16*(i+(32-Byte2_len))+15:16*(i+(32-Byte4_len))><←Va<16*i+15:16*i>
    ELSE
```

Vd<16*(i-Byte2_len)+15:16*(i-Byte2_len)><←Vb<16*i+15:16*i>

END

异常： 无

助记符： VCONH 半字向量元素拼接

指令描述：

半字向量拼接，以半字分量为单位，通常和 VLDB_U 配合使用。其中 Va 存放不对界高段，Vb 存放不对界低段，Rc 存放不对界地址低 6 位（忽略其低 1 位）。根据不对界地址低 6 位，将 Va 存放的高段数据和 Vb 存放的低段数据拼接后形成新的向量，存放到 Vd 中。

6.8.5.2 插入操作（3 条）

6.8.5.2.1 字向量元素插入

格式： VINSW Ra.rw,Vb.rw,#c.ib,Vd.ww //浮点复合运算指令格式

操作：

```
temp←0
FOR(i=0;i<16;i++)
BEGIN
    IF  TEST(i==#c)  THEN  temp<32*i+31:32*i><←Rav<31:0>
    ELSE  temp<32*i+31:32*i><←Vbv<32*i+31:32*i>
END
Vd←temp
```

异常： 无

助记符： VINSW 字向量元素插入

指令描述：

操作数为 Ra、Vb、#c 和 Vd，其中#c 为 6 位立即数。功能是将寄存器 Ra 低 32 位字整数，替代 Vb 中由#c 低 4 位（有效值为 0~15）指定的 32 位字元素，组成新的字整数向量写入 Vd。#c 的[5:4]位被忽略。

6.8.5.2.2 SP 浮点向量元素插入

格式： VFINSS Ra.rw,Vb.rw,#c.ib,Vd.ww //浮点复合运算指令格式

操作：

```
temp←0
FOR(i=0;i<16;i++)
BEGIN
    IF  TEST(i==#c)  THEN  temp<32*i+31:32*i><←MAP_D2S(Rav<63:0>)
```

ELSE temp<32*i+31:32*i>←Vbv<32*i+31:32*i>

END

Vd←temp

异常： 无

助记符： VFINSS SP浮点向量元素插入

指令描述：

操作数为 Ra、Vb、#c 和 Vd，其中#c 为 6 位立即数。功能是将寄存器 Ra 中的 64 位扩展 SP 浮点数，替代 Vb 中由#c 低 4 位（有效值为 0~15）指定的 32 位基本 SP 浮点元素，组成新的 SP 浮点向量写入 Vd。#c 的[5:4]位被忽略。取自寄存器 Ra 中的 64 位扩展 SP 浮点源操作数需重新排序以符合基本 SP 浮点格式，然后将重新排序后的结果写入寄存器中。忽略 Ra 的<61:59>和<28:0>位，但并不检查这些位的合法性。

6.8.5.2.3半字向量元素插入

格式： VINSH Ra.rh,Vb.rh,#c.ib,Vd.wh //浮点复合运算指令格式

操作： temp←0

FOR(i=0;i<32;i++)

BEGIN

IF TEST(i==#c) THEN temp<16*i+15:16*i>←Rav<15:0>

ELSE temp<16*i+15:16*i>←Vbv<16*i+15:16*i>

END

Vd←temp

异常： 无

助记符： VINSH 半字向量元素插入

指令描述：

操作数为 Ra、Vb、#c 和 Vd，其中#c 为 6 位立即数。功能是将寄存器 Ra 低 16 位半字整数，替代 Vb 中由#c 低 5 位（有效值为 0~31）指定的 16 位半字元素，组成新的半字整数向量写入 Vd。#c 的[5]位被忽略。

6.8.5.3 抽取操作（3 条）

6.8.5.3.1字向量元素提取

格式： VEXTW Vb.rw,#c.ib,Rd.wl //浮点复合运算指令格式

操作： FOR(i=0;i<16;i++)

BEGIN

IF TEST(i==#c) THEN

BEGIN

Rd<63:0>←SEXT(Vbv<32*i+31:32*i>)

END

END

异常： 无

助记符： VEXTW 字向量元素提取

指令描述：

操作数寄存器为 Va（无意义）、Vb、#c 和 Rd，其中#c 为 6 位立即数。功能是将 Vb 中由#c 低 4 位（有效值为 0~15）指定的 32 位字元素，符号扩展到 64 位，写入寄存器 Rd。#c 的[5:4]位被忽略。

6.8.5.3.2 SP 浮点向量元素提取

格式： VFEXTS Vb.rw,#c.ib,Rd.wl //浮点复合运算指令格式

操作： FOR(i=0;i<16;i++)

BEGIN

IF TEST(i==#c) THEN

BEGIN

Rd<63:0>←MAP_S2D(Vbv<32*i+31:32*i>)

END

END

异常： 无

助记符： VFEXTS SP浮点向量元素提取

指令描述：

操作数寄存器为 Va（无意义）、Vb、#c 和 Rd，其中#c 为 6 位立即数。功能是将 Vb 中由#c 低 4 位（有效值为 0~15）指定的 32 位基本 SP 浮点元素，映射为 64 位扩展 SP 浮点元素，写入寄存器 Rd。#c 的[5:4]位被忽略。

6.8.5.3.3 半字向量元素提取

格式： VEXTH Vb.rh,#c.ib,Rd.wl //浮点复合运算指令格式

操作： FOR(i=0;i<32;i++)

BEGIN

IF TEST(i==#c) THEN

BEGIN

Rd<63:0>←SEXT(Vbv<16*i+15:16*i>)

END

END

异常： 无

助记符： VEXTH 半字向量元素提取

指令描述：

操作数寄存器为 Va（无意义）、Vb、#c 和 Rd，其中#c 为 6 位立即数。功能是将 Vb 中由#c 低 5 位（有效值为 0~31）指定的 16 位半字元素，符号扩展到 64 位，写入寄存器 Rd。#c 的[5]位被忽略。

6.8.5.4 拷贝操作（3 条）

6.8.5.4.1 字向量元素拷贝

格式： VCPYW Ra.rw,Vd.ww //复合运算指令格式

操作： temp←0

FOR（i=0；i<16；i++）

BEGIN

temp<32*i+31:32*i>←Ra<31:0>

END

Vd←temp

异常： 无

助记符： VCPYW 字向量元素拷贝

指令描述：

将Ra低32位字整数，复制成16个相同元素组成新的字整数向量写入Fd；

6.8.5.4.2 SP 浮点向量元素拷贝

格式： VFCPYS Ra.rw,Vd.ww //复合运算指令格式

操作： temp←0

FOR（i=0；i<16；i++）

BEGIN

temp<32*i+31:32*i>←MAP_D2S(Ra<63:0>)

END

Vd←temp

异常： 无

助记符： VFCPYS SP浮点向量元素拷贝

指令描述：

将Ra中64位扩展SP浮点，转换为32位基本SP浮点元素后复制16份组成新的SP浮点向量写入Vd。取自寄存器Ra中的64位扩展SP浮点源操作数需重新排序以符合基本SP浮点格式，然后将重新排序后的结果写入寄存器中。忽略Ra的<61:59>和<28:0>位，但并不检查这些位的合法性。

6.8.5.4.3半字向量元素拷贝

格式： VCPYH Ra.rh,Vd.wh //复合运算指令格式

操作： temp←0

FOR（i=0；i<32；i++）

BEGIN

temp<16*i+15:16*i>←Ra<15:0>

END

Vd←temp

异常： 无

助记符： VCPYH 半字向量元素拷贝

指令描述：

将Ra低16位半字整数，复制成32个相同元素组成新的半字整数向量写入Vd；

6.9 通道类指令（4 条）

通道是从核与外部通信的主要接口之一。

表 0-45 通道指令列表

序号	指令	操作码	描述
1	dma Va,Rb,Rc	0x0.b	发起 DMA
2	rma Va,Rb,Rc	0x0.a	发起 RMA
3	wrch Va,Rb,idx	0x0.2	写通道
4	ace Rb,type	0x0.1	发起异步计算处理

6.9.1 发起 DMA

格式： DMA Va.ro,Rb.rl,Rc.rl //存储器指令（杂项指令）格式

操作： {请求本地LDM和主存之间的数据传输操作}

异常： 通道异常

数据流异常

助记符： DMA 发起DMA

指令描述：

请求本地 LDM 和主存之间的数据传输操作。Disp[11:6]用作 Rc[5:0]。DMA 命令对从核发起 DMA 操作的所有信息进行配置，使用指令中 rb 和 rc 分别配置主存地址和本地 LDM 地址，其余配置参数均放入 Va 向量寄存器中，其中 VaF1v 和 VaF0v 分别表示向量寄存器 Va 的 2 个低 64 位。Va 的 6 个高 64 目前暂不使用。DMA 命令的格式如下图：

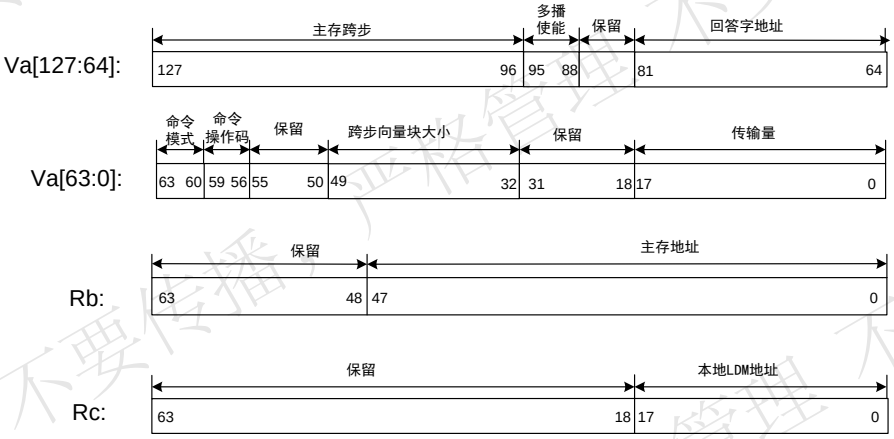


图 0-23 DMA 命令格式

采用以上格式，经常需要修改的主存地址和 LDM 地址分别放在 Rb 和 Rc 寄存器，不需要和其它参数进行拼接，可直接使用；次常改变的参数，回答字地址和传输量，在 VaF0v 和 VaF1v 的低 32 位，便于改变。对 Va 其他的配置可以一次配置好，如果遇到类似的 DMA 数据传送，就可以复用 Va 寄存器。

表 0-46 DMA 命令格式详表

Bits	名称	描述
Va		
17:0	数据传输量	一次 DMA 的数据传输量，原始以字节为单位，要求 4B 倍数。转置 DMA_GET 操作仅支持 32*32*2B=2KB 或 16*16*4B=1KB 两种规格传输量。相应地此时可根据[10]位取值来确定转置矩阵元素大小（[10]=0/2B、[10]=1/4B），传输量不能为 0
31:18	保留	
49:32	跨步向量块大小	DMA 跨步传输时有效，表示 DMA 传输的

		跨步向量块大小，原始以字节为单位，要求 4B 倍数。转置 DMA_GET 操作仅支持 32*32*2B=2KB 或 16*16*4B=1KB 两种规格传输量。相应地跨步向量块大小必须为 64B
55:50	保留	
59:56	命令操作码	DMA 命令的操作码： ● 0x0: dma_put ● 0x1: dma_get ● 0x2: dma_put（物理地址） ● 0x3: dma_get（物理地址） ● 0x5: dma_barrier ● 0x6: all_barrier ● 其他：保留 [59]位：转置标志，为 1 表示转置。该标志仅在 dma_get（即[58:56]=0x1 或 0x3）时有效
63:60	命令模式	DMA 命令模式： ● 0x0: 单从核模式 ● 0x1: 行多播模式，仅 dma_get 使用 ● 0x2: 列多播模式，仅 dma_get 使用 其他：保留 [63]位要求为 0
81:64	回答字地址	DMA 传输的回答字返回地址，为发起 DMA 从核局存的内部地址。本地址为字节地址，但要求 4B 对界
87:82	保留	
95:88	多播有效向量	多播位有效时，每 bit 位对应 1 行或 1 列。为 1 时标示对应位参与多播。 行多播时仅低 4 位有效，高位保留。
127:96	主存跨步	表示远程跨步距离 remtstride。原始以字节为单位，要求 4B 倍数。转置 DMA_GET 操作仅支持 32*32*2B=2KB 或 16*16*4B=1KB 两种规格传输量。相应地远程跨步距离必须为 64B 倍数
Rb		
47:0	主存地址	DMA 传输时主存地址，以字节为单位，可能为源地址或目标地址。
Rc		

17:0	本地 LDM 地址	DMA 传输时本地 LDM 的地址，以字节为单位，可能为源地址或目标地址。

发起 DMA 过程中，需要进行描述符静态检查，可能由于描述符非法产生相应的警告或异常，具体见《SW-AI 处理器软件接口手册》。

DMA 操作访问本地 LDM 和主存过程中，可能产生地址越权、越界的数据流异常，详见《SW-AI 处理器软件接口手册》。

6.9.2 发起 RMA

格式： RMA Va.ro,Rb.rl,Rc.rl //存储器指令（杂项指令）格式

操作： {请求本地LDM和远程LDM之间的数据传输操作}

异常： 通道异常

数据流异常

助记符： RMA 发起RMA

指令描述：

请求本地 LDM 和远程 LDM 之间的数据传输操作，目前远程 LDM 仅限本核组其它从核的 LDM。Disp[11:6]用作 Rc[5:0]。RMA 命令对从核发起 RMA 操作的所有信息进行配置，使用指令中 rb 和 rc 分别配置远程 LDM 地址和本地 LDM 地址，其余配置参数均放入 Va 向量寄存器中，其中 VaF1v 和 VaF0v 分别表示向量寄存器 Va 的 2 个低 64 位。Va 的 6 个高 64 目前暂不使用。RMA 命令的格式如下图：

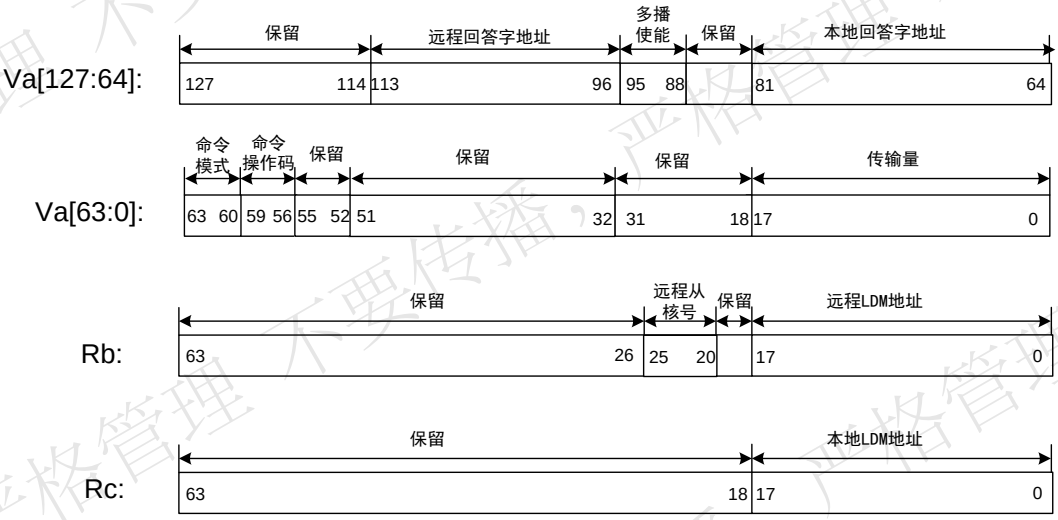


图 0-24 RMA 命令格式

采用以上格式，经常需要修改的远程 LDM 地址和本地 LDM 地址分别放在 Rb 和 Rc 寄存器，不需要和其它参数进行拼接，可直接使用；次常改变的参数，回答字地址和传输量，在 VaF0v 和 VaF1v 的低 32 位，便于改变。对 Va 其他的配置可以一次配置好，如果遇到类似的 RMA 数据传送，就可以复用 Va 寄存器。

表 0-47 RMA 命令格式详表

Bits	名称	描述

Va		
17:0	数据传输量	一次 RMA 的数据传输量，原始以字节为单位，要求 4B 倍数。传输量不允许为 0
51:18	保留	
55:52	保留	
59:56	命令操作码	RMA 命令的操作码： ● 0x0: rma_put ● 0x1: rma_get ● 0x5: rma_barrier ● 0x6: all_barrier ● 其他：保留 [59]位：强制为 0
63:60	命令模式	RMA 命令模式： ● 0x0: 单从核模式 ● 0x1: 行多播模式 ● 0x2: 列多播模式 ● 0x5: 行多播模式（本地从核接收数据） ● 0x6: 列多播模式（本地从核接收数据） 其他：保留 [63]位要求为 0
81:64	本地回答字地址	RMA 传输的本地回答字返回地址，发起命令的从核的 LDM 地址。本地址为字节地址，但要求 4B 对界
87:82	保留	
95:88	多播有效向量	行多播时表示参与多播的行，某 bit 为 1 表示对应的行参加多播，仅低 4 位有效； 列多播时表示参与多播的列，某 bit 为 1 表示对应的列参加多播；
113:96	远程回答字地址	RMA 传输的远程回答字返回地址，参与传输的其他从核的 LDM 地址。本地址为字节地址，但要求 4B 对界。
Rb		
17:0	远程 LDM 地址	RMA 传输时其他参与从核的 LDM 地址，以字节为单位。
25:20	远程从核号	单从核模式下有效，用于指定目标从核
其余	保留	
Rc		
17:0	本地 LDM 地址	RMA 传输时命令发起从核的 LDM 地址，

		以字节为单位。

发起 RMA 过程中，需要进行描述符静态检查，可能由于描述符非法产生相应的警告或异常，具体见《SW-AI 处理器软件接口手册》。

RMA 操作访问本地 LDM 和远程 LDM 过程中，可能产生地址越权、越界的数据流异常，详见《SW-AI 处理器软件接口手册》。

6.9.3 写通道

格式： WRCH Va.ro,Rb,index //存储器指令（杂项指令）格式

操作： {向指定通道中写入信息，请求特定操作}

异常： 非法指令

助记符： WRCH 写通道

指令描述：

向 8 位 Disp[7:0]为通道号所指定的通道中写入信息请求特定操作。具体操作由写入的信息描述决定。对于访问无效通道，或未实现（保留）的通道，则由核组向主核发出异常。

SW-AI 从核的写通道操作，目前只用于发送用户中断，通道号为 0x20，Va 和 Rb 的信息无意义，使用时需固定使用$31 作为源和目标寄存器。

写通道操作中，如果使用了未定义的通道号，将引起非法指令异常。

6.9.4 发起异步计算处理

格式： ACE Rb.ro,#type //存储器指令（杂项指令）格式

操作： {请求异步计算加速引擎处理特定的操作}

异常： 非法指令

数据流异常

助记符： ACE 发起异步计算处理

指令描述：

请求异步计算加速引擎处理特定的操作，处理类型由 8 位#type[7:0]指定，其中#type[7:4]代表异步计算加速引擎号，#type[3:0]代表对应计算加速引擎内的操作种类，具体操作由描述符 Rb 决定。对于无效的或保留的引擎号或操作种类，则由核组向主核发出异常。

目前仅支持 0x1 的脉动阵列计算加速引擎，其余引擎号保留。脉动阵列是一个由 32×32 排列的 cell 组成的计算阵列，每一列可执行一种乘累加操作，结果存放到每一列对应的累加器缓冲内。每一列的累加器缓冲共有 256 条目，分成为工作区和结果写回区，各 128 条目。脉动阵列计算加速引擎内的操作种类和对应的描述符 Rb 的格式如下表所示，需要注意的是，目前脉动阵列只支持访问本地 LDM 私有空间，如果访问其它空间，将产生地址越权访问异常：

表 0-48 脉动阵列计算加速引擎的操作种类和对应的描述符

#type[3:0]	操作种类	描述符 Rb
0x1	配置脉动阵列	配置脉动阵列操作模式。该配置作用前，会等待之前的所有脉动阵列运算完成，即隐含栏栅语义 位域说明： 1、[3:0]：4 位，表示脉动阵列参与的运算类型。目前支持以下几种配置。脉动阵列复位后默认的运算类型为浮点运算 16*16+32： ● 4’b0001：浮点运算 16*16+32 ● 4’b0010：整数运算 16*16+32 ● 4’b0100：整数运算(8*8+16)×2 ● 其余：保留
0x2	加载北向数据	加载北向数据，目前默认加载 32 个 64B 数据（对于浮点运算 16*16+32，64B 数据为 32 个 16 位基本 HP 浮点；对于整数运算 8*8+16，64B 数据为 64 个 8 位字节整数；对于整数运算 16*16+32，64B 数据为 32 个 16 位半字整数） **位域说明：** 1、[13:0]：14 位，加载北向数据时本地 LDM 起始访问地址，以 64B 为粒度，对应 LDM 字节地址的[19:6]； 2、[63:32]：32 位，访问北向数据掩码，每 1 位对应 1 个 64B 大小的数据加载，为 1’b1 时表示加载 512’h0，为 1’b0 时表示从 LDM 访问加载。其中[32]对应第 0 个 64B 数据，[33]对应第 1 个 64B 数据，依此类推。需要注意的是，所有需要从 LDM 访问加载的数据必须连续存放，比如[63:32]=32’h7e，则第[39]和[32]位指示加载的数据在 LDM 中是连续存放的
0x3	加载西向数据	加载西向数据，以及通知处理结果写入累加器缓冲工作区的位置等信息 **位域说明：** 1、[13:0]：14 位，表示加载西向数据时本地 LDM 起始访问地址，以 64B 为粒度，对应 LDM 字节地址的[19:6]； 2、[22:16]：7 位，表示本次计算结果在累加器缓冲的起始写入地址。目前累加器缓冲正在计算用的工作区条目数有 128 个。其中 7’h0 表示从累加器缓冲的第 0 个位置开始写入； 3、[23]：1 位，表示本次计算结果在累加器缓冲的位置是否使用新的起始写入地址。设置为 1’b1 表示使用[22:16]显式设置的起始写入地址，设置为 1’b0 表示 continue，即紧跟在前一次计算结果之后写入，此时[22:16]显式设置的写入地址无效。若前一次结果写入已经到达累加器缓冲的最后位置，则下一次加载西向数据必须显式设置累加器缓冲的起始写入地址，否则会产生不可预知的结果； 4、[39:32]：8 位，表示本次计算需要加载的西向数据量。目前低 7 位有效，其中 7’h0 表示需加载 128 个 64B 数据（对于浮点运算

		16*16+32，64B 数据为 32 个 16 位基本 HP 浮点；对于整数运算(8*8+16)×2，64B 数据为 64 个 8 位字节整数；对于整数运算 16*16+32，64B 数据为 32 个 16 位半字整数）。其它数值代表实际加载的 64B 数据个数； 5、[55:48]：8 位，表示数据加载跨步，64B 为单位。实际跨步为 8 位值+1，即值 8'h0 表示跨步为 1； 6、[56]：1 位，表示本次计算是否为最后一次计算。为 1'b1 表示本次西向数据加载完后会切换累加器缓冲的工作区和结果写回区； 7、[59]：1 位，用于区分西向数据中高 32B 还是低 32B 参与乘累加运算，该位仅在配置为整数运算(8*8+16)×2 时有效，其余运算类型配置下忽略。其中 1'b0 表示低 32B 参与乘累加运算，1'b1 表示高 32B 参与乘累加运算（由于整数运算(8*8+16)×2 时，北向数据只能加载 32 行到脉动阵列，但是一次可以做 32×2=64 列乘累加，因此要完成 64 行×64 列运算，需要拆分为 2 次 32 行×64 列运算，即将西向数据 64B 拆分为高、低 2 个 32B，分别与北向数据 64 行数据中的高、低 2 个 32 行进行 2 次乘累加运算）； 8、[63]：1 位，表示本次计算是否需要更新北向数据。为 1 表示更新
0x4	累加结果写回	写回累加器缓冲结果写回区中暂存的结果，目前默认从结果写回区的第 0 个条目开始写回 **位域说明：** 1、[13:0]：14 位，表示累加器缓冲数据写回本地 LDM 的起始访问地址，以 64B 为粒度，对应 LDM 字节地址的[19:6] 2、[23:16]：8 位，表示本次需要写回的条目数。目前低 7bit 有效，其中 7'h0 表示需写回累加缓冲 128 个条目数据，其它数值表示写回<128 个条目数据； 3、[51:32]：20 位，表示本次异步计算处理的回答字地址，对应 LDM 字节地址的[19:0]，要求 4B 对界（硬件忽略低 2 位），不支持 NULL 空回答字； 4、[63]：1 位，表示累加结果写回的精度，含义如下： ● 1'b1：浮点运算 16*16+32 和整数运算 16*16+32 时按 32 位结果写回；整数运算 8*8+16 时按 16 位结果写回； ● 1'b0：浮点运算 16*16+32 和整数运算 16*16+32 时按 16 位结果写回；整数运算 8*8+16 时按 8 位结果写回；
0x8	累加结果写回栏栅	累加结果写回栏栅，用于确保累加结果真正写回 LDM。此类型请求对描述符没有要求，可以使用$31

发起异步计算处理的过程中，需要进行描述符静态检查，可能由于描述符非法产生相应的警告或异常，具体见《SW-AI 处理器软件接口手册》。

ACE 操作访问本地 LDM 过程中，可能产生地址越权、越界的数据流异常，详见《SW-AI 处理器软件接口手册》。

6.10 专用指令（4 条）

专用指令是指为 AI 应用加入的特定指令，主要用以加快处理速度。

6.10.1 并行查找表指令（2 条）

并行查找表指令包含采用复合运算指令格式的2条指令。

表 0-49 并行查找表指令列表

序号	指令	操作码	描述
1	xplutw Va, Vb, #idx, Vd	0x9.6	扩展的并行字查表指令
2	pwrtw Va, Vb, Vc	0x9.7	并行字造表指令

说明：扩展的并行字查表指令和并行字造表指令允许同时访问 16 个 32 位粒度的数据，且允许访问地址各不相同。为此，将 LDM 划分为 16 个存储体使用，每个体 32 位宽。为方便使用，每个存储体的访问使用字节粒度的体内地址。最终访问用的体内地址通常是由基地址和偏移相加获得，两者也都是采用体内地址。普通访存指令使用的 LDM 地址称为全局地址，其与体内地址的关系如下图所示，其中体内地址为{A[17:6], A[1:0]}（A[17:6]在图中标示为“体内地址（高段）”，A[1:0]标示为“体内地址（低段）”），A[5:2]为存储体号。4B 对界的 LDM 全局地址与体地址的换算关系：

LDM 全局地址=存储体号*4B+体内地址（高段）*64B+体内地址（低段）。

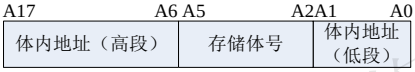


图 0-25 LDM 全局地址和体内地址关系

扩展的并行字查表指令和并行字造表指令只能访问 LDM 私有空间，访问过程中会发生数据流异常，包括：1）如果任一存储体的体内地址访问非法空间，将产生越权访问异常；2）如果任一存储体的体内地址超出对应空间容量，则发生越界异常；3）LDM 全 0 地址访问使能（CSR_LDM_CFG[0]）未打开（即取值为 1‘b0）时，如果访问 LDM 0 地址，则产生地址保护的越权访问异常。此处的 0 地址需要注意，是指最终的访问每个存储体的体内地址均为 0。

6.10.1.1 扩展的并行字查表指令

格式： XPLUTW Va.rw, Vb.rw, #type.ib, Vd.ww // 复合运算指令格式

操作：

```
FOR (i=0; i<=15; i++)
BEGIN
    temp<32*i+31:32*i>←Table_LookUp(Vav<32*i+13:32*i>,Vbv<32*i+13: 2*i>,#type)
END
Vd<511:0> ← temp<511:0>
```

异常： 数据流异常

对界 （Alignment）

查表异常

助记符： XPLUTW 扩展的并行字查表

指令描述：

Vb 中每个 32 位存放每个表（共 16 个表）的字节粒度首地址（低 14 位有效，硬件忽略高 18 位），类似于普通访存指令的基地址；Va 中每个 32 位用于产生每个表（共 16 个表）的字节粒度偏移，加上 Vb 中每 32 位的首地址后形成每个表的有效地址，并行读出存放在 LDM 中 16 个表的数据，返回的数据写回 Vd。

通过配置#type 的值可实现简单或扩展的查表操作，当#type[5:0]==6'h0 时，每个 32 位内的低 14 位（硬件忽略其高 18 位）表示一个有符号表内偏移（1B 粒度），其中[13]为偏移符号位；当#type[5:0]≠6'h0 时，每个 32 位存放一个 SP 浮点，根据#type 值选取 SP 浮点数据的整数域和小数域（#type[5:3]代表该 SP 浮点数据的整数域取多少位，#type[2:0]代表该 SP 浮点数据的小数域取多少位）转换得到表内正偏移（4B 粒度）。

需要注意的是本指令中的 Vb 的每个字分量中，使用的地址虽然也是用来访问 LDM，但与常规访存的 LDM 地址不同。Vb 中使用的是 LDM 的体内地址。

扩展的并行字查表指令可用来并行查找 AI 应用中的激活函数构造的表，加快激活值的获得。此时，满足以下任一情况时，发生查表异常：

1）类型指示符异常：主要是考虑硬件实现的简单性，对#type[5:0]的指示做了约束，包括：#type[5:3]与#type[2:0]相加之和大于 8，即大于硬件实现的索引范围；或者，(#type[5:3]==3'h0) || (#type[2:0]==3'h0)；

2）线性区溢出异常：浮点数[30]位为 1（代表尾数左移），且，（浮点数[29:27]不为全 0（表示左移位数大于 8），或，(浮点数[26:23]+4'h1) > #type[5:3]（表示左移位数超过最大可表示位数）），即整数部分超出了最大可表示的线性区。

如果每个表的有效地址不是字边界对界，则产生地址对界异常。

访存过程中还可能发生数据流异常，比如所有表的有效地址不在同一个LDM半区时，则产生数据流异常，具体参见《SW-AI处理器软件接口手册》。

6.10.1.2 并行字造表指令

格式： PWRTW Va.rw, Vb.rw, Vc.rw //复合运算指令格式

操作：

```
FOR (i=0; i<=15; i++)
BEGIN
    Vaddr_i ←{Vbv< 32*i+13: 32*i> + SEXT(Vav< 32*i+13 : 32*i>)}
     (Vaddr_i)<31 :0>←Vcv < 32*i+31: 32*i>
END
```

异常： 数据流异常

对界 （Alignment）

助记符： PWRTW 并行字写表

指令描述：

Va 中每个 32bit 内的低 14 位表示一个有符号表内偏移，其中[13]为偏移符号位，硬件忽略其高 18 位；Vb 中每个 32 位是每个表（共 16 个表）的字节粒度的首地址（低 14 位有效，硬件忽略高 18 位），类似于普通访存指令的基地址。每个 14 位的表内偏移经符号扩展后加上对应的 14 位首地址值形成每个表的有效地址 Vaddr，并根据每个表的有效地址，并行将 Vc 中每个 32 位字写入 LDM 中 16 个存储体。

需要注意的是本指令中的 Va 和 Vb 的每个字分量中，使用的地址虽然也是用来访问 LDM，但与常规访存的 LDM 地址不同。Va 和 Vb 中使用的是 LDM 的体内地址。

并行字写表指令可用于 AI 应用中 LDM 内激活表的构造。

如果每个表的有效地址不是字边界对界，则产生地址对界异常。

访存过程中还可能发生数据流异常，具体参见《SW-AI处理器软件接口手册》。

6.10.2 并行分位处理指令（2 条）

并行分位处理指令包含采用简单运算指令格式的2条指令。

表 0-50 并行分位处理指令列表

序号	指令	操作码	描述
1	vfexte30s Vb, Rc	0x9.4.4	SP 浮点向量固定抽分位（指数第 30 位）指令
2	vfdepe30s Rb, Vc	0x9.4.5	SP 浮点向量固定置分位（指数第 30 位）指令

6.10.2.1 SP 浮点向量固定抽分位指令

格式： VFEXTE30S Vb.rw, Rc.ww //简单运算指令格式

操作：

```
Rc<63:0>←0
FOR (i=0; i<=15; i++)
BEGIN
    Rc<i>←Vb<32*i+30>
END
```

异常： 无

助记符： VFEXTE30S SP浮点向量固定抽分位（指数第30位）指令

指令描述：

Vb 中每个 32bit 存放一个 SP 浮点数，并行抽取每个 SP 浮点数的指数最高位（SP 浮点数的第 30

位，其它位忽略），依次存放在 Rc 的低 16 位，Rc 的高 48 位置 0。

SP 浮点向量固定抽分位指令相当于将每个 SP 浮点分量“+2.0”或“0.0”压缩为 1 位存放，主要用于 AI 应用中 SP 浮点向量比较结果的压缩存放。

6.10.2.2 SP 浮点向量固定置分位指令

格式： VFDEPE30S Rb.rw, Vc.ww //简单运算指令格式

操作：

```
temp<31:0>←0
FOR (i=0; i<=15; i++)
BEGIN
    temp<30>←Rb<i>
    Vc<32*i+31:32*i>←temp<31:0>
END
```

异常： 无

助记符： VFDEPE30S SP浮点向量固定置分位（指数第30位）指令

指令描述：

Vc 中每个 32bit 存放一个 SP 浮点数，将 Rb 中低 16 位值并行置位到每个 SP 浮点数的指数最高位（SP 浮点数的第 30 位），Vc 中其它位置 0。

SP 浮点向量固定置分位指令相当于将多个“位 1”或“位 0”变换为多个 SP 浮点“+2.0”或“0.0”，主要用于 AI 应用中比较结果的展开恢复。

6.11 其它指令（2 条）

6.11.1 从核内部寄存器访问指令（2 条）

从核内部寄存器访问指令用于访问控制与状态寄存器 CSR，包括 RCSR、WCSR 两条指令。采用存储器指令格式，偏移值[15:8]位作为指令功能码域使用，偏移值[7:0]位用作 CSR 寄存器的索引。

表 0-51 从核内部寄存器访问指令列表

序号	指令	操作码	描述
1	rcsr Ra,idx	0x0.8	读从核内部寄存器
2	wcsr Ra,idx	0x0.9	写从核内部寄存器

RCSR/WCSR 指令可访问的从核 CSR 寄存器及其格式说明，详见本文档章节“第 7 章”。访问无效的从核 CSR 地址，将引起非法指令异常。

6.11.1.1 读从核内部状态寄存器

格式： RCSR Ra.wl,idx //存储器指令(杂项指令)格式

操作： Ra←{64位内部寄存器值}

异常： 非法指令

助记符： RCSR 读从核内部寄存器

指令描述：

读从核内部寄存器，内容保存到 Ra 中。Idx 是内部寄存器索引，共 8 位，用于确定一个唯一的内部寄存器。

6.11.1.2 写从核内部状态寄存器

格式： WCSR Ra.rl,idx //存储器指令(杂项指令)格式

操作： {64位内部寄存器值}←Ra

异常： 非法指令

助记符： WCSR 写从核内部寄存器

指令描述：

将寄存器 Ra 的值，写入从核内部寄存器。Idx 是内部寄存器索引，共 8 位，用于确定一个唯一的内部寄存器。

7 RCSR/WCSR 指令可访问的内部寄存器

错误!未找到引用源。列出了可以通过 RCSR/WCSR 指令访问的从核 CSR 寄存器的名称、符号和属性。属性为 RW 表示 RCSR/WCSR 指令均可访问，属性为 RO 表示只有 RCSR 指令可以访问。访问无效的从核 CSR 地址，将引起非法指令异常。RCSR/WCSR 指令固定从流水线 1 发射，且带有指令栏栅语义。

表 0-52 RCSR、WCSR 指令可访问的内部寄存器列表

idx[7:0]	符号	名称	属性
8’h00	CSR_CID	从核唯一 ID	RO
8’h01	CSR_ROW	从核行号	RO
8’h02	CSR_COL	从核列号	RO
8’h03	CSR_GRP	从核核组号	RO
8’h04	CSR_TC	周期计数器	RW
8’h05~8’h07	—	保留	—
8’h08	CSR_PCRC	性能计数器控制	RW
8’h09~8’h0d	—	保留	—
8’h0e	CSR_LDM_CFG	LDM 配置	RW
8’h0f	CSR_GATER_CTL	门控时钟控制	RW
8’h10	—	保留	—
8’h11	CSR_LRF_FLAG	寄存器临时变量标志位	RW
8’h12~8’h1f	—	保留	—
8’h20	CSR_PCR0	性能计数器 0（节拍计数）	RW
8’h21	CSR_PCR1	性能计数器 1	RW
8’h22	CSR_PCR2	性能计数器 2	RW
8’h23	CSR_PCR3	性能计数器 3	RW
8’h24	CSR_PCR4	性能计数器 4	RW
8’h25	CSR_PCR5	性能计数器 5	RW
8’h26	CSR_PCR6	性能计数器 6	RW
8’h27	CSR_PCR7	性能计数器 7	RW
8’h28~8’h30	—	保留	—
8’h31	CSR_LDST_CREDIT	访存缓冲剩余信用计数	RO
8’h32	CSR_DMA_CREDIT	DMA/RMA 缓冲剩余信用计数	RO
8’h33	CSR_DMA_IDLE	DMA/RMA 空闲状态	RO
8’h34~8’h7f	—	保留	—
8’h80~8’h8f	CSR_LOG3R_TRUTH	LOG3R 真值表	RW
8’h90~8’h9f	—	保留	—
8’hc4	CSR_USR_DEFINE0	用户自定义项 0	RW
8’hc5	CSR_USR_DEFINE1	用户自定义项 1	RW
8’hc6	CSR_USR_DEFINE2	用户自定义项 2	RW
8’hc7	CSR_USR_DEFINE3	用户自定义项 3	RW
8’hc8~8’hff	—	保留	—

各寄存器的详细格式定义如下：

7.1 CSR_CID（从核唯一 ID）

地址：idx[7:0]=8'h00

属性：RO

格式：

字段	*说明*
63:8	保留，读出为 0。
7:6	从核所在阵列的核组号，范围为 0~3。
5	保留，读出为 0
4:3	从核在阵列中的行号，范围为 0~3。
2:0	从核在阵列中的列号，范围为 0~7。

说明：

（1）该寄存器可以用作从核的唯一识别码。

7.2 CSR_ROW（从核行号）

地址：idx[7:0]=8'h01

属性：RO

格式：

字段	*说明*
63:2	保留，读出为 0。
1:0	从核在阵列中的行号，范围为 0~3。

7.3 CSR_COL（从核列号）

地址：idx[7:0]=8'h02

属性：RO

格式：

字段	*说明*

字段	说明
63:3	保留，读出为 0。
2:0	从核在阵列中的列号，范围为 0~7。

7.4 CSR_GRP（从核核组号）

地址：idx[7:0]=8'h03

属性：RO

格式：

字段	说明
63:2	保留，读出为 0。
1:0	从核所在阵列的核组号，范围为 0~3。

7.5 CSR_TC（周期计数器）

地址：idx[7:0]=8'h04

属性：RW

格式：

字段	属性	说明
63:56	RO	保留，读出为 0。
55:0	RW	周期计数器。无复位值，每个时钟周期自动"+1"。

7.6 CSR_PCRC（性能计数控制寄存器）

地址：idx[7:0]=8'h08

属性：RW,W1C

格式：

字段	属性	默认	说明
63:24	RO	40'b0	保留，读出为 0。
23:16	R,W1C	8'b0	8 个性能计数器的溢出状态，16~23 位分别对应性能计数器 0~7。为 1 表示对应计数器发生过计数溢出，按位写 1 清。

字段	属性	默认	说明
15:8	RW	8’b0	8 个性能计数器的溢出中断使能，8~15 位分别对应性能计数器 0~7。为 1 表示对应计数器在发生计数溢出后（本寄存器[23:16]中对应位为 1），将发起硬件中断通知主核，直到计数溢出状态（本寄存器[23:16]中对应位）被清除。
7:0	RW	8’b0	8 个性能计数器的计数使能，0~7 位分别对应性能计数器 0~7。为 1 表示对应计数器允许计数。

说明：

（1）复位后本寄存器初始值为：64’h0000_0000_0000_0000。

7.7 CSR_LDM_CFG（LDM 配置）

地址：idx[7:0]=8’h0e

属性：RW

格式：

字段	属性	默认	说明
63:16	RO	48’b0	保留，读出为 0。
15	RW	1’b0	本从核 LDM 连续共享段使能，为 1 时表示存在 LDM 连续共享段。本位为 0 时，[10:8]位无意义。
14:12	RW	3’b0	LDM 连续共享模式编码，只在“连续共享段使能”（本寄存器第[15]位）为 1 时有意义。编码具体含义如下： 3’b000：单簇模式 3’b001：双簇模式 3’b010：四簇模式 3’b100：全阵列簇模式 3’b101：全阵列行模式 其余：保留
11	RO	1’b0	保留，读出为 0
10:8	RW	3’b0	LDM 连续共享段容量编码，只在“连续共享段使能”（本寄存器第[15]位）为 1 时有意义。实际容量为 4KB*(2^编码)。编码具体含义如下： 3’b000 表示 4KB，3’b001 表示 8KB，3’b010 表示 16KB，3’b011 表示 32KB，3’b100 表示 64KB，3’b101 表示 128KB。
7	RO	1’b0	保留，读出为 0。
6:5	RO	2’b0	保留，读出为 0。
4	RO	1’b0	保留，读出为 0。

字段	属性	默认	说明
3:1	RO	3'b0	保留，读出为 0。
0	RW	1'b0	LDM 全 0 地址访问使能。为 1 时允许访存指令对 LDM 的全 0 地址进行访问；为 0 时不允许访存指令对 LDM 的全 0 地址进行访问，并导致数据流故障（越权访问异常）。

说明：

（1）复位后本寄存器初始值为：64'h0000_0000_0000_0000。

（2）软件需要保证同一共享区间内各个从核的连续共享段配置一致，硬件不对配置一致性进行检查；

（3）主核可通过访问从核 IO 寄存器 SPE_EBXIO_CTRL，对本配置进行修改。由于主从核执行的异步性，不建议在从核运行过程中，通过主核动态修改配置；

（4）从核可通过 WCSR 指令动态修改配置，但在修改前需要保证影响范围内的从核都没有未完成的访存请求和 DMA/RMA 请求。具体流程参见《软件接口手册》"4.3.4.4.从核 LDM 空间代换与保护"中有关内容。

（5）针对第（4）点，硬件实现时尽最大可能做了检查（本从核存在未完成的访存请求或 DMA、RMA 请求时，对 CSR_LDM_CFG 的 WCSR 访问将导致非法指令异常），但并不完备。

7.8 CSR_GATER_CTL（门控时钟控制）

地址：idx[7:0]=8'h0f

属性：RW

格式：

字段	属性	默认	说明
63:0	RW	64'b0	待定

说明：

（1）本寄存器中，每位对应一类门控时钟，为 1 时表示门控时钟持续翻转，不再受对应门控使能信号控制，为 0 时表示门控时钟受对应门控使能信号控制。

（2）复位后本寄存器初始值为：64'h0000_0000_0000_0000，即不对任何门控时钟进行特殊控制。

（3）从核自身可以通过 RCSR、WCSR 指令来完成对本寄存器的读写访问。主核和维护部件可以通过 IO 请求来完成对本寄存器的读写访问。外界 IO 请求与从核流水线执行 RCSR/WCSR 或使用配置信息，是两个独立的异步事件。硬件不保证其先后关系。

（4）根据要求，本寄存器只受上电复位控制，不受冷复位和从核阵列软复位控制。

7.9 CSR_LRF_FLAG（寄存器临时变量标志位）

地址：idx[7:0]=8'h11

属性：RW

格式：

字段	属性	默认	说明
63:0	RW	64'b0	指示对应寄存器是否为临时变量，为 0 时表示不为临时变量，为 1 时表示临时变量。

说明：

（1）复位后本寄存器初始值为：64'h0000_0000_0000_0000。

7.10 CSR_PCR0（性能计数器 0）

地址：idx[7:0]=8'h20

属性：RW

格式：

字段	属性	默认	说明
63:61	RO	3'h0	保留，读出为 0。
60:56	RO	5'h0	保留，读出为 0。性能计数器 0 固定用作节拍计数器。
55:0	RW	x	性能计数器 0 的当前计数值。无复位值。

7.11 CSR_PCR1（性能计数器 1）

地址：idx[7:0]=8'h21

属性：RW

格式：

字段	属性	默认	说明
63:61	RO	3'h0	保留，读出为 0。
60:56	RW	5'h1f	性能计数器 1 的计数事件选择。无复位值。具体定义如下（待定）：
55:0	RW	x	性能计数器 1 的当前计数值。无复位值。

7.12 CSR_PCR2（性能计数器 2）

地址：idx[7:0]=8'h22

属性：RW

格式：

字段	属性	默认	说明
63:61	RO	3'h0	保留，读出为 0。
60:56	RW	5'h1f	性能计数器 2 的计数事件选择。无复位值。具体定义如下（待定）：
55:0	RW	x	性能计数器 2 的当前计数值。无复位值。

7.13 CSR_PCR3（性能计数器 3）

地址：idx[7:0]=8'h23

属性：RW

格式：

字段	属性	默认	说明
63:61	RO	3'h0	保留，读出为 0。
60:56	RW	5'h1f	性能计数器 3 的计数事件选择。无复位值。具体定义如下（待定）：
55:0	RW	x	性能计数器 3 的当前计数值。无复位值。

7.14 CSR_PCR4（性能计数器 4）

地址：idx[7:0]=8'h24

属性：RW

格式：

字段	属性	默认	说明
63:61	RO	3'h0	保留，读出为 0。
60:56	RW	5'h1f	性能计数器 4 的计数事件选择。无复位值。具体定义如下（待定）：
55:0	RW	x	性能计数器 4 的当前计数值。无复位值。

7.15 CSR_PCR5（性能计数器 5）

地址：idx[7:0]=8'h25

属性：RW

格式：

字段	属性	默认	说明
63:61	RO	3'h0	保留，读出为 0。
60:56	RW	5'h1f	性能计数器 5 的计数事件选择。无复位值。具体定义如下（待定）：
55:0	RW	x	性能计数器 5 的当前计数值。无复位值。

7.16 CSR_PCR6（性能计数器 6）

地址：idx[7:0]=8'h26

属性：RW

格式：

字段	属性	默认	说明
63:61	RO	3'h0	保留，读出为 0。
60:56	RW	5'h1f	性能计数器 6 的计数事件选择。无复位值。具体定义如下（待定）：
55:0	RW	x	性能计数器 6 的当前计数值。无复位值。

7.17 CSR_PCR7（性能计数器 7）

地址：idx[7:0]=8'h27

属性：RW

格式：

字段	属性	默认	说明
63:61	RO	3'h0	保留，读出为 0。
60:56	RW	5'h1f	性能计数器 7 的计数事件选择。无复位值。具体定义如下（待

字段	属性	默认	说明
			定）：
55:0	RW	x	性能计数器 7 的当前计数值。无复位值。

7.18 CSR_LDST_CREDIT（访存缓冲剩余信用计数）

地址：idx[7:0]=8’h31

属性：RO

格式：

字段	属性	默认	说明
63:3	RO	61’b0	保留，读出为 0。
2:0	RO	3’h6	当前的访存缓冲剩余信用，取值范围 0~6。为 0 表示访存请求缓冲满，后续的访存请求将导致流水线停顿。

7.19 CSR_DMA_CREDIT（DMA/RMA 缓冲剩余信用计数）

地址：idx[7:0]=8’h32

属性：RO

格式：

字段	属性	默认	说明
63:3	RO	61’b0	保留，读出为 0。
2:0	RO	3’h4	当前的 DMA/RMA 剩余信用，取值范围 0~4。为 0 表示 DMA/RMA 请求缓冲满，后续的 DMA/RMA 请求将导致流水线停顿。

7.20 CSR_DMA_IDLE（DMA/RMA 空闲状态）

地址：idx[7:0]=8’h33

属性：RO

格式：

字段	属性	默认	说明

字段	属性	默认	说明
63:1	RO	63’b0	保留，读出为 0。
0	RO	1’b0	DMA/RMA 空闲状态，为 1 表示本从核之前发出的所有 DMA/RMA 请求都已经完成。

7.21 CSR_LOG3R_TRUTH（LOG3R 真值表）

地址：idx[7:0]=8’h80~8’h8f

属性：RW

格式：

字段	属性	默认	说明
63:8	RO	56’b0	保留，读出为 0。
7:0	RW	x	LOG3R 指令的 8 位真值表，无复位值。

说明：

（1）从核内实现 16 个 LOG3R 指令的真值表配置寄存器，格式完全一致，如上表所示。

（2）本寄存器无复位值，软件在执行 LOG3R 指令之前，必须通过 WCSR 指令对对应的真值表寄存器进行正确配置，否则将导致不可预知的错误。

7.22 CSR _USR_DEFINE0（用户自定义项 0）

地址：idx[7:0]=8’hc4

属性：RW

格式：

字段	属性	默认	说明
63: 0	RW	x	无复位值

7.23 CSR _USR_DEFINE1（用户自定义项 1）

地址：idx[7:0]=8’hc5

属性：RW

格式：

字段	属性	默认	说明
63: 0	RW	x	无复位值

7.24 CSR _USR_DEFINE2（用户自定义项 2）

地址：idx[7:0]=8'hc6

属性：RW

格式：

字段	属性	默认	说明
63: 0	RW	x	无复位值

7.25 CSR _USR_DEFINE3（用户自定义项 3）

地址：idx[7:0]=8'hc7

属性：RW

格式：

字段	属性	默认	说明
63: 0	RW	x	无复位值

8 附录 1 指令按操作码及操作类型排序列表

序号	操作码	指令	格式	指令类型	说明
1	0x0.1	ace	Rb,#type	通道类	发起异步计算处理
2	0x0.2	wrch	Va,Rb,idx		写通道
3	0x0.3	memb		存储栏栅	存储器栏栅
4	0x0.4.0	synr	Ra	同步类	核组内行同步
5	0x0.4.1	sync	Ra		核组内列同步
6	0x0.4.2	synp	Ra		点点同步
7	0x0.6	jmp	Ra,Rb[,disp]	转移	寄存器间接转移
8	0x0.7	halt		停机	停机
9	0x0.8	rcsr	Ra,idx	CSR 访问	读内部寄存器
10	0x0.9	wcsr	Ra,idx		写内部寄存器
11	0x0.a	rma	Va,Rb,Rc	通道类	RMA
12	0x0.b	dma	Va,Rb,Rc		DMA
13	0x0.c	faal	Ra,disp(Rb)	原子操作	长字取并加 1
14	0x0.d	updt	Va,disp(Rb)		原子更新
15	0x0.e	casw	Va,disp(Rb)	比较并置换	字比较并置换
16	0x0.f	casl	Va,disp(Rb)		长字比较并置换
17	0x1	br	Ra,disp	转移类	无条件转移
18	0x2.1	beq	Ra,disp	整数条件转移	若 Ra＝“0”则转移
19	0x2.2	blt	Ra,disp		若 Ra＜“0”则转移
20	0x2.3	ble	Ra,disp		若 Ra≤“0”则转移
21	0x2.4	bgt	Ra,disp		若 Ra＞“0”则转移
22	0x2.5	bge	Ra,disp		若 Ra≥“0”则转移
23	0x2.6	bne	Ra,disp		若 Ra≠“0”则转移
24	0x2.9	fbeq	Ra,disp	浮点条件转移	若 Ra＝“0”则转移
25	0x2.a	fblt	Ra,disp		若 Ra＜“0”则转移
26	0x2.b	fble	Ra,disp		若 Ra≤“0”则转移
27	0x2.c	fbgt	Ra,disp		若 Ra＞“0”则转移
28	0x2.d	fbge	Ra,disp		若 Ra≥“0”则转移
29	0x2.e	fbne	Ra,disp		若 Ra≠“0”则转移
30	0x3.0	vconw	Va,Vb,Vc,Vd	向量整理	字向量或 SP 浮点向量拼接
31	0x3.2	vconh	Va,Vb,Rc,Vd		半字向量拼接
32	0x3.4	vcpyw	Va,Vb,Rc,Vd		字向量拷贝
33	0x3.5	vfcpys	Va,Vb,Rc,Vd		SP 浮点拷贝
34	0x3.6	vcpyh	Va,Vb,Rc,Vd		半字向量拷贝
35	0x3.8	vinsw	Ra,Vb,#c,Vd		字向量元素插入
36	0x3.9	vfinss	Ra,Vb,#c,Vd		SP 浮点向量元素插入

序号	操作码	指令	格式	指令类型	说明
37	0x3.a	vinsh	Ra,Vb,#c,Vd		半字向量元素插入
38	0x3.c	vextw	Vb,#c,Rd		字向量元素抽取
39	0x3.d	vfexts	Vb,#c,Rd		SP 浮点向量元素抽取
40	0x3.e	vexth	Vb,#c,Rd		半字向量元素抽取
41	0x4.0.0	addw	Ra,Rb,Rc		字加
42	0x4.0.1	subw	Ra,Rb,Rc		字减
43	0x4.0.2	addl	Ra,Rb,Rc		长字加
44	0x4.0.3	subl	Ra,Rb,Rc		长字减
45	0x4.0.8	sll	Ra,Rb,Rc		逻辑左移
46	0x4.0.9	srl	Ra,Rb,Rc		逻辑右移
47	0x4.0.a	sra	Ra,Rb,Rc		算术右移
48	0x4.0.b	rotl	Ra,Rb,Rc		循环左移
49	0x4.0.c	s32addl	Ra,Rb,Rc		乘 32 的长字加
50	0x4.0.d	s32subl	Ra,Rb,Rc		乘 32 的长字减
51	0x4.0.e	s64addl	Ra,Rb,Rc		乘 64 的长字加
52	0x4.0.f	s64subl	Ra,Rb,Rc		乘 64 的长字减
53	0x4.1.0	addw	Ra,#b,Rc		字加
54	0x4.1.1	subw	Ra,#b,Rc		字减
55	0x4.1.2	addl	Ra,#b,Rc		长字加
56	0x4.1.3	subl	Ra,#b,Rc		长字减
57	0x4.1.8	sll	Ra,#b,Rc		逻辑左移
58	0x4.1.9	srl	Ra,#b,Rc		逻辑右移
59	0x4.1.a	sra	Ra,#b,Rc	标量整数	算术右移
60	0x4.1.b	rotl	Ra,#b,Rc		循环左移
61	0x4.1.c	s32addl	Ra,#b,Rc		乘 32 的长字加
62	0x4.1.d	s32subl	Ra,#b,Rc		乘 32 的长字减
63	0x4.1.e	s64addl	Ra,#b,Rc		乘 64 的长字加
64	0x4.1.f	s64subl	Ra,#b,Rc		乘 64 的长字减
65	0x4.2.0	s4addl	Ra,Rb,Rc		乘 4 的长字加
66	0x4.2.1	s4subl	Ra,Rb,Rc		乘 4 的长字减
67	0x4.2.2	s8addl	Ra,Rb,Rc		乘 8 的长字加
68	0x4.2.3	s8subl	Ra,Rb,Rc		乘 8 的长字减
69	0x4.2.4	mulw	Ra,Rb,Rc		字乘
70	0x4.2.5	umulw	Ra,Rb,Rc		无符号字乘
71	0x4.2.6	mull	Ra,Rb,Rc		长字乘
72	0x4.2.7	umulh	Ra,Rb,Rc		无符号长字乘高位
73	0x4.2.9	cmpeq	Ra,Rb,Rc		有符号数等于比较
74	0x4.2.a	cmplt	Ra,Rb,Rc		有符号数小于比较
75	0x4.2.b	cmple	Ra,Rb,Rc		有符号数小于等于比较
76	0x4.2.e	cmpult	Ra,Rb,Rc		无符号数小于比较
77	0x4.2.f	cmpule	Ra,Rb,Rc		无符号数小于等于比较
78	0x4.3.0	s4addl	Ra,#b,Rc		乘 4 的长字加

序号	操作码	指令	格式	指令类型	说明
79	0x4.3.1	s4subl	Ra,#b,Rc		乘 4 的长字减
80	0x4.3.2	s8addl	Ra,#b,Rc		乘 8 的长字加
81	0x4.3.3	s8subl	Ra,#b,Rc		乘 8 的长字减
82	0x4.3.4	mulw	Ra,#b,Rc		字乘
83	0x4.3.5	umulw	Ra,#b,Rc		无符号字乘
84	0x4.3.6	mull	Ra,#b,Rc		长字乘
85	0x4.3.7	umulh	Ra,#b,Rc		无符号长字乘高位
86	0x4.3.9	cmpeq	Ra,#b,Rc		有符号数等于比较
87	0x4.3.a	cmplt	Ra,#b,Rc		有符号数小于比较
88	0x4.3.b	cmple	Ra,#b,Rc		有符号数小于等于比较
89	0x4.3.e	cmpult	Ra,#b,Rc		无符号数小于比较
90	0x4.3.f	cmpule	Ra,#b,Rc		无符号数小于等于比较
91	0x4.4.0~f	log2x<truth>	Ra,Rb,Rc		两操作数任意逻辑运算
92	0x4.5.0~f	log2x<truth>	Ra,#b,Rc		两操作数任意逻辑运算
93	0x4.6.0	ctpop	Rb,Rc		数“1”
94	0x4.6.1	ctlz	Rb,Rc		数头“0”
95	0x4.6.2	cttz	Rb,Rc		数尾“0”
96	0x4.6.4	zap	Ra,Rb,Rc		字节清除
97	0x4.6.5	zapnot	Ra,Rb,Rc		字节反码清除
98	0x4.6.6	sextb	Rb,Rc		字节符号扩展
99	0x4.6.7	sexth	Rb,Rc		半字符号扩展
100	0x4.7.4	zap	Ra,#b,Rc		字节清除
101	0x4.7.5	zapnot	Ra,#b,Rc		字节反码清除
102	0x4.7.6	sextb	#b,Rc		字节符号扩展
103	0x4.7.7	sexth	#b,Rc		半字符号扩展
104	0x4.8	conl	Ra,Rb,Rc,Rd	标量拼接	标量按字节拼接长字
105	0x4.9	insb	Ra,Rb,#c,Rd		标量字节粒度插入
106	0x4.a	seleq	Ra,Rb,Rc，Rd	标量整数	若 Ra=“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd
107	0x4.b	seleq	Ra,Rb,#c，Rd		若 Ra=“0”则选择 Rc 写入 Rd，否则将 6 位立即数#c 零扩展到 64 位写入 Rd
108	0x4.c	sellt	Ra,Rb,Rc，Rd		若 Ra＜“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd
109	0x4.d	sellt	Ra,Rb,#c，Rd		若 Ra＜“0”则选择 Rc 写入 Rd，否则将 6 位立即数#c 零扩展到 64 位写入 Rd
110	0x4.e	selle	Ra,Rb,Rc，Rd		若 Ra≤“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd
111	0x4.f	selle	Ra,Rb,#c，Rd		若 Ra≤“0”则选择 Rc 写入 Rd，否则将 6 位立即数#c 零扩展到 64 位写入 Rd
112	0x5.0.0	vaddw	Va,Vb,Vc	向量整	字向量加

序号	操作码	指令	格式	指令类型	说明
113	0x5.0.1	vsubw	Va,Vb,Vc	数	字向量减
114	0x5.0.2	vmulw	Va,Vb,Vc		字向量乘
115	0x5.0.4	vaddh	Va,Vb,Vc		半字向量加
116	0x5.0.5	vsubh	Va,Vb,Vc		半字向量减
117	0x5.0.6	vmulh	Va,Vb,Vc		半字向量乘
118	0x5.1.0	vaddw	Va,#b,Vc		字向量加
119	0x5.1.1	vsubw	Va,#b,Vc		字向量减
120	0x5.1.2	vmulw	Va,#b,Vc		字向量乘
121	0x5.1.4	vaddh	Va,#b,Vc		半字向量加
122	0x5.1.5	vsubh	Va,#b,Vc		半字向量减
123	0x5.1.6	vmulh	Va,#b,Vc		半字向量乘
124	0x5.2.0~f	vlog2x<truth>	Va,Vb,Vc		两操作数 SIMD 可重构运算
125	0x5.3.0~f	vlog2x<truth>	Va,#b,Vc		两操作数 SIMD 可重构运算
126	0x5.4.0	vcmpeqw	Va,Vb,Vc		字向量等于比较
127	0x5.4.1	vcmpltw	Va,Vb,Vc		字向量小于比较
128	0x5.4.2	vcmplew	Va,Vb,Vc		字向量小于等于比较
129	0x5.4.4	vcmpeqh	Va,Vb,Vc		半字向量等于比较
130	0x5.4.5	vcmplth	Va,Vb,Vc		半字向量小于比较
131	0x5.4.6	vcmpleh	Va,Vb,Vc		半字向量小于等于比较
132	0x5.5.0	vcmpeqw	Va,#b,Vc		字向量等于比较
133	0x5.5.1	vcmpltw	Va,#b,Vc		字向量小于比较
134	0x5.5.2	vcmplew	Va,#b,Vc		字向量小于等于比较
135	0x5.5.4	vcmpeqh	Va,#b,Vc		半字向量等于比较
136	0x5.5.5	vcmplth	Va,#b,Vc		半字向量小于比较
137	0x5.5.6	vcmpleh	Va,#b,Vc		半字向量小于等于比较
138	0x5.6	vseleqw	Va,Vb,Vc,Vd		字向量等于“0”条件判断并选择
139	0x5.7	vseleqh	Va,Vb,Vc,Vd		半字向量等于“0”条件判断并选择
140	0x5.8	vnmah	Va,Vb,Vc,Vd	向量整数指令	半字向量负乘加
141	0x5.9	vnmaw	Va,Vb,Vc,Vd		字向量负乘加
142	0x5.a	selltw	Va,Vb,Vc,Vd		字向量小于“0”条件判断并选择
143	0x5.b	sellth	Va,Vb,Vc,Vd		半字向量小于“0”条件判断并选择
144	0x5.c	vmah	Va,Vb,Vc,Vd		半字向量乘加
145	0x5.d	vmaw	Va,Vb,Vc,Vd		字向量乘加
146	0x5.e	vsellew	Va,Vb,Vc,Vd		字向量小于等于“0”条件判断并选择
147	0x5.f	vselleh	Va,Vb,Vc,Vd		半字向量小于等于“0”条件判断并选择
148	0x6.0~f	log3r<idx>	Ra,Rb,Rc,Rd	标量三操作数可重构	三操作数任意逻辑运算
149	0x7.0~f	vlog3r<idx>	Va,Vb,Vc,Vd	向量三操作数	三操作数 SIMD 可重构运算

序号	操作码	指令	格式	指令类型	说明
				可重构	
150	0x8.0.0	fadds	Ra,Rb,Rc	标量浮点指令	SP 加
151	0x8.0.1	faddd	Ra,Rb,Rc		DP 加
152	0x8.0.2	fsubs	Ra,Rb,Rc		SP 减
153	0x8.0.3	fsubd	Ra,Rb,Rc		DP 减
154	0x8.0.4	fmuls	Ra,Rb,Rc		SP 乘
155	0x8.0.5	fmuld	Ra,Rb,Rc		DP 乘
156	0x8.0.8	fcpys	Ra,Rb,Rc		拷贝符号
157	0x8.0.9	fcpyse	Ra,Rb,Rc		拷贝符号和指数
158	0x8.0.a	fcpysn	Ra,Rb,Rc		拷贝符号的反码
159	0x8.0.c	fcmpun	Ra,Rb,Rc		浮点不可排序比较
160	0x8.0.d	fcmpeq	Ra,Rb,Rc		浮点等于比较
161	0x8.0.e	fcmplt	Ra,Rb,Rc		浮点小于比较
162	0x8.0.f	fcmple	Ra,Rb,Rc		浮点小于等于比较
163	0x8.1.0	fcvtds	Rb,Rc		双精度转单精度
164	0x8.1.1	fcvtsd	Rb,Rc		单精度转双精度
165	0x8.1.2	fcvtls	Rb,Rc		长字转单精度
166	0x8.1.3	fcvtld	Rb,Rc		长字转双精度
167	0x8.1.4	fcvtdl	Rb,Rc		双精度转长字整数
168	0x8.1.5	fcvtdlr	Ra,#b,Rc		双精度转长字整数，可自带舍入模式（由 8 位立即数#b 的低[2:0]位指定）
169	0x8.1.6	fcvtsh	Rb,Rc		单精度转半精度
170	0x8.1.7	fcvths	Rb,Rc		半精度转单精度
171	0x8.1.8	fdivs	Ra,Rb,Rc		SP 除
172	0x8.1.9	fdivd	Ra,Rb,Rc		DP 除
173	0x8.1.a	fsqrts	Ra,Rc		SP 平方根
174	0x8.1.b	fsqrtd	Ra,Rc		DP 平方根
175	0x8.1.c	rfpcr	Vc		读浮点控制寄存器 FPCR
176	0x8.1.d	wfpcr	Va		写浮点控制寄存器 FPCR
177	0x8.5	fseleq	Ra,Rb,Rc,Rd		若 Ra＝“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd
178	0x8.6	fsellt	Ra,Rb,Rc,Rd		若 Ra＜“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd
179	0x8.7	fselle	Ra,Rb,Rc,Rd		若 Ra≤“0”则选择数据 Rb 写入 Rd，否则选择 Rc 写入 Rd
180	0x8.8	fmas	Ra,Rb,Rc,Rd		SP 乘加
181	0x8.9	fmad	Ra,Rb,Rc,Rd		DP 乘加
182	0x8.a	fmss	Ra,Rb,Rc,Rd		SP 乘减
183	0x8.b	fmsd	Ra,Rb,Rc,Rd		DP 乘减
184	0x8.c	fnmas	Ra,Rb,Rc,Rd		SP 负乘加
185	0x8.d	fnmad	Ra,Rb,Rc,Rd		DP 负乘加
186	0x8.e	fnmss	Ra,Rb,Rc,Rd		SP 负乘减

序号	操作码	指令	格式	指令类型	说明
187	0x8.f	fnmsd	Ra,Rb,Rc,Rd		DP 负乘减
188	0x9.0.0	vfadds	Va,Vb,Vc	向量浮点	SP 浮点向量加
189	0x9.0.2	vfsubs	Va,Vb,Vc		SP 浮点向量减
190	0x9.0.4	vfmuls	Va,Vb,Vc		SP 浮点向量乘
191	0x9.0.8	vfcpyss	Va,Vb,Vc		SP 浮点向量拷贝符号
192	0x9.0.9	vfcpyses	Va,Vb,Vc		SP 浮点向量拷贝符号和指数
193	0x9.0.a	vfcpysns	Va,Vb,Vc		SP 浮点向量拷贝符号的反码
194	0x9.0.c	vfcmpuns	Va,Vb,Vc		SP 浮点向量不可排序比较
195	0x9.0.d	vfcmpeqs	Va,Vb,Vc		SP 浮点向量等于比较
196	0x9.0.e	vfcmplts	Va,Vb,Vc		SP 浮点向量小于比较
197	0x9.0.f	vfcmples	Va,Vb,Vc		SP 浮点向量小于等于比较
198	0x9.1	vfseleqs	Va,Vb,Vc,Vd		SP 浮点向量等于“0”条件判断并选择
199	0x9.2	vfsellts	Va,Vb,Vc,Vd		SP 浮点向量小于“0”条件判断并选择
200	0x9.3	vfselles	Va,Vb,Vc,Vd		SP 浮点向量小于等于“0”条件判断并选择
201	0x9.4.0	vfcvtsh	Vb,Vc		浮点向量单精度转半精度
202	0x9.4.1	vfcvths	Vb,Vc		浮点向量半精度转单精度
203	0x9.4.4	vfexte30s	Vb,Rc	专用指令	SP 浮点向量固定抽分位
204	0x9.4.5	vfdepe30s	Rb,Vc		SP 浮点向量固定置分位
205	0x9.6	xplutw	Va,Vb,#t,Vd		扩展的并行查表
206	0x9.7	pwrtw	Va,Vb,Vc		并行造表
207	0x9.8	vfmas	Va,Vb,Vc,Vd	向量浮点	SP 浮点向量乘加
208	0x9.a	vfmss	Va,Vb,Vc,Vd		SP 浮点向量乘减
209	0x9.c	vfnmas	Va,Vb,Vc,Vd		SP 浮点向量负乘加
210	0x9.e	vfnmss	Va,Vb,Vc,Vd		SP 浮点向量负乘减
211	0xa.0	vfldh_u	Ra,disp(Rb)	向量不对界访存	HP 浮点向量不对界装入
212	0xa.1	vldb_u	Ra,disp(Rb)		字节整数向量不对界装入
213	0xa.2	vldh_u	Ra,disp(Rb)		半字整数向量不对界装入
214	0xa.3	vldw_u	Ra,disp(Rb)		字整数向量不对界装入
215	0xa.4	fldh	Ra,disp(Rb)	小粒度访存	HP 浮点读
216	0xa.5	ldbu	Ra,disp(Rb)		装入字节并作“0”扩展
217	0xa.6	ldhu	Ra,disp(Rb)		装入半字并作“0”扩展
218	0xa.7	ldl_u	Ra,disp(Rb)	标量不对界访存	长字不对界装入
219	0xa.8	vfsth_u	Ra,disp(Rb)	向量不对界访存	HP 浮点向量不对界存储
220	0xa.9	vstb_u	Ra,disp(Rb)		字节整数向量不对界存储
221	0xa.a	vsth_u	Ra,disp(Rb)		半字整数向量不对界存储
222	0xa.b	vstw_u	Ra,disp(Rb)		字整数向量不对界存储
223	0xa.c	fsth	Ra,disp(Rb)	小粒度访存	存储 HP 浮点
224	0xa.d	stb	Ra,disp(Rb)		存储字节
225	0xa.e	sth	Ra,disp(Rb)		存储半字
226	0xa.f	stl_u	Ra,disp(Rb)	标量不	长字不对界存储

序号	操作码	指令	格式	指令类型	说明
				对界访存	
227	0xb.0	flds	Ra,disp(Rb)	浮点访存	SP 浮点读
228	0xb.1	fsts	Ra,disp(Rb)		SP 浮点写
229	0xb.2	ldw	Ra,disp(Rb)	对界访存	装入字并符号扩展
230	0xb.3	stw	Ra,disp(Rb)		存储字
231	0xc.4	ldl	Ra,disp(Rb)		装入长字
232	0xc.5	stl	Ra,disp(Rb)		存储长字
233	0xd.0	vfldh	Ra,disp(Rb)	向量存储访问	HP 浮点向量装入
234	0xd.1	vfsth	Ra,disp(Rb)		HP 浮点向量存储
235	0xd.2	vldw	Ra,disp(Rb)		字向量装入
236	0xd.3	vstw	Ra,disp(Rb)		字向量存储
237	0xd.4	vldh	Ra,disp(Rb)		半字向量装入
238	0xd.5	vsth	Ra,disp(Rb)		半字向量存储
239	0xd.6	vldb	Ra,disp(Rb)		字节向量装入
240	0xd.7	vstb	Ra,disp(Rb)		字节向量存储
241	0xe	ldi	Ra,disp(Rb)	装入立即数	立即数装入
242	0xf	ldih	Ra,disp(Rb)		立即数高段装入

9 附录 2 指令分操作码列表

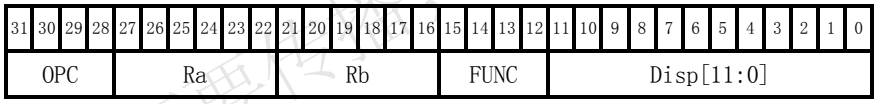
下表是从核指令集中指令操作码 OPC[3:0]使用情况。

		OPC[3:2]			
		0x0	**0x1**	**0x2**	**0x3**
OPC[1:0]	**0x0**	WRCH、ACE、MEMB、同步、JMP、HALT、RCSR、WCSR、RMA、DMA、原子操作、CAS	标量整数、LOG2X 标量拼接	标量浮点	LDL、STL
	0x1	BR	向量整数、专用指令、VLOG2X	向量浮点	向量对界访存
	0x2	整数条件转移、浮点条件转移	LOG3R	FLDH、FSTH、LDBU、LDHU、STB、STH、标量不对界访存	LDI
	0x3	向量整理（含向量拼接）	VLOG3R	FLDS、FSTS、LDW、STW	LDIH

9.1 杂项指令-4’h0

杂项指令的操作码 OPC[3:0]为 0x0，包括发起异步计算 ACE、写通道 WRCH、访存栏栅 MEMB、同步 SYN、寄存器间接转移 JMP、停机 HALT、CSR 访问、DMA、原子操作、比较并转换指令，采用杂项指令格式，指令格式如下。

杂项指令格式



各指令根据功能码 FUNC[3:0]区分功能，对应情况如下表所示：

		FUNC[3:2]			
		0x0	**0x1**	**0x2**	**0x3**
FUNC[1:0]	**0x0**	保留	SYN	RCSR	FAAL
	0x1	ACE	保留	WCSR	UPDT
	0x2	WRCH	JMP	RMA	CASW
	0x3	MEMB	HALT	DMA	CASL

各类指令对 Disp[11:0]的使用方式不同，具体如下：

1） WRCH 指令

Disp[11:8]无意义；Disp[7:0]为通道号索引：目前仅使用 8’h20（表示用户中断）；

2） ACE 指令

Disp[11:8]无意义；Disp[7:0]为处理类型；

3） SYN 指令

Disp[11:10]用作功能码，区分不同类型的同步指令，如下表所示：

		DISP[11]	
		0x0	0x1
DISP[10]	0x0	SYNR	SYNP
	0x1	SYNC	保留

4） JMP 指令

Disp[11:0]用作转移预测的地址偏移；

5） RCSR/WCSR 指令

Disp[11:8]无意义；Disp[7:0]为 CSR 寄存器索引；

6） DMA 指令

Disp[5:0]无意义；Disp[11:6]用作 Rc[5:0]；

7） RMA 指令

Disp[5:0]无意义；Disp[11:6]用作 Rc[5:0]；

8） FAAL/UPDT 指令

Disp[11:0]用作字节访存地址的立即数偏移[11:0]。

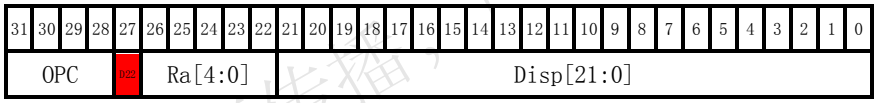
9） 比较并置换指令

Disp[11:0]用作字节访存地址的立即数偏移[11:0]。

9.2 无条件转移指令 BR-4’h1

BR 指令的操作码 OPC[3:0]为 0x1，采用转移指令格式，指令格式如下。

转移指令格式

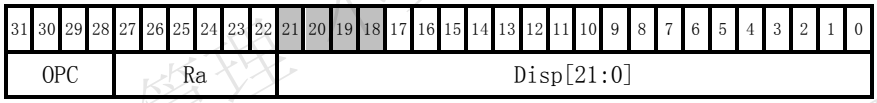


要求目标寄存器限制在标量寄存器（$0~$31）范围内，6 位目标寄存器号的最高位（即 32 位指令编码的第 27 位）与指令编码的 Disp[21:0]位共同构成 23 位的有符号 PC 偏移[24:2]，可实现±16MB 代码空间范围内的跳转。

9.3 条件转移指令-4’h2

条件转移指令的操作码 OPC[3:0]为 0x2，采用转移指令格式，指令格式如下。

转移指令格式



其中 Disp[21:18]位置用作功能码，区分不同的指令，Disp[17:0]为立即数指令偏移地址的[19:2]位。功能码与指令的对应关系如下表所示：

		DISP[21:20]			
		0x0	0x1	0x2	0x3
DISP[19:18]	0x0	保留	BGT	保留	FBGT
	0x1	BEQ	BGE	FBEQ	FBGE
	0x2	BLT	BNE	FBLT	FBNE
	0x3	BLE	保留	FBLE	保留

9.4 向量整理指令-4'h3

向量整理指令的操作码 OPC[3:0]为 0x3，使用下列两种指令格式。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
复合运算(寄存器格式)	OPC				Ra						Rb						FUNC				Rc						Rd					
复合运算(立即数格式)	OPC				Ra						Rb						FUNC				#c[5:0]						Rd					

根据功能码 FUNC[3]区分所使用的指令格式，具体如下：

FUNC[3]==1'b0：复合运算（寄存器）指令格式

FUNC[3]==1'b1：复合运算（立即数）指令格式

各指令根据功能码 FUNC[3:0]区分功能，对应情况如下表所示：

		FUNC[3]	
		0x0	0x1
FUNC[2:0]	0x0	VCONW(R)	VINSW(I)
	0x1	保留	VFINSS(I)
	0x2	VCONH(R)	VINSH(I)
	0x3	保留	保留
	0x4	VCPYW(R)	VEXTW(I)
	0x5	VFCPYS(R)	VFEXTS(I)
	0x6	VCPYH(R)	VEXTH(I)
	0x7	保留	保留

9.5 标量整数指令-4'h4

标量整数指令的操作码 OPC[3:0]为 0x4，使用下列四种指令格式。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
简单运算(寄存器格式)	OPC				Ra						Rb						FUNC				SUBF				0	0	Rc					
简单运算(立即数格式)	OPC				Ra						#b[5:0]						FUNC				SUBF				#b[7:6]		Rc					
复合运算(寄存器格式)	OPC				Ra						Rb						FUNC				Rc						Rd					

复合运算（立即数格式）	OPC	Ra	Rb	FUNC	#c[5:0]	Rd

根据功能码 FUNC[3:0]区分所使用的指令格式，具体如下：

FUNC[3]==0/1：简单运算/复合运算指令格式

FUNC[3:0]==0/2/4/6/8/a/c/e：寄存器指令格式

FUNC[3:0]==1/3/5/7/9/b/d/f：立即数指令格式

各指令根据功能码 FUNC[3:0]和子功能码 SUBF[3:0]区分功能，对应情况如下表所示：

		FUNC[3:0]							
		0x0 0x1	0x4 0x5	0x2 0x3	0x6 0x7	0x8	0x9	0xa 0xb	0xc 0xd	0xe 0xf
SUBF[3:0]	0x0	ADDW	LOG2X0	S4ADDL	CTPOP*	CONL	INSB	SELEQ	SELLT	SELLE
	0x1	SUBW	LOG2X1	S4SUBL	CTLZ*					
	0x2	ADDL	LOG2X2	S8ADDL	CTTZ*					
	0x3	SUBL	LOG2X3	S8SUBL	保留					
	0x4	保留	LOG2X4	MULW	ZAP					
	0x5	保留	LOG2X5	UMULW	ZAPNOT					
	0x6	保留	LOG2X6	MULL	SEXTB					
	0x7	保留	LOG2X7	UMULH	SEXTH					
	0x8	SLL	LOG2X8	保留	保留					
	0x9	SRL	LOG2X9	CMPEQ	保留					
	0xa	SRA	LOG2XA	CMPLT	保留					
	0xb	ROTL	LOG2XB	CMPLE	保留					
	0xc	S32ADDL	LOG2XC	保留	保留					
	0xd	S32SUBL	LOG2XD	保留	保留					
	0xe	S64ADDL	LOG2XE	CMPULT	保留					
	0xf	S64SUBL	LOG2XF	CMPULE	保留					

注：带“*”号的指令不存在立即数格式。

9.6 向量整数指令-4’h5

向量整数指令的操作码 OPC[3:0]为 0x5，使用下列四种指令格式。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
简单运算（寄存器格式）	OPC				Ra						Rb						FUNC				SUBF				0	0	Rc					
简单运算（立即数格式）	OPC				Ra						#b[5:0]						FUNC				SUBF				#b[7:6]		Rc					
复合运算（寄存器格式）	OPC				Ra						Rb						FUNC				Rc						Rd					
复合运算（立即数格式）	OPC				Ra						Rb						FUNC				#c[5:0]						Rd					

FUNC[3:0]为 0x0~0x5 的指令使用简单运算指令格式（其中 4’h0、4’h2、4’h4 使用寄存器格式，4’h1、4’h3、4’h5 使用立即数格式。各指令根据功能码 FUNC[3:0]和子功能码 SUBF[3:0]区分功能，对应情况如下表所示：

	FUNC[3:0]					
	0x0	0x1	0x2	0x3	0x4	0x5

SUBF[3:0]	0x0	VADDW	VADDW(I)	VLOG2X0	VLOG2X0(I)	VCMPEQW	VCMPEQW(I)
	0x1	VSUBW	VSUBW(I)	VLOG2X1	VLOG2X1(I)	VCMPLTW	VCMPLTW(I)
	0x2	VMULW	VMULW(I)	VLOG2X2	VLOG2X2(I)	VCMPLEW	VCMPLEW(I)
	0x3	保留	保留	VLOG2X3	VLOG2X3(I)	保留	保留
	0x4	VADDH	VADDH(I)	VLOG2X4	VLOG2X4(I)	VCMPEQH	VCMPEQH(I)
	0x5	VSUBH	VSUBH(I)	VLOG2X5	VLOG2X5(I)	VCMPLTH	VCMPLTH(I)
	0x6	VMULH	VMULH(I)	VLOG2X6	VLOG2X6(I)	VCMPLEH	VCMPLEH(I)
	0x7	保留	保留	VLOG2X7	VLOG2X7(I)	保留	保留
	0x8	保留	保留	VLOG2X8	VLOG2X8(I)	保留	保留
	0x9	保留	保留	VLOG2X9	VLOG2X9(I)	保留	保留
	0xa	保留	保留	VLOG2XA	VLOG2XA(I)	保留	保留
	0xb	保留	保留	VLOG2XB	VLOG2XB(I)	保留	保留
	0xc	保留	保留	VLOG2XC	VLOG2XC(I)	保留	保留
	0xd	保留	保留	VLOG2XD	VLOG2XD(I)	保留	保留
	0xe	保留	保留	VLOG2XE	VLOG2XE(I)	保留	保留
	0xf	保留	保留	VLOG2XF	VLOG2XF(I)	保留	保留

FUNC[3:0]为 0x6~0xf 的指令使用复合运算指令寄存器格式。各指令根据功能码 FUNC[3:0]区分功能，对应情况如下表所示：

		FUNC[3:2]			
		0x0	0x1	0x2	0x3
FUNC[1:0]	0x0	已使用	已使用	VNMAH	VMAH
	0x1	已使用	已使用	VNMAW	VMAW
	0x2	已使用	VSELEQW	VSELLTW	VSELLEW
	0x3	已使用	VSELEQH	VSELLTH	VSELLEH

9.7 LOG3R 指令-4'h6

LOG3R 指令的操作码 OPC[3:0]为 0x6，采用复合运算寄存器指令格式，指令格式如下。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
复合运算（寄存器格式）	OPC				Ra						Rb						FUNC				Rc						Rd					

其中，FUNC[3:0]用作真值表寄存器索引，最多可支持 16 个真值表寄存器。

9.8 VLOG3R 指令-4'h7

VLOG3R 指令的操作码 OPC[3:0]为 0x7，采用复合运算寄存器指令格式，指令格式如下。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

复合运算（寄存器格式）	OPC	Ra	Rb	FUNC	Rc	Rd

其中，FUNC[3:0]用作真值表寄存器索引，最多可支持 16 个真值表寄存器。

9.9 标量浮点指令-4'h8

标量浮点指令的操作码 OPC[3:0]为 0x8，使用下列三种指令格式。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
简单运算（寄存器格式）	OPC				Ra						Rb						FUNC				SUBF				0	0	Rc					
简单运算（立即数格式）	OPC				Ra						#b[5:0]						FUNC				SUBF				#b[7:6]		Rc					
复合运算（寄存器格式）	OPC				Ra						Rb						FUNC				Rc								Rd			

根据功能码 FUNC[3]区分所使用的指令格式，具体如下：

FUNC[3:1]==3'b000：简单运算指令格式，FCVTDLR 使用立即数格式，其它指令使用寄存器格式；

FUNC[3:1]!=3'b000：复合运算寄存器指令格式

对于复合运算指令（乘加、选择），各指令根据功能码 FUNC[3:0]区分功能，对应情况如下表所示：

		FUNC[3:2]			
		0x0	**0x1**	**0x2**	**0x3**
FUNC[1:0]	**0x0**	**简单运算**	保留	FMAS	FNMAS
	0x1	**浮点指令**	FSELEQ	FMAD	FNMAD
	0x2	保留	FSELLT	FMSS	FNMSS
	0x3	保留	FSELLE	FMSD	FNMSD

对于简单运算指令（加、减、乘、除、平方根、比较、符号拷贝、转换、FPCR 访问），各指令根据功能码 FUNC[3:0]和子功能码 SUBF[3:0]区分功能，对应情况如下表所示：

		FUNC[3:0]	
		0x0	**0x1**
SUBF[3:0]	**0x0**	FADDS	FCVTDS
	0x1	FADDD	FCVTSD
	0x2	FSUBS	FCVTLS
	0x3	FSUBD	FCVTLD
	0x4	FMULS	FCVTDL
	0x5	FMULD	FCVTDLR
	0x6	保留	FCVTSH
	0x7	保留	FCVTHS
	0x8	FCPYS	FDIVS
	0x9	FCPYSE	FDIVD
	0xa	FCPYSN	FSQRTS

	0xb	保留	FSQRTD
	0xc	FCMPUN	RFPCR
	0xd	FCMPEQ	WFPCR
	0xe	FCMPLT	保留
	0xf	FCMPLE	保留

9.10 向量浮点指令、专用指令-4'h9

向量浮点指令和专用指令的操作码 OPC[3:0]为 0x9，使用下列两种指令格式。

	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
简单运算（寄存器格式）	OPC				Ra						Rb						FUNC				SUBF				0	0	Rc					
复合运算（寄存器格式）	OPC				Ra						Rb						FUNC				Rc						Rd					
复合运算（立即数格式）	OPC				Ra						Rb						FUNC				#c[5:0]						Rd					

根据功能码 FUNC[3]区分所使用的指令格式，具体如下：

FUNC[3:0]==4'h0 | 4'h4：简单运算寄存器指令格式

FUNC[3:0]!=4'h0 | 4'h4：复合运算指令格式

对于复合运算指令（乘加、选择），各指令根据功能码 FUNC[3:0]区分功能，其中 FUNC[3:0]为 4'h6 使用复合运算立即数格式，其余均使用复合运算寄存器格式。对应情况如下表所示：

		FUNC[3:2]			
		0x0	**0x1**	**0x2**	**0x3**
功能码[1:0]	**0x0**	**简单运算 1**	简单运算 2	VFMAS	VFNMAS
	0x1	VFSELEQS	保留	保留	保留
	0x2	VFSELLTS	XPLUTW	VFMSS	VFNMSS
	0x3	VFSELLES	PWRTW	保留	保留

对于简单运算指令（加、减、乘、比较、符号拷贝、转换），FUNC[3:0]固定为 4'h0，各指令根据功能码和子功能码 SUBF[3:0]区分功能，对应情况如下表所示：

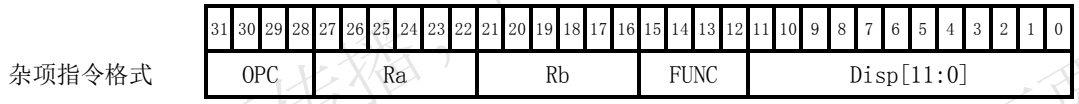
		FUNC[3:0]	
		0x0	**0x4**
SUBF[3:0]	**0x0**	VFADDS	VFCVTSH
	0x1	保留	VFCVTHS
	0x2	VFSUBS	保留
	0x3	保留	保留
	0x4	VFMULS	VFEXTE30S
	0x5	保留	VFDEPE30S
	0x6	保留	保留

	0x7	保留	保留
	0x8	VFCPYSS	保留
	0x9	VFCPYSES	保留
	0xa	VFCPYSNS	保留
	0xb	保留	保留
	0xc	VFCMPUNS	保留
	0xd	VFCMPEQS	保留
	0xe	VFCMPLTS	保留
	0xf	VFCMPLES	保留

9.11 小粒度访存指令、标/向量不对界访存指令-4’ha

小粒度存储指令的操作码 OPC[3:0]为 0xa。

小粒度访存指令（即访问粒度为字节、半字和 HP 浮点的访存指令）、标/向量不对界访存指令使用杂项指令格式：



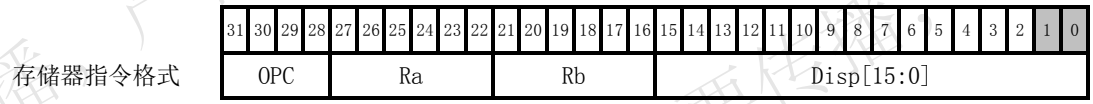
各指令根据功能码 FUNC[3:0]区分功能，对应情况如下表所示：

		FUNC[3]	
		0x0	**0x1**
FUNC[2:0]	**0x0**	VFLDH_U	VFSTH_U
	0x1	VLDB_U	VSTB_U
	0x2	VLDH_U	VSTH_U
	0x3	VLDW_U	VSTW_U
	0x4	FLDH	FSTH
	0x5	LDBU	STB
	0x6	LDHU	STH
	0x7	LDL_U	STL_U

LDL_U、STL_U、VFLDH_U、VFSTH_U、VLDB_U、VSTB_U、VLDH_U、VSTH_U、VLDW_U 和 VSTW_U 为标/向量不对界访存指令，其中 LDL_U、VFLDH_U、VLDB_U、VLDH_U、VLDW_U 直接把地址强制对界后取数(由软件自行通过标/向量拼接指令整理数据)，STL_U、VFSTH_U、VSTB_U、VSTH_U、VSTW_U 直接把地址强制对界后存储。

9.12 FLDS、FSTS、LDW、STW-4’hb

FLDS、FSTS、LDW、STW 的操作码 OPC[3:0]为 0xb，采用存储器指令格式，指令格式如下。



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPC				Ra						Rb						Disp[15:0]															

存储器指令格式

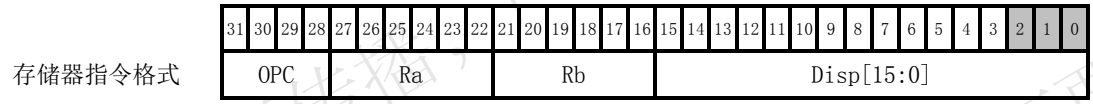
其中 Disp[1:0]位置用作功能码 FUNC[1:0]，Disp[15:2]为立即数字节偏移地址的[15:2]位。

各指令根据功能码 FUNC[1:0]区分功能，对应情况如下表所示：

		FUNC[1]	
		0x0	0x1
FUNC[0]	0x0	FLDS	LDW
	0x1	FSTS	STW

9.13 LDL、STL -4’hc

LDL、STL 的操作码 OPC[3:0]为 0xc，采用存储器指令格式，指令格式如下。



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPC				Ra						Rb						Disp[15:0]															

存储器指令格式

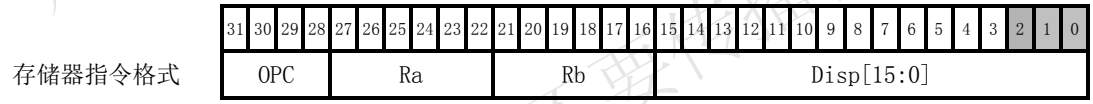
其中 Disp[2:0]位置用作功能码 FUNC[2:0]，各指令根据功能码 FUNC[2:0]区分功能，对应情况如下表所示：

		FUNC[2:1]			
		0x0	0x1	0x2	0x3
FUNC[0]	0x0	保留	保留	LDL	保留
	0x1	保留	保留	STL	保留

对于长字粒度访存指令 LDL、STL，Disp[15:3]为立即数字节偏移地址的[15:3]位。

9.14 向量对界访存指令-4’hd

向量对界访存指令的操作码 OPC[3:0]为 0xd，采用存储器指令格式，指令格式如下。



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPC				Ra						Rb						Disp[15:0]															

存储器指令格式

其中 Disp[2:0]位置用作功能码 FUNC[2:0]，Disp[15:3]为立即数字节偏移地址的[15:3]位。

各指令根据功能码 FUNC[2:0]区分功能，对应情况如下表所示：

	FUNC[2:1]			
	0x0	0x1	0x2	0x3

FUNC[0]	**0x0**	VFLDH	VLDW	VLDH	VLDB
	0x1	VFSTH	VSTW	VSTH	VSTB

对于向量对界访存指令 VLDW、VSTW，Disp[15:6]为立即数字节偏移地址的[15:6]位。

对于向量对界访存指令 VFLDH、VFSTH、VLDH、VSTH、VLDB、VSTB，Disp[15:5]为立即数字节偏移地址的[15:5]位。

9.15 LDI 指令-4'he

LDI 指令的操作码 OPC[3:0]为 0xe，采用存储器指令格式，指令格式如下。

存储器指令格式

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPC				Ra						Rb						Disp[15:0]															

9.16 LDIH 指令-4'hf

LDIH 指令的操作码 OPC[3:0]为 0xf，采用存储指令格式，指令格式如下。

存储器指令格式

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPC				Ra						Rb						Disp[15:0]															

10 附录 3 指令数量统计

SW-AI 处理器共包含指令 201 条。其中基本指令 121 条，包括控制类指令 5 条，转移指令 14 条，标量访存指令 20 条，整数运算类指令 39 条，标量整理指令 2 条，整数可重构逻辑指令 2 条，浮点运算类指令 38 条；扩展指令 70 条，包括 SIMD 扩展指令 70 条，通道类指令 4 条，专用指令 4 条，其他指令 2 条。

标量访存指令包括 3 类指令：对界访存指令 14 条，不对界访存指令 2 条，原子操作 4 条。

SIMD 扩展指令包括 6 类指令：对界 SIMD 访存指令 16 条、整数 SIMD 运算指令 22 条、整数 SIMD 可重构指令 2 条、浮点 SIMD 运算指令 19 条，SIMD 整理指令 11 条。

专用指令是指因为某些特殊用途而加入的指令，目前包括并行查表 1 类指令。

11 附录 4 向量寄存器中的高位

向量寄存器的宽度为 512 位，其中低 64 位就是通用寄存器，在非 SIMD 指令操作的结果写入目标寄存器时，这个寄存器对应的向量寄存器高 448 位内容保持原值。

12 附录 5 浮点条件判断与浮点比较的区别

SW-AI 处理器从核在实现浮点条件选择指令和浮点条件转移指令时，都是直接根据操作数的“符号位值”和“非符号位是否为全 0”来进行条件判断。对于 64 位扩展半精度(EHP64)浮点、64 位扩展单精度(ESP64)或者 64 位双精度浮点，“符号位”指 64 位数据的第[63]位，“非符号位”指 64 位数据的第[62:0]位。对于 32 位扩展半精度(EHP32)浮点或者基本单精度（32 位）浮点，“符号位”指 32 位数据的第[31]位，“非符号位”指 32 位数据的第[30:0]位。

符号位	非符号位	fblt fsellt vsellt	fbeq fseleq vseleq	fble fselle vselle	fbgt	fbge	fbne
0	全 0	假	真	真	假	真	假
0	非全 0	假	假	假	真	真	真
1	全 0	假	真	真	假	真	假
1	非全 0	真	假	真	假	假	真

在操作数是非数（NaN）时，条件判断的结果，和“操作数与 0”比较的结果可能会不一致。

符号位	非符号位	fblt fsellt vsellt	fbeq fseleq vseleq	fble fselle vselle	fbgt	fbge	fbne
0	非全 0	假	假	假	真	真	真
1	非全 0	真	假	真	假	假	真
符号位	非符号位	fcmplt0	fcmpeq0	fcmple0	>0	≥0	≠0
0	非全 0	假	假	假	假	假	假
1	非全 0	假	假	假	假	假	假

另外，通知的非数（SignalingNaN）做比较操作会报无效操作异常（INV），静止的非数（QuietNaN）做含有排序语义（<、≤、≥、>）的比较时会报无效操作异常（INV）；而非数作为浮点条件选择指令和浮点条件转移指令的判断条件时，不会报异常。

13 附录 6 立即数说明

SW-AI 从核指令集中存在大量使用立即数作为操作数的指令，现对立即数的数据类型作如下说明：

1） 访存指令、转移指令里面的地址偏移是有符号整数，在指令格式中表示为 disp；

2） LDI、LDIH 指令的立即数是有符号整数，在指令格式中表示为 disp；

3） 整数加减、比较运算类指令（含标量加减、标量比较、向量加减）中的立即数#b 都是 8 位无符号整数（表示范围为 0~255），零扩展到操作数数据宽度后参与计算；

4） 其它所有指令中的立即数都是无符号整数，在不同指令格式中可能表示为#b、#c 或 idx。

整数 SIMD 运算类指令中的立即数，需要同时参加多个分量的运算，具体如下：

1） 整数字向量和半字向量加减中的立即数#b，复制 16 份或 32 份，同时与 Va 中 16 个 32 位分量或 32 个 16 位分量进行计算；

两操作数 SIMD 可重构指令中的立即数#b，复制 16 份，同时与 Va 中 16 个 32 位分量进行计算。对立即数复制 16 份的原因，是通过两操作数 SIMD 可重构指令来支持整数字向量逻辑操作指令。

14 附录 7 图表目录

图目录：

图 0-1 从核指令格式 7
图 0-2 字节的格式 10
图 0-3 半字的格式 10
图 0-4 字的格式 11
图 0-5 长字的格式 11
图 0-6 IEEE 浮点数格式 11
图 0-7 半字整数向量格式 12
图 0-8 字整数向量格式 12
图 0-9 HP 浮点向量的存储器格式 13
图 0-10 HP 浮点向量的寄存器格式 13
图 0-11 SP 浮点向量的存储器格式 13
图 0-12 通用寄存器文件结构 14
图 0-13 向量寄存器 Vi 表示半字整数向量格式 15
图 0-14 向量寄存器 Vi 表示字整数向量格式 15
图 0-15 向量寄存器 Vi 表示 SP 浮点向量或 HP 浮点向量格式 15
图 0-16 小端地址寻址格式 16
图 0-17 SW-AI 处理器从核指令格式 21
图 0-18 简单运算指令格式 21
图 0-19 浮点复合运算指令格式 22
图 0-20 杂项指令格式 22
图 0-21 存储器指令格式 22
图 0-22 转移指令格式 23
图 0-23 DMA 命令格式 109
图 0-24 RMA 命令格式 111
图 0-25 LDM 全局地址和体内地址关系 116

表目录：

表 0-1 寄存器操作数表示符号 17
表 0-2 操作数访问类型表示符号 18
表 0-3 操作数数据类型表示符号 18
表 0-4 操作符一览表 19
表 0-5 各种浮点格式在存储器和寄存器中的存储格式 24
表 0-6 SW-AI 处理器表示的浮点数范围 26
表 0-7 SW-AI 处理器所表示的各种浮点数 26
表 0-8 IEEE 四种舍入方式对应的上溢和下溢条件 27
表 0-9 SW-AI 处理器浮点指令与 IEEE 舍入方法的对应关系 28
表 0-10 输入操作数中包含非规格化数时处理方式 30
表 0-11 结果为非规格化数时处理方式 30
表 0-12 浮点控制寄存器 FPCR 内容描述 31
表 0-13 控制指令列表 37

表 0-14 转移指令列表 39
表 0-15 对界访存指令列表(存储器指令格式) 42
表 0-16 对界访存指令列表(杂项指令格式) 42
表 0-17 不对界访存指令列表 46
表 0-18 原子操作指令列表 47
表 0-19 算术运算指令列表 50
表 0-20 移位指令列表 57
表 0-21 整数比较指令列表 58
表 0-22 条件选择指令列表 59
表 0-23 标量拼接指令列表 60
表 0-24 可重构指令列表 61
表 0-25 两操作数逻辑运算真值表 62
表 0-26 常见逻辑运算对应的可重构指令 62
表 0-27 三操作数逻辑运算真值表 63
表 0-28 单双精度浮点算术运算指令列表 64
表 0-29 比较指令列表 66
表 0-30 转换指令列表 67
表 0-31 FCVTDLR 指令立即数（舍入模式）含义 70
表 0-32 符号拷贝指令列表 70
表 0-33 浮点条件选择指令列表 71
表 0-34 乘加指令列表 72
表 0-35 读写浮点控制寄存器指令列表 74
表 0-36 对界 SIMD 访存指令列表 75
表 0-37 不对界 SIMD 访存指令列表 80
表 0-38 整数 SIMD 运算指令列表(简单运算指令格式) 86
表 0-39 整数 SIMD 运算指令列表（复合运算指令格式） 86
表 0-40 SIMD 可重构指令列表 92
表 0-41 常见向量逻辑运算指令对应的向量可重构指令 93
表 0-42 浮点 SIMD 运算指令列表（简单运算指令格式） 94
表 0-43 浮点 SIMD 运算指令列表（复合运算指令格式） 94
表 0-44 SIMD 整理指令列表 102
表 0-45 通道指令列表 108
表 0-46 DMA 命令格式详表 109
表 0-47 RMA 命令格式详表 111
表 0-48 脉动阵列计算加速引擎的操作种类和对应的描述符 113
表 0-49 并行查找表指令列表 116
表 0-50 从核内部寄存器访问指令列表 119
表 0-51 RCSR、WCSR 指令可访问的内部寄存器列表 121

15 附录 8 修改记录

日期	发布人	版本	内容	审核人
2019-03-04	过锋	V0.20	在 7A 从核结构手册基础上，整理完成 SW-AI 从核结构手册征求意见稿。 主要设计原则为： 1、坚持从核心为 64 位 RISC 架构 2、标量指令尽可能兼容已有指令集，并新增 HP 浮点格式相关指令，其中 HP 浮点在标量时均扩展为 64 位 HP 浮点。标量 SP 保持为扩展单精度格式浮点 3、向量指令支持标准 SP 向量、字整数向量和半字整数向量三类格式的指令，不支持扩展 HP 浮点格式运算，不再支持标准 DP 浮点向量和扩展 SP 浮点向量 4、专用指令主要用于 AI 中激活查表 5、删除 7A 面向 Linpack 的专用加速指令和特殊用途加速指令 6、删除数 0 类指令、向量不对界访存指令、向量移位指令 7、整数乘加指令只支持向量，不支持标量 8、浮点指令均带有“F”前缀，具体数据格式再根据后缀“D”、“S”或“H”确定是双精度浮点、单精度浮点或半精度浮点 9、指令修订的详细描述后面再补加 指令修订的详细描述如下： **1、删除以下指令** 1）标量部分 分类指令：CLASS_ST、CLASS_RD、CLASS_WR 转移预测提示指令：PWS 带标记的加减指令：ADDW_C、SUBW_C、ADDL_C、SUBL_C 数 0 指令：CTPOP、CTLZ、CTTZ 2）向量部分 向量对界访存指令：VLDS、VLDD、VSTS、VSTD	

			向量不对界访存：VLDS_U、VLDW_U、VSTS_U、VSTW_U 长字向量运算指令：VADDL、VSUBL 向量整数移位指令：VSLLW、VSRLW、VSRAW、VROTLW DP 浮点向量指令：VMAD、VMSD、VNMAD、VNMSD、VADDD、VSUBD、VMULD、 并行查表：PLUTW、AES 特殊用途加速指令：行向量扩展乘加、多精度运算、不进位乘、混洗、抽分位集中 **2、修改以下指令** 1）标量部分 SYNC 和 SYNP：调整从核行号，由原来的 3 位调整为 2 位 2）向量部分 向量浮点运算指令：向量比较、符号拷贝和条件选择指令后缀增加符号“S”，均为 SP 浮点向量运算 向量整理指令：调整操作码 3）其它 DMA 指令：增加矩阵转置功能 **3、新增以下指令** 1）标量部分 访存指令：FLDH、FSTH 整数算术运算指令：S32ADDL、S32SUBL、S64ADDL、S64SUBL 标量整理指令：INSB 浮点转换指令：FCVTHD、FCVTDH、FCVTLH 2）向量部分 访存指令：VFLDH、VFSTH、VLDW、VSTW、VLDH、VSTH、VLDB、VSTB 整数运算类指令：字整数向量乘/乘加/比较/条件选择、半字整数向量加/减/乘/乘加/比较/条件选择 向量整理指令：单精度浮点向量的插入/抽取/拷贝 3）其它 发起异步计算处理：ACE 并行激活值查表：AI_VATLS	
2019-03-05	过锋 吴铁彬	V0.21	1、修订“附录 5 浮点条件判断与浮点比较的区别” 2、调整专用指令：删除 AI_VATLS，恢复为原向量查表指令，并扩展向量查表 PLUTW 为 XPLUTW 指令，增	

			加立即数域指示表类型 3、调整 VMAW 指令操作码：为了兼容原并行查表指令的编码，将 VMAW 指令操作码从 0x5.8 调整为 0x5.d 4、恢复数 0 类指令：根据与软件基础语言编译器交流，考虑到库函数会使用数 0 类指令，恢复之 5、增加向量浮点转换指令 VFCVTSH，用于向量浮点运算完后，按 HP 格式写回存储器前判断是否存在转换异常 6、调整浮点向量指令名称：所有浮点向量运算指令，以及向量整理中的浮点相关指令，均带 VF*标记，表示向量（V）指令、浮点（F）运算，以与浮点标量指令对应，提高指令名称的可读性	
2019-03-07	过锋	V0.22	1、修订调整 ACE 描述符中部分内容的描述	
2019-03-11	过锋	V0.23	1、添加 XPLUTW 指令中#type 位宽的说明	
2019-03-12	过锋	V0.24	1、修改 6.7.1、6.7.3 和 6.7.6 节中操作描述，原描述有问题：将单精度标量按 32 位来运算。新的操作描述按 64 位扩展单精度运算进行 2、修改 6.7.3 浮点转换指令中对非规格化数的描述。转换指令中所有的输入，不论是 SP 还是 HP，如果是 DNO，则当+0 处理 3、修正 6.8.1.1.6 和 6.8.1.1.8 节中指令描述，存储到存储器中的内容都是每个分量的低位部分内容 4、修正 6.8.5.2.1 指令描述中循环次数，原来写的是循环 32 次，实际循环 16 次 5、修正一些章节标题中指令计数不准确的情况	
2019-03-13	过锋	V0.25	1、修正 6.8.1.1.2、6.8.1.1.3 和 6.8.1.1.4 节指令助记符和指令名称对不上的情况 2、修正 9.5 节中 insb 指令描述，由寄存器格式改为立即数格式	
2019-03-15	过锋	V0.26	1、修改 6.9.4 节，调整结果写回描述符第[63]位累加结果写回的精度的含义 2、修改 6.9.4 节，调整西侧数据加载描述符中第[23]位的含义	
2019-03-16	过锋	V0.27	1、删除 6.4.1.9 字乘加指令，由于该指令需要 3 个源操作数，目前指令编码很难放下，故暂不支持标量的字乘加指令。	

			2、修改 6.7.3.1、6.8.4.1.3 和 6.8.4.6.2 格式和助记符不一致节，调整助记符与指令描述不一致的地方 3、调整浮点指令描述中对输入为 DNO 时的处理说明。原描述为“当输入为 DNO 时，输入均处理为+0”，统一改为“当输入为 DNO 时，输入均处理为同符号 0” 4、修改 6.8.2.2 和 6.8.2.3 节，整数向量比较和条件选择指令中的分量，在指令描述中明确说明为有符号整数 5、修改 5.1.3 节，明确控制位 DNOE、UNDZ 与 EXUN 在不同精度指令下的配置情况 6、6.7.3 节删除 fcvthd 和 fcvtdh 指令，调整为 fcvths 和 fcvtsh；删除 fcvtlh，如果要使用此功能，可以用 fcvtls 和 fcvtsh 替代。6.8.4.4 节增加浮点向量半精度转单精度 vfcvths 指令 7、调整标量整数乘 32 加/减、标量整数乘 64 加/减指令的编码，以保持与-9 的兼容	
2019-03-18	过锋	V0.28	1、修订 9.6 节，明确 func=0x5 时采用简单运算立即数格式	
2019-03-20	过锋	V0.29	1、修订 6.9.4 节，西向数据加载中累加器缓冲的起始写入地址描述中由 8 位改为 7 位	
2019-03-21	过锋	V0.30	1、修订 5.1.3 节，明确控制位 DNOE、UNDZ 与 EXUN 仅对标量双精度浮点指令（除浮点转换指令）有效	
2019-03-26	过锋	V0.31	1、修订 6.9.4 异步计算描述符，北侧数据加载增加掩码方式 2、调整并行查表指令，将 XPLUTW 指令的编码从 0x5.8 调整为 0x9.6，PWRTW 编码从 0x5.9 调整为 0x9.7 3、增加整数负乘加指令，包括整数字向量负乘加 VNMAW 和整数半字向量负乘加 VNMAH	
2019-03-28	过锋	V0.32	1、修订 1.2 节描述，原描述未说明 32 个 16 位半字的数据存放格式 2、修改 6.9.4 异步计算描述符中“配置脉动阵列常用的参数”的内容，调整 8*8+16 整数和 16*16+32 整数的配置值	
2019-04-01	过锋	V0.33	1、增加向量不对界访存指令 VFLDH_U、VFSTH_U、VLDB_U、VSTB_U、VLDH_U、VSTH_U、VLDW_U 和 VSTW_U 2、修改 6.9.4 节异步计算描述符，增加“累加结果写回	

			栏栅”请求	
2019-04-02	过锋	V0.34	1、修改 9.6 节说明，0x6~0xf 指令均使用复合运算寄存器格式，原描述中写的 0x8 使用复合运算立即数格式是有问题的	
2019-04-03	过锋	V0.35	1、修订第 8 章，根据第 6 章描述的指令，修订目前指令数为 240 条 2、修改 6.8.1.2.7 节，修改操作描述和助记符的描述，原来的描述有问题，描述成为了 VFSTH_U，应该是 VSTH_U	
2019-04-09	过锋	V0.36	1、修订 6.9.4 节异步计算描述符，为“加载西向数据”描述符增加[59]位，描述西向数据中高、低 32B 参与运算的选择	
2019-04-16	过锋	V0.37	1、修订 6.8.5.3 节抽取操作的描述中，应该是从向量 Vb 中抽取标量，而非从 Va 中抽取（Va 在指令中无意义）	
2019-04-19	孙红辉	V0.38	1、修订 6.10.1 节对并行查找（造）表指令概述部分，将涉及 AES 指令的部分删除，因为 AI 指令集中没有 AES 指令 2、修订 6.8.1.2.8 标题为“存储不对界字向量/存储不对界 SP 浮点向量”，原标题为“存储不对界字向量/存储不对界 DP 浮点向量” 3、修订 6.8.1.1 节表格中第 4 项，应将“装入字向量或 SP 浮点向量”改为“存储字向量或 SP 浮点向量” 4、 修订附录 1 中序号 30，vconw 指令说明应将“字向量混洗”改为“字向量或 SP 浮点向量拼接” 5、修订附录 1 中序号 31，vconh 指令说明应将“字向量拼接”改为“半字向量拼接”	
2019-04-24	过锋	V0.39	1、修订 6.8.1.2.3 节和 6.8.1.2.5 节，指令描述中由“地址低 6 位清 0”改为“地址低 5 位清 0”	
2019-04-28	过锋	V0.40	1、修订 6.9.1 节，明确[59]位转置标志仅在 dma_get（即[58:56]=0x1 或 0x3）时有效	
2019-04-30	过锋	V0.50	1、修订 3.3 节和 6.2 节，从核目前支持全局 PC 访问，删除短 PC 的描述和短 PC 溢出异常 2、修订 6.2.3 节和 9.2 节，BR 指令和 9A 兼容，调整指令编码，将 disp 由<21:0>扩展到<22:0> 由征求意见稿升级成为正式版	

2019-05-07	过锋	V0.51	1、修订 6.8.5.1 节，向量拼接指令描述中增加对配套使用指令的说明	
2019-05-08	施晶晶	V0.52	1、修订 6.9,1 节,DMA_GET 在 AI 中支持行多播	
2019-05-17	施晶晶	V0.53	修订 6.9.1 和 6.9.2 传输量相关描述，传数量长度不允许为 0	
2019-07-29	过锋 陈正博	V0.54	修订 6.10.1.1，完善扩展的并行字查表指令描述	
2019-10-23	过锋 孙红辉	V0.55	1、修订 6.10.1.1，完善扩展的并行字查表指令描述 2、修改 6.8.5.1 向量拼接指令的描述，明确最后一句中“Va 的高段”和“Vb 的低段”是“Va 存放的高段”和“Vb 存放的低段”	
2019-11-08	过锋	V0.56	1、修订 6.8.4.4.2，浮点向量半精度转单精度的助记符应该为：VFCVTHS（原先笔误写成为 VFCVTSH） 2、修订 6.8.4.4，VFCVTHS 和 VFCVTSH 由原先的使用浮点复合运算指令格式改为简单运算指令格式，同时修订 9.10 中相关的指令格式描述 3、增加固定并行抽分位和并行置位 2 条专用指令，并在 9.10 中增加相应指令编码	